



tina4-js — The 1.5KB Reactive Core

Signals, Web Components, and a frontend you can fit in
your head

v3.12.4 • tina4.com

The Intelligent Native Application 4ramework

Table of Contents

Getting Started 17

Your First 5 Minutes	17
1. What Is tina4-js	17
What It Is Not	17
2. Prerequisites	17
3. Create a Project	18
4. Project Structure	18
The @/ Import Alias	19
5. The Entry Point	20
6. Your First Signal	20
7. Your First Route	21
8. Adding a Component	22
9. The Build	22
10. What Just Happened	23
Summary	24

Signals 25

Reactive State Without the Drama	25
1. What Is a Signal	25
The Three Forms: Read This Before Anything Else	25
2. Why Object.is() Matters	26
3. computed() -- Derived State	27
Computed Is Eager, Not Lazy	27
Computed Is Read-Only	27
4. effect() -- Side Effects	27

Disposing Effects	28
Auto-Tracking	28
Re-subscription on Re-run	28
5. batch() -- Grouping Updates	29
When Do You Need batch()?	29
6. peek() -- Read Without Tracking	30
7. Sharing Signals Across Files -- The Store Pattern	30
One file, all your app-wide signals	31
Import and read in any component	31
Import and write from a different component	31
Why it works	31
The rules	32
8. Debug Labels	32
9. isSignal() -- Type Check	32
10. Common Mistakes	33
Mistake 1: Reading .value in Templates	33
Mistake 2: Mutating in Place	33
Mistake 3: Creating Effects in Loops	33
Mistake 4: Forgetting That false Renders as Text	33
11. Putting It Together -- A Todo List	34
Persisting Signals Across Refreshes	35
Summary	36
Persistent Signals	37
The Dangers List, Up Front	37
Options	38
Cross-tab Sync	38
Wipe on Logout	38

Version Migration	38
Safety Guarantees	39
Bundle Cost	39
HTML Templates	40
DOM Without the Framework Tax	40
1. What html`` Does	40
2. Static Values: \${value}	40
3. Reactive Text: \${signal}	41
4. Reactive Blocks: \${() => expr}	41
Conditional Rendering	42
Lists	42
Nested Reactivity	42
5. DocumentFragments and Arrays	42
6. Event Handlers -- @event	43
Auto-Batching	43
7. Boolean Attributes -- ?attr	43
8. Property Bindings -- .prop	44
CRITICAL: Never Put Inputs Inside Reactive Blocks	45
9. Dynamic Attributes: attr=\${value}	46
Reactive Classes	46
10. The false/null/undefined Trap	46
11. Template Caching	47
12. Putting It All Together	47
Summary	49
Components	50
Web Components That Don't Suck	50

1. What Are Tina4 Components	50
2. Your First Component	51
3. static props -- Declaring Reactive Props	51
Type Coercion	52
4. this.prop(name) -- Reading Props	52
5. static styles -- Scoped CSS	53
The :host Selector	53
6. static shadow -- Light DOM vs Shadow DOM	54
7. this.emit() -- Custom Events	54
Passing Data with Events	55
8. Lifecycle Hooks	55
onMount()	55
onUnmount()	56
9. Composing Components	56
10. The Store Pattern -- Shared Signals	57
11. Complete Example -- A Card Component	59
12. Wildcard Imports -- Registering Every Component	60
Summary	61
Routing	62
Navigation Without the Router Library	62
1. How Routing Works	62
2. Registering Routes	62
3. Route Parameters	63
How Matching Works	63
4. The Wildcard -- 404 Routes	63
5. Route Guards	63

Admin Guard Pattern	64
6. Starting the Router	65
target	65
mode	65
7. Programmatic Navigation	66
Link Interception	66
8. Route Change Events	66
9. Async Routes	67
Loading States	67
10. Effect Cleanup on Route Change	68
11. Complete Example -- Multi-Page App	69
12. Wildcard Imports -- Loading Every Route File	70
Keep the catch-all out of the glob	71
Summary	72

API 73

Talking to Your Backend	73
1. The API Client	73
2. Configuration	73
Options	74
3. Making Requests	74
GET	74
POST	75
PUT (Full Replace)	75
PATCH (Partial Update)	75
DELETE	75
4. RequestOptions	75
Per-Request Headers	75

Query Params on Any Method	75
5. Auth Flow	76
1. Bearer Token	76
2. formToken	76
3. FreshToken Rotation	76
Login Example	76
6. Error Handling	77
Pattern: Centralized Error Handling with Interceptors	77
7. Interceptors	77
Request Interceptor	77
Response Interceptor	78
8. Real Example: CRUD Data Table	78
9. File Upload	80
Summary	81
WebSocket	82
Real-Time Without the Headache	82
1. The WebSocket Client	82
2. Connecting	82
With Options	82
Options Reference	83
3. Reactive Signals	83
Status Flow	83
4. Sending Messages	84
5. Listening for Events	84
Message Parsing	85
6. pipe() -- Stream Messages Into Signals	85

Multiple Pipes	85
Unsubscribing	86
7. Auto-Reconnect	86
Disabling Reconnect	87
Limiting Attempts	87
8. Closing the Connection	87
9. Real Example: Chat Room	87
10. Real Example: Live Dashboard	88
Summary	90
SSE / NDJSON Streaming	91
Streaming Without WebSocket	91
1. The SSE Client	91
2. EventSource Mode (Default)	91
3. Fetch Mode (NDJSON)	92
4. Options	92
5. Reactive Signals	93
6. Event Handlers	93
7. Named Events (EventSource Mode)	94
8. Pipe to Signal	94
9. Auto-Reconnect	95
10. Closing	95
11. SSE vs WebSocket	95
12. Real-World Example: AI Chat Streaming	97
13. Real-World Example: Live Notification Feed	98
Bundle Size	98
Summary	99

GraphQL	100
One Method, Two Worlds	100
1. Your First Query	100
2. Variables	100
3. Mutations	101
Create	101
Update	102
Delete	102
4. Authentication	102
5. Error Handling	102
GraphQL Errors (query-level)	103
Network Errors (transport-level)	103
6. Custom Headers	103
7. Tina4 Backend Integration	104
8. TypeScript	104
9. Reactive Queries with Signals	105
Summary	105
PWA	107
Make It Installable	107
1. What pwa.register() Does	107
2. Configuration	107
Required	108
Optional	108
3. Cache Strategies	108
network-first (default)	108
cache-first	109

stale-while-revalidate	109
4. Precaching	110
5. Offline Route	110
6. The Generated Manifest	111
Build-Time Generation	111
7. Complete Example	112
8. Tips	112
Summary	113
Debug Overlay	114
See Everything	114
1. Enabling the Debug Overlay	114
2. Opening the Overlay	114
3. The Signals Panel	114
What to Look For	115
4. The Components Panel	115
What to Look For	115
5. The Routes Panel	116
What to Look For	116
6. The API Panel	116
What to Look For	116
7. How It Works Internally	117
8. Best Practices	117
Always Use Debug Labels	117
Import Conditionally	117
Use During Development, Not Just Debugging	117
Summary	118

tina4-css	119
Optional Styling	119
1. What Is tina4-css	119
2. Installation	119
With the CLI	119
Manual Installation	119
3. The Reset	120
4. Grid System	120
5. Buttons	120
6. Forms	121
7. Tables	122
8. Cards	123
9. Badges and Alerts	124
Badges	124
Alerts	124
10. Navigation	124
11. Modals	125
12. Dark Theme	126
13. Using with Shadow DOM Components	126
Option 1: Light DOM	126
Option 2: Import CSS into Shadow DOM	126
Summary	127
Backend Integration	128
The Full Stack	128
1. The Tina4 Stack	128
2. tina4-js + tina4-php	128

Setup	128
tina4-php Routes	129
tina4-js Frontend	129
3. tina4-js + tina4-python	130
Setup	130
tina4-python Routes	130
4. Authentication End-to-End	130
Step 1: Login	130
Step 2: Authenticated Requests	131
Step 3: Token Rotation	131
Step 4: Route Guards	131
Step 5: 401 Handling	132
5. Building for Backend Embedding	132
Build	132
Deploy to Backend	132
The CLI Build Command	132
Backend Template	133
6. Islands Architecture	133
How It Works	133
Example: Adding a Live Search to a Server-Rendered Page	133
Multiple Islands	134
Shared State Between Islands	135
7. Development Workflow	135
Separate Frontend and Backend	135
Embedded Frontend	135
Summary	136

The Admin Dashboard	137
1. Project Setup	137
2. Global Store	137
3. App Entry	139
4. Routes	140
5. Layout Component	141
6. Login Page	142
7. Stat Card Component	144
8. Notification Bell Component	145
9. Dashboard Page with Live Stats	147
10. CRUD Users Page	150
11. What We Built	154
Summary	155
Patterns and Pitfalls	156
What We Learned the Hard Way	156
1. When to Use tina4-js	156
2. The New-Reference Rule	157
Helper Functions	157
3. Computed Is Eager, Not Lazy	158
4. Inputs Must Stay Outside Reactive Blocks	158
5. Event Handler Auto-Batching	159
6. Effect Cleanup on Route Navigation	159
7. Do Not Mix addEventListener Inside Reactive Blocks	160
8. The false/null Rendering Trap	161
9. Signal Scope and Lifetime	161
Avoid Global Signals for Temporary State	162

Use Global Signals for Shared State	162
10. Debugging Techniques	162
Add Labels to Every Signal	162
Check Subscriber Counts	162
Use effect() for Debugging	163
11. Performance Patterns	163
Keep Lists Reasonable	163
Avoid Unnecessary Computed	163
Batch Async Results	163
12. Common Error Messages	164
"[tina4] computed signals are read-only"	164
"[tina4] WebSocket is not connected"	164
"[tina4] Router target '...' not found in DOM"	164
"[tina4] Prop '...' not declared in static props"	164
13. Reactive <select>: Bind ?selected, Not .value	164
Summary	165

Vibe Coding with AI 166

Let AI Write Your tina4-js	166
1. Why tina4-js Works Well with AI	166
2. CLAUDE.md -- The AI's Instruction Manual	166
3. llms.txt -- AI Context for the Framework	167
4. The tina4-js Skill	167
Rule 1: Static vs Reactive	167
Rule 2: New References	168
Rule 3: Boolean Attributes	168
5. What AI Gets Wrong	168
Using .value in Templates	168

The && Pattern	168
Missing @event Syntax	169
Forgetting Reactive Blocks for Conditionals	169
Using route() with Handler First	169
6. Working with Claude Code on tina4-js Projects	169
Be Specific About Modules	169
Reference Existing Patterns	170
Let It Run the Dev Server	170
Review Generated Templates	170
7. The AI Advantage	170
8. Setting Up Your Project for AI	171
Summary	171
Review the AI's work with the Tina4 Code Viewer	171
Internationalization	172
One App, Every Language	172
Create an instance	172
Translate	172
Switch the locale, and everything updates	173
Format numbers, currency, and dates	173
Right to left	173
Load bundles at runtime	174
The default instance	174
The full API	174
Real-time (WebRTC)	175
Peer-to-Peer Without a Media Server	175
1. The rtc Module	175

2. Bootstrapping: GET /api/rtc/config	176
RtcConfig	176
Fetch once, reuse everywhere	176
3. Calls: rtc.call(room, options?)	177
CallOptions	177
CallSession	177
The status signal	178
Binding streams into the DOM	178
Controls	178
Receive-only viewer	178
Leaving	178
4. Chat: rtc.chat(channel, options?)	179
ChatOptions	179
ChatSession	180
A minimal chat panel	180
Sending, typing, and read receipts	180
Paging older messages	181
Teardown	181
5. Files: rtc.upload and rtc.fetchBlob	181
6. Real Example: A Call + Chat Room	182
7. Hard Rules	183
8. Backend Pairing	184
Summary	185

Getting Started

Your First 5 Minutes

A terminal. A single command. A browser tab. In five minutes you will have a running tina4-js project with reactive state, a rendered page, and a dev server with hot reload. The counter on screen will respond to your clicks before you understand why. That is the point -- you ship first, then you learn.

1. What Is tina4-js

tina4-js is a reactive JavaScript framework with a 1.5KB gzipped core. The full framework ships under 6KB gzipped. Eight modules, each solving one problem:

- **Signals** for reactive state -- no stores, no reducers, no actions
- **Tagged template literals** for DOM rendering -- no JSX, no virtual DOM, no compiler
- **Web Components** for encapsulation -- real custom elements, not a framework abstraction
- **A router** for SPA navigation
- **An HTTP client** built for tina4-php and tina4-python backends
- **WebSocket** with auto-reconnect and signal integration
- **PWA** support with one function call
- **A debug overlay** that shows you everything

One npm package. Zero dependencies. The entire framework weighs less than most favicons.

What It Is Not

tina4-js is not React. It has no virtual DOM. It does not diff trees. When a signal changes, the exact text node or attribute that depends on it updates. Nothing else moves.

tina4-js is not Angular. No dependency injection. No decorators. No module system. No zone.js.

tina4-js is not a meta-framework. No server-side rendering. No file-based routing. No data loading conventions. It is a client-side framework that does one thing well: build reactive UIs with the smallest possible footprint.

2. Prerequisites

You need three things. Nothing else.

- **Node.js 18 or later** -- check with:

```
node --version
```

You should see [v18.0.0](#) or higher.

The Intelligent Native Application Framework

- **npm** -- comes with Node.js. Check with:

```
npm --version
```

- **A text editor** -- VS Code, Cursor, Zed, Vim, anything.

3. Create a Project

The primary way to scaffold is the Tina4 Rust CLI:

```
tina4 init js my-app
```

This scaffolds a complete project with TypeScript, Vite, routing, and a sample page.

*The **tina4 Rust CLI** is the unified installer across all Tina4 frameworks (Python, PHP, Ruby, Node.js, JS). For npm-only environments where the CLI isn't available, every command has an **npx tina4js ... fallback** -- for example **npx tina4js create my-app** is equivalent to **tina4 init js my-app**.*

Want the optional CSS framework included? The Rust CLI does not yet accept a **--css** flag on **init**, so scaffold first and then add the dependency:

```
tina4 init js my-app
```

To add Tina4 CSS, edit **package.json** to add the **tina4-css** dependency, or use the fallback **npx tina4js create my-app --css** which accepts the flag directly. This adds **tina4-css** to your dependencies -- a utility CSS library with reset, grid, buttons, forms, tables, cards, and dark mode built in. More on this in Chapter 12.

Want PWA support from the start? The Rust CLI also does not yet expose a **--pwa** flag on **init**. Scaffold with **tina4 init js my-app** and enable PWA manually (see Chapter 10), or use the fallback **npx tina4js create my-app --pwa** to get the PWA preset directly. You can combine the flags on the fallback: **npx tina4js create my-app --css --pwa**.

Now install and run:

```
cd my-app
npm install
npm run dev
```

Open **http://localhost:3000**. A welcome page appears with a counter. Click the minus button. Click the plus button. The number updates. No page reload. No visible delay. That is signals at work -- reactive state flowing from data to DOM without you writing a single line of update logic.

4. Project Structure

Look at what the CLI created:

```
my-app/
  index.html          # Entry point -- loads src/main.ts
  package.json        # Dependencies: tina4js, vite, typescript
```

The Intelligent Native Application 4ramework

```

tsconfig.json          # TypeScript config
vite.config.ts         # Vite dev server config
src/
  main.ts              # App entry -- imports routes, starts router
  routes/
    index.ts           # Route definitions
  pages/
    home.ts            # Home page handler
  components/
    app-header.ts      # Example web component
  public/
    css/
      default.css      # Default styles

```

The structure is intentional:

- `src/routes/` -- Route definitions. One file per feature or group.
- `src/pages/` -- Page handler functions. Each returns a template.
- `src/components/` -- Web Components (Tina4Element subclasses).
- `src/public/` -- Static assets (CSS, images, fonts).

This is a convention, not a requirement. tina4-js does not care where your files live. But this structure scales. Every tina4-js project looks the same, which matters when you onboard new team members or let AI generate code. Consistency compounds.

The @/ Import Alias

The scaffold wires up an `@` alias pointing at `src/`, so you import by absolute path instead of counting `../` hops:

```

// Without the alias - brittle, breaks when you move the file
import { homePage } from '../pages/home';
import { user } from '../../store';

// With the alias - always anchored at src/, move files freely
import { homePage } from '@pages/home';
import { user } from '@store';

```

It is configured in two places, and **both must agree**. TypeScript uses `tsconfig.json` for editor go-to-definition and type-checking; Vite uses `vite.config.ts` for the actual build:

```

// tsconfig.json
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": { "@/*": ["src/*"] }
  }
}

// vite.config.ts
import { resolve } from 'path';

export default defineConfig({
  resolve: {
    alias: { '@': resolve(__dirname, 'src') },
  },
});

```

`tina4 init js` writes both for you, so new projects get `@/` imports out of the box. If you scaffolded before this was the default, add the two blocks above and you're set.

5. The Entry Point

Open `src/main.ts`:

```
import { signal, computed, html, route, router, navigate, api } from 'tina4js';
import './routes/index';

// Configure API (uncomment to connect to tina4-php/python backend)
// api.configure({ baseUrl: '/api', auth: true });

// Start router
router.start({ target: '#root', mode: 'hash' });
```

Three things happen here:

- **Import everything from one package.** All seven modules export from `tina4js`. No sub-package installs.
- **Import your routes.** The route file calls `route()` to register paths and handlers.
- **Start the router.** It finds `#root` in the DOM and renders matched routes into it.

The `mode: 'hash'` means URLs look like `http://localhost:3000/#/about`. For clean URLs without the hash, use `mode: 'history'` -- but you will need server-side URL rewriting in production.

6. Your First Signal

Open `src/pages/home.ts`:

```
import { signal, computed, html } from 'tina4js';

export function homePage() {
  const count = signal(0);
  const doubled = computed(() => count.value * 2);

  return html`
    <div class="page">
      <h1>Welcome</h1>

      <div class="counter">
        <button @click=${() => count.value--}>-</button>
        <span>${count}</span>
        <button @click=${() => count.value++}>+</button>
      </div>
      <p class="muted">Doubled: ${doubled}</p>
    </div>
  `;
}
```

Four things happen in this code. Each one is a core concept you will use in every tina4-js application.

`signal(0)` creates a reactive value. Read it with `count.value`. Write it with `count.value = 5`. When you write, everything that depends on it updates. Not eventually. Not on the next tick. Right now.

`computed(() => count.value * 2)` creates a derived signal. It reads `count.value` inside the function, so tina4-js knows to recompute whenever `count` changes. You cannot write to a computed -- it is read-only.

`**html\...\`**` is a tagged template literal. It returns a real DocumentFragment -- actual DOM nodes, not a string, not a virtual tree. When you put `${count}` in the template, tina4-js creates a text node that updates in place when `count` changes. No diffing. No reconciliation. Direct DOM mutation.

`@click=${() => count.value--}` adds a click event listener. The `@` prefix means "event handler." Since v1.0.9, event handlers are wrapped in `batch()`, so multiple signal writes inside one handler trigger one update.

7. Your First Route

Open `src/routes/index.ts`:

```
import { route, html } from 'tina4js';
import { homePage } from '../pages/home';

// Home
route('/', homePage);

// About
route('/about', () => html`
  <div class="page">
    <h1>About</h1>
    <p>Built with tina4-js.</p>
    <a href="/">Back home</a>
  </div>
`);

// 404
route('*', () => html`
  <div class="page">
    <h1>404</h1>
    <p>Page not found.</p>
    <a href="/">Go home</a>
  </div>
`);
```

`route(pattern, handler)` -- pattern is always the first argument. The handler is a function that returns a template. The router calls it when the URL matches.

`route('*', handler)` -- the wildcard catches any URL that no other route matched. Put it last.

Links work automatically. The router intercepts clicks on `<a>` tags that point to same-origin paths and navigates without a page reload. You do not need a special `<Link>` component.

8. Adding a Component

Open `src/components/app-header.ts`:

```
import { Tina4Element, html } from 'tina4js';

class AppHeader extends Tina4Element {
  static props = { title: String };
  static styles = `
    :host { display: block; padding: 1rem 0; border-bottom: 1px solid #e5e7eb; }
    h1 { margin: 0; font-size: 1.5rem; }
    nav { display: flex; gap: 1rem; margin-top: 0.5rem; }
    a { color: #2563eb; text-decoration: none; }
  `;

  render() {
    return html`
      <h1>${this.prop('title')}</h1>
      <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
      </nav>
    `;
  }
}

customElements.define('app-header', AppHeader);
```

Then use it in any template:

```
html`
  <app-header title="My App"></app-header>
  <div class="content">...</div>
`
```

`static props` declares reactive attributes. When the `title` attribute changes, the component re-renders that part.

`static styles` scopes CSS to the component via Shadow DOM. Your styles cannot leak out. External styles cannot leak in.

`this.prop('title')` returns a signal for the `title` prop. Drop it in the template and it updates reactively.

`customElements.define()` registers the tag name. Use a hyphenated name -- that is a Web Components requirement, not a tina4-js thing.

9. The Build

Development is done. Time to ship:_____

The Intelligent Native Application 4ramework

```
npm run build
```

Vite bundles everything into `dist/`. The tina4-js runtime adds under 6KB to your bundle. Your entire app -- framework, routes, components, everything -- will likely be smaller than React's runtime alone.

To preview the production build:

```
npm run preview
```

10. What Just Happened

Five minutes. One command to scaffold. One command to install. One command to run. And you covered:

- Reactive state with `signal()`
- Derived state with `computed()`
- DOM rendering with `html` tagged templates
- Event handling with `@click`
- A web component with `Tina4Element`
- Client-side routing with `route()` and `router.start()`
- A production build with Vite

The rest of this book goes deep on each of these. But you already have a working app. You already have a production build. Everything from here is precision and power.

Summary

What	How
Create project	<code>tina4 init js my-app</code> (fallback: <code>npx tina4js create my-app</code>)
With CSS framework	<code>tina4 init js my-app + add tina4-css dep</code> (fallback: <code>npx tina4js create my-app --css</code>)
With PWA	<code>tina4 init js my-app</code> + enable PWA manually (fallback: <code>npx tina4js create my-app --pwa</code>)
Dev server	<code>npm run dev</code>
Production build	<code>npm run build</code>
Reactive state	<code>signal(initialValue)</code>
Derived state	<code>computed(() => expression)</code>
DOM rendering	<code>`html<p>\${signal}</p>`</code>
Event handling	<code>@click=\${handler}</code>
Components	<code>class X extends Tina4Element</code>
Routes	<code>route(pattern, handler)</code>
Start router	<code>router.start({ target, mode })</code>

Signals

Reactive State Without the Drama

A variable changes. The DOM does not. You write `document.getElementById('counter').textContent = count` in twelve places. You miss one. The bug report arrives on Friday afternoon.

Signals end this. A signal is a reactive value. Change it, and everything that depends on it -- text nodes, attributes, computed values, side effects -- updates. You never write update logic again.

This chapter covers the entire reactivity system in tina4-js: creating signals, deriving values, running side effects, batching updates, and avoiding the mistakes that trip up every new user.

1. What Is a Signal

A signal is a reactive value. When it changes, everything that depends on it updates.

```
import { signal } from 'tina4js';

const count = signal(0);

// Read
console.log(count.value); // 0

// Write
count.value = 5;
console.log(count.value); // 5
```

That is the entire API for basic signals. `.value` to read. `.value =` to write. Nothing else.

When you write a new value, every subscriber -- effects, computed signals, DOM bindings -- fires. Not eventually. Not on the next tick. Synchronously, right now.

The Three Forms: Read This Before Anything Else

This is the rule that decides whether your UI updates. Everything else in this chapter is detail; this is the part you cannot skip. When you put a signal into an `html` template, there are **three forms**, and they are not interchangeable:

```
const count = signal(0);

html`<p>${count}</p>` // ■ REACTIVE - pass the signal itself
html`<p>${() => count.value * 2}</p>` // ■ REACTIVE - wrap an expression in a function
html`<p>${count.value}</p>` // ■ STATIC - renders "0" once, never updates again
```

You write	The template receives	Behaviour
<code>`\${count}`</code>	the signal object	Reactive text node, updates on every change

<code>\${() => count.value > 5}</code>	a function	Reactive block, re-runs on change; use for expressions, conditionals, lists
<code>\${count.value}</code>	a plain number	Static, evaluated once, frozen forever

Why? `${count.value}` is evaluated by JavaScript **before** the template ever sees it. The template gets the number `0`, a dead value with no way to know when `count` changes. `${count}` hands the template the live signal, so it can subscribe. `${() => ...}` hands it a function it can re-run.

If your UI renders once and never updates, you almost certainly wrote `${signal.value}` where you needed `${signal}` or `${() => ...}`. This one mistake accounts for the large majority of "my signal isn't working" reports. Burn the three forms into memory now and the rest of this chapter is easy.

*The same split shows up in attributes and properties (`${signal}` is reactive, `${signal.value}` is frozen), covered fully in the next chapter. The principle never changes: **pass the signal, or pass a function. Never pass `.value` into a template.***

2. Why `Object.is()` Matters

Signals use `Object.is()` to decide if a value changed. If the new value is the same as the old value by `Object.is()`, nothing happens. No subscribers fire. No DOM updates.

This matters for objects and arrays:

```
const items = signal(['apple', 'banana']);

// WRONG -- mutating in place, same reference, no update
items.value.push('cherry');
// Object.is(oldArray, newArray) === true -- nothing happens!

// RIGHT -- new array reference
items.value = [...items.value, 'cherry'];
// Object.is(oldArray, newArray) === false -- subscribers fire!
```

Same rule for objects:

```
const user = signal({ name: 'Alice', age: 30 });

// WRONG -- mutating in place
user.value.age = 31;

// RIGHT -- new object reference
user.value = { ...user.value, age: 31 };
```

This is the single most common mistake in tina4-js. UI not updating? Check this first. New reference, or mutation in place?

The rule fits on an index card: **always assign a new value to `.value`**. Spread arrays. Spread objects. Never mutate.

3. `computed()` -- Derived State

A shopping cart has items. Each item has a price and a quantity. The total is derived from those values. You do not store the total -- you compute it. When an item changes, the total recalculates.

That is what `computed()` does. It derives a value from other signals and keeps it current.

```
import { signal, computed } from 'tina4js';

const price = signal(10);
const quantity = signal(3);
const total = computed(() => price.value * quantity.value);

console.log(total.value); // 30

price.value = 20;
console.log(total.value); // 60
```

The function you pass to `computed()` runs immediately and then re-runs whenever any signal it reads changes. tina4-js tracks which signals were read inside the function -- you do not need to declare dependencies manually.

Computed Is Eager, Not Lazy

This is different from some other frameworks. In tina4-js, `computed()` re-evaluates as soon as a dependency changes, not when you read `.value`. This means:

- Computed values are always up to date
- There is no stale-read problem
- But unnecessary computed values do cost CPU

If you have an expensive computation that is only needed sometimes, do not put it in a computed. Use an effect with a conditional instead.

Computed Is Read-Only

```
const total = computed(() => price.value * quantity.value);

total.value = 100; // throws Error: '[tina4] computed signals are read-only'
```

If you need a writable derived value, use a signal and an effect.

4. `effect()` -- Side Effects

An effect runs a function immediately, then re-runs it whenever any signal it reads changes.

```
import { signal, effect } from 'tina4js';

const name = signal('World');
```

The Intelligent Native Application Framework

```
const dispose = effect(() => {
  console.log(`Hello, ${name.value}!`);
});
// logs: "Hello, World!"

name.value = 'Tina4';
// logs: "Hello, Tina4!"
```

Effects bridge reactive state to the outside world. Logging. DOM manipulation. Network requests. Analytics. Anything that needs to happen when data changes -- effects handle it.

Disposing Effects

`effect()` returns a dispose function. Call it to stop the effect from running:

```
const dispose = effect(() => {
  document.title = `Count: ${count.value}`;
});

// Later, when you are done:
dispose();

count.value = 99; // effect does NOT run
```

This matters for cleanup. Component unmounts. Route changes. Without disposal, effects keep running -- memory leaks and ghost updates accumulate. The router handles this for you: effects created during route rendering are disposed when the route changes. More on that in Chapter 5.

Auto-Tracking

You do not tell tina4-js which signals an effect depends on. It figures it out by running the function and seeing which `.value` getters are called:

```
const a = signal(1);
const b = signal(2);
const showB = signal(true);

effect(() => {
  if (showB.value) {
    console.log(a.value + b.value);
  } else {
    console.log(a.value);
  }
});
```

When `showB` is `true`, the effect tracks `showB`, `a`, and `b`. When `showB` becomes `false`, on the next run it only reads `showB` and `a` -- so changes to `b` no longer trigger the effect. The dependency set is dynamic.

Re-subscription on Re-run

Every time an effect re-runs, it unsubscribes from all previous signals and re-subscribes to whatever it reads this time. This is automatic. You do not need to manage subscriptions.

5. batch() -- Grouping Updates

Picture a form with a first name field and a last name field. A greeting effect reads both. Without batching, updating both fields fires the effect twice -- once with stale data, once with correct data. The user sees a flicker. The console logs an intermediate state that never should have existed.

```
const first = signal('Alice');
const last = signal('Smith');

effect(() => {
  console.log(`${first.value} ${last.value}`);
});
// logs: "Alice Smith"

first.value = 'Bob';    // logs: "Bob Smith"
last.value = 'Jones';  // logs: "Bob Jones"
```

That is two effect runs. With `batch()`, you get one:

```
import { batch } from 'tina4js';

batch(() => {
  first.value = 'Bob';
  last.value = 'Jones';
});
// logs: "Bob Jones" -- once
```

Inside a batch, signal writes are deferred. Subscribers are queued and only fire once when the batch completes.

When Do You Need batch()?

Since v1.0.9, **event handlers in templates are automatically batched**. So this already only triggers one update:

```
html`
  <button @click=${() => {
    first.value = 'Bob';
    last.value = 'Jones';
  }}>Update</button>
`
```

You need explicit `batch()` only when writing to multiple signals outside event handlers:

- `setTimeout` or `setInterval` callbacks
- `fetch().then()` handlers
- WebSocket message handlers
- Any async code

Inside a template event handler, batching is free. Everywhere else, wrap your writes.

6. peek() -- Read Without Tracking

An effect tracks every signal it reads. But sometimes you need a value without creating a dependency. You want to read a configuration signal once, not re-run the effect every time the config changes. That is what `peek()` does:

```
const count = signal(0);
const multiplier = signal(2);

effect(() => {
  // Tracks count, but NOT multiplier
  console.log(count.value * multiplier.peek());
});

count.value = 5; // effect runs
multiplier.value = 3; // effect does NOT run
```

`peek()` returns the current value without registering the read. The effect does not re-run when `multiplier` changes.

This is useful for:

- Reading config values that rarely change
- Avoiding infinite loops when an effect writes to a signal it also reads
- Performance optimization -- skipping unnecessary re-runs

7. Sharing Signals Across Files -- The Store Pattern

A signal created in one component file is local to that file. To share state across your app -- the logged-in user, the shopping cart, the theme -- you put signals in their own module and import them wherever you need them. That module is your store. There is no `createStore()`, no provider, no context wrapper. **A signal that is exported from a module IS global state.**

One file, all your app-wide signals

```
// src/store.ts
import { signal, computed } from 'tina4js';

// ■■ Auth ████████████████████████████████████████████████████████████
export const user      = signal<User | null>(null);
export const authToken = signal('');

// ■■ Cart ████████████████████████████████████████████████████████████
export const cart = signal<CartItem[]>([]);

// ■■ UI state ██████████████████████████████████████████████████████████
export const theme     = signal<'light' | 'dark'>('light');
export const sidebar   = signal(false);

// ■■ Derived state -- computed signals belong here too ■
export const isLoggedIn = computed(() => user.value !== null);
export const cartCount  = computed(() => cart.value.length);
export const cartTotal  = computed(() =>
    cart.value.reduce((sum, i) => sum + i.price * i.qty, 0)
);
```

That is the entire store. Every `export const` is a piece of global state. Computed signals go here too -- derived state your whole app reads.

Import and read in any component

```
// src/components/Navbar.ts
import { html } from 'tina4js';
import { user, isLoggedIn, cartCount } from '../store';

export const Navbar = () => html`
  <nav>
    <a href="/cart">Cart (${cartCount})</a>
    ${() => isLoggedIn.value
      ? html`<span>Hi, ${user}</span>`
      : html`<a href="/login">Log in</a>`}
  </nav>`;
```

Import and write from a different component

```
// src/components/ProductCard.ts
import { html } from 'tina4js';
import { cart } from '../store';

export const ProductCard = (product: Product) => html`
  <button @click=${() => {
    cart.value = [...cart.value, { ...product, qty: 1 }];
  }}>Add to cart</button>`;
```

`ProductCard` writes `cart.value`. `Navbar` read `cartCount` (which reads `cart`). The `Cart (N)` text node updates -- across two files, automatically.

Why it works

ES modules are singletons. Every file that does `import { cart } from '../store'` gets a reference to the **exact same signal object** -- the module is evaluated once and cached. Write to it from anywhere, every reader updates. That is your entire state-management layer.

The Intelligent Native Application 4ramework

The rules

Do	Don't
Keep app-wide signals in <code>src/store.ts</code>	Scatter <code>signal()</code> calls across component files
<code>import { cart } from '../store'</code> everywhere	<code>const myCart = signal(cart.value)</code> -- that is a disconnected copy , not the shared signal
Put <code>computed()</code> derived state in the store too	Recompute the same derivation in five components
Keep component-local state inside the component	Promote every signal to global "just in case"

A form's draft fields, a dropdown's open/closed state -- those stay as `signal()` *inside* the component function. Only state that genuinely spans components belongs in `store.ts`.

***One signal, one source of truth.** The most common store mistake is "copying" a signal -- `signal(user.value)` in a second file. That reads the value once and creates a brand-new, disconnected signal. Always import the original.*

8. Debug Labels

Signals accept an optional second argument -- a debug label:

```
const count = signal(0, 'count');
const username = signal('', 'username');
const items = signal([], 'cart-items');
```

Labels do nothing in production. In the debug overlay (Chapter 11), labels appear in the Signals panel. Instead of staring at `Signal<number>` with value `7` and wondering which signal that is, you see `count: 7`. That difference matters at 2 AM when something is not updating.

Add labels to every signal you might need to debug. The overhead is zero when the debug module is not imported.

9. `isSignal()` -- Type Check

```
import { isSignal } from 'tina4js';

const count = signal(0);

isSignal(count);    // true
isSignal(42);       // false
isSignal(null);     // false
isSignal({ value: 1 }); // false -- must be a real tina4 signal
```


Useful when writing utilities that accept either a signal or a plain value.

10. Common Mistakes

Mistake 1: Reading .value in Templates

```
// WRONG -- evaluates once, never updates
html`<p>${count.value}</p>`

// RIGHT -- pass the signal directly
html`<p>${count}</p>`

// RIGHT -- use a function for expressions
html`<p>${() => count.value * 2}</p>`
```

When you write `${count.value}`, JavaScript evaluates `count.value` before the template function sees it. The template gets the number `0`, not the signal. It cannot subscribe to changes.

When you write `${count}`, the template receives the signal object. It creates a text node and subscribes to changes. The DOM updates automatically.

Mistake 2: Mutating in Place

```
// WRONG
items.value.push(newItem);

// RIGHT
items.value = [...items.value, newItem];
```

Covered above, but it is worth repeating. This is the number one bug in tina4-js applications.

Mistake 3: Creating Effects in Loops

```
// WRONG -- creates a new effect every time the list re-renders
html`${() => items.value.map(item => {
  effect(() => console.log(item)); // leaks!
  return html`<div>${item}</div>`;
})}`
```

Effects created inside reactive blocks are fine -- the router and html renderer handle cleanup. But manually creating effects in a loop that runs inside a reactive function will leak unless you dispose them. Let the template engine handle reactivity. Use `${signal}` and `${() => expr}` instead of manual effects for DOM updates.

Mistake 4: Forgetting That false Renders as Text

```
// WRONG -- if condition is false, renders the TEXT "false"
html`${showDetails && html`<div>Details here</div>}`

// RIGHT -- use a ternary
html`${() => showDetails.value ? html`<div>Details here</div>` : null}`
```

In JavaScript, `false && anything` evaluates to `false`. The template receives the boolean `false` and renders it as the string `"false"`. Use the ternary pattern. Always.

11. Putting It Together -- A Todo List

Signals. Computed. Effects. Batch. Here they are, working together in a complete todo application:

```
import { signal, computed, effect, batch, html } from 'tina4js';

function todoApp() {
  const items = signal<{ text: string; done: boolean }[]>([], 'todo-items');
  const input = signal('', 'todo-input');
  const filter = signal<'all' | 'active' | 'done'>('all', 'todo-filter');

  const filtered = computed(() => {
    const list = items.value;
    if (filter.value === 'active') return list.filter(i => !i.done);
    if (filter.value === 'done') return list.filter(i => i.done);
    return list;
  });

  const remaining = computed(() =>
    items.value.filter(i => !i.done).length
  );

  // Side effect: update document title
  effect(() => {
    document.title = `Todos (${remaining.value} left)`;
  });

  const addItem = () => {
    const text = input.value.trim();
    if (!text) return;
    batch(() => {
      items.value = [...items.value, { text, done: false }];
      input.value = '';
    });
  };

  const toggleItem = (index: number) => {
    items.value = items.value.map((item, i) =>
      i === index ? { ...item, done: !item.done } : item
    );
  };

  const removeItem = (index: number) => {
    items.value = items.value.filter((_, i) => i !== index);
  };

  return html`
    <div>
      <h1>Todos</h1>

      <form @submit=${(e: Event) => { e.preventDefault(); addItem(); }}>
        <input
          type="text"
          placeholder="What needs to be done?"
          .value=${input}
          @input=${(e: Event) => { input.value = (e.target as HTMLInputElement).value; }}
        />
      </form>
    </div>
  `;
}
```

```

    <button type="submit">Add</button>
  </form>

  <div>
    <button @click=${() => filter.value = 'all'}>All</button>
    <button @click=${() => filter.value = 'active'}>Active</button>
    <button @click=${() => filter.value = 'done'}>Done</button>
  </div>

  <ul>
    ${() => filtered.value.map((item, index) => html`
      <li>
        <input
          type="checkbox"
          @click=${() => toggleItem(index)}
          ?checked=${item.done}
        />
        <span style="${item.done ? 'text-decoration: line-through' : ''}">${item.text}</span>
        <button @click=${() => removeItem(index)}>x</button>
      </li>
    `)}
  </ul>

  <p>${remaining} items left</p>
</div>
`;
}

```

Every concept from this chapter is present:

- `items`, `input`, and `filter` are signals with debug labels -- visible in the overlay
- `filtered` and `remaining` are computed -- they recalculate when `items` or `filter` change
- `effect()` syncs the document title as a side effect
- `batch()` groups the add-and-clear into one update
- The list uses `${() => filtered.value.map(...)}` -- a reactive block that re-renders when `filtered` changes
- New references everywhere: `map()` returns a new array, `filter()` returns a new array, the spread creates new objects

No mutation. No manual DOM updates. No event bus. Signals handle all of it.

Persisting Signals Across Refreshes

A signal lives in memory. Refresh the page and it forgets. For values the user picked themselves -- theme, sidebar state, the last filter, a draft reply -- forgetting is rude. Wrap the signal with `persist()` and its value reads from `localStorage` on creation and writes back on every change.

```

import { signal } from 'tina4js';
import { persist } from 'tina4js/storage';

```

The Intelligent Native Application 4ramework

```
const theme = persist(signal('light'), { key: 'theme' });
theme.value = 'dark'; // saved. Survives a refresh.
```

That is the whole idea. The full reference -- the never-store dangers list, every option, cross-tab sync, version migration, and the logout wipe -- lives in its own chapter: [Persistent Signals](#). ---

Summary

Concept	API	Purpose
Signal	<code>signal(value, label?)</code>	Reactive value
Read	<code>sig.value</code>	Get current value (tracks dependency)
Write	<code>sig.value = x</code>	Set value (notifies subscribers)
Computed	<code>computed(() => expr)</code>	Derived read-only signal
Effect	<code>effect(() => { ... })</code>	Side effect, returns dispose function
Batch	<code>batch(() => { ... })</code>	Group writes, one notification pass
Peek	<code>sig.peek()</code>	Read without tracking
Check	<code>isSignal(x)</code>	Returns true for tina4 signals
Label	<code>signal(0, 'name')</code>	Debug overlay identification
Persist	<code>persist(sig, { key })</code> from <code>tina4js/storage</code>	Survives a page refresh. Never for credentials.
Clear	<code>clearPersistedKeys(['k1', 'k2'])</code>	Wipe a list of keys, e.g. on logout

Persistent Signals

A signal lives in memory. Refresh the page and it forgets. For values the user picked themselves -- theme, sidebar collapsed state, the last-used filter, draft text, a guest cart -- forgetting is rude. This chapter wraps a signal so its value survives a refresh, and draws the hard line around what you must never keep there.

A signal lives in memory. Refresh the page and it forgets. For values the user picked themselves -- theme, sidebar collapsed state, last-used filter, draft text, guest cart contents -- forgetting is rude.

`tina4js/storage` wraps a signal so its value reads from `localStorage` on creation and writes back on every change. Opt-in per signal. Zero dependencies. Tree-shakeable, so apps that do not import it ship zero bytes.

```
import { signal } from 'tina4js';
import { persist, clearPersistedKeys } from 'tina4js/storage';

const theme = persist(signal('light'), { key: 'theme' });

theme.value = 'dark'; // saved to localStorage. Survives a refresh.
```

The wrapper returns the same signal you passed in, with two extras attached: `.clear()` removes the key from storage, `.dispose()` stops the write effect.

The Dangers List, Up Front

`localStorage` is XSS-readable. Any script that runs on your origin reads every value. So `persist()` is the right tool for small, safe, user-chosen preferences. It is the wrong tool for the following, no exceptions:

- Auth tokens, JWTs, session IDs, API keys. Use `httpOnly` cookies.
- Passwords, including ones you think you encrypted client-side.
- Personal data: names, emails, phone numbers, addresses, IDs.
- Payment data: card numbers, CVV, expiry.
- Permission flags, roles, `isAdmin` booleans. The user can edit them in devtools.
- Encryption keys, OTP seeds, secrets.
- Server-of-record state: orders, balances, ledger entries. Fetch fresh from the database.

If you ignore this list, the framework warns you in the console. It looks at the key name (`token`, `password`, `secret`, `apikey`, `auth`, `credential`, `jwt`, `bearer`, `otp`, `private_key`, `session_id`) and at the value shape (a JWT, a long base64 string, an object with a credential-shape field). The warning is loud, once per key, and on purpose. See `STORAGE.md` in the `tina4-js` repo for the full table and the reasoning behind each row.

Options

```
persist(signal(0), {
  key: 'count', // required
  storage: 'local', // 'local' (default) or 'session'
  serializer: { read, write }, // default: JSON
  version: 1, // stored-shape version
  migrate: (oldValue, oldVersion) => 0, // run when versions disagree
  syncTabs: false, // 'storage' event sync, opt-in
  silenceCredentialWarning: false, // for false positives like tokenColor
});
```

Cross-tab Sync

Two tabs of the same app, both running `persist(signal([]), { key: 'cart', syncTabs: true })`. Add an item to the cart in tab A, and tab B sees it without a refresh. The `storage` event fires in tabs that did not write the value, so the framework subscribes there and updates the signal.

It is opt-in per signal. You decide which values cross tabs. No global broadcast.

Wipe on Logout

When a user logs out, persisted state can leak to the next user on the same machine. The cure is `clearPersistedKeys()` on the logout path:

```
import { clearPersistedKeys } from 'tina4js/storage';

function logout() {
  api.post('/auth/logout');
  clearPersistedKeys(['cart', 'lastFilter', 'draftReply']);
  window.location.reload();
}
```

The function removes only the keys you name. Other persisted state survives.

Version Migration

A deploy changes the stored shape. Old browsers still hold the old shape. Without `migrate`, the framework discards the stored value and logs a warning. With `migrate`, you convert in place:

```
// v1 stored: { name: 'Alice' }
// v2 wants: { firstName: 'Alice', lastName: '' }

const user = persist(signal({ firstName: '', lastName: '' }), {
  key: 'user',
  version: 2,
  migrate: (old) => ({
    firstName: (old as { name?: string }).name ?? '',
    lastName: '',
  }),
});
```

Safety Guarantees

- **SSR-safe.** No `window` or `localStorage`? `persist()` is a silent no-op. The signal still works in memory.
- **Quota-safe.** `QuotaExceededError` is logged and skipped; the signal still updates.
- **No "encrypted" option.** Encryption with a key sitting in the same bundle is theatre. The framework refuses to ship that knob.
- **Cross-tab sync is opt-in.** Off by default.

Bundle Cost

`dist/storage.es.js` is 1.67 KB gzipped. Apps that never import from `tina4js/storage` ship zero bytes from this module. The 1.5 KB core promise is untouched.

HTML Templates

DOM Without the Framework Tax

Open your browser's DevTools. Inspect a React application. You see a `<div id="root">` containing a tree of elements that a virtual DOM diffing algorithm built for you. Somewhere between your JSX and those elements, a reconciler compared two trees, computed a minimal set of patches, and applied them. For a counter displaying the number 5.

tina4-js skips all of that. The `html` tagged template creates real DOM nodes. It binds signals to specific text nodes and attributes. When a signal changes, that one text node updates. No tree comparison. No patch computation. No framework standing between your data and your DOM.

This chapter covers every binding syntax in the `html` tagged template -- reactive text, reactive blocks, events, boolean attributes, property bindings, and class bindings. You will know which ones update and which ones are static, and you will never fall into the `false/null` rendering trap.

1. What `html` Does

```
import { html } from 'tina4js';

const fragment = html`<h1>Hello, World!</h1>`;
document.body.appendChild(fragment);
```

`html` is a tagged template literal function. It parses your markup once, caches the result, then clones it for each call. The return value is a real `DocumentFragment` -- actual DOM nodes. Not a string. Not a virtual tree. Not an intermediate representation.

No `render()` loop. No reconciliation. No diffing. The template creates DOM, binds signals to specific nodes, and walks away. Updates happen through direct signal subscriptions on individual nodes. The framework is not in the middle of the conversation between your data and your page.

2. Static Values: `${value}`

Plain values go in as text nodes. Once. Never again.

```
const name = 'Alice';
html`<p>Hello, ${name}</p>`
```

The template evaluates `name`, gets the string `"Alice"`, creates a text node, and inserts it. If `name` changes later, the DOM does not update. It is static.

Everything is XSS-safe by default. Values are inserted as text nodes, not HTML:

```
const userInput = '<script>alert("xss")</script>';
html`<p>${userInput}</p>`
// Renders as the literal text: <script>alert("xss")</script>
// NOT executed as HTML
```

The Intelligent Native Application Framework

You cannot accidentally inject HTML through interpolation. This is by design.

3. Reactive Text: `${signal}`

Pass a signal directly (not `.value`) to create a reactive text node:

```
import { signal, html } from 'tina4js';

const count = signal(0);

html`<p>Count: ${count}</p>`
```

When `count.value` changes, the text node updates. Nothing else in the DOM moves. The `<p>` element stays. The `"Count: "` text stays. Only the number changes.

This is the most common pattern in tina4-js. One signal. One text node. Live updates forever.

Critical rule: Pass the signal, not its value.

```
// WRONG -- static, evaluates once
html`<p>${count.value}</p>`

// RIGHT -- reactive, updates on change
html`<p>${count}</p>`
```

When you write `${count.value}`, JavaScript evaluates `count.value` (gets `0`) and passes the number `0` to the template. The template sees a plain number, not a signal. It creates a static text node.

When you write `${count}`, JavaScript passes the signal object. The template detects it (via `isSignal()`), creates a text node, and subscribes to changes. When the signal updates, the text node updates.

4. Reactive Blocks: `${() => expr}`

Functions are reactive blocks. The function runs immediately, and re-runs whenever any signal read inside it changes:

```
const show = signal(true);
const name = signal('Alice');

html`
  <div>
    ${() => show.value
      ? html`<p>Hello, ${name}</p>`
      : null
    }
  </div>
`
```

When `show` or `name` changes, the function re-runs. The previous nodes are removed and new nodes are inserted. This is how you do conditional rendering and lists.

Conditional Rendering

```
const isLoggedIn = signal(false);

html`
  ${() => isLoggedIn.value
    ? html`<p>Welcome back!</p>`
    : html`<a href="/login">Log in</a>`}
`
```

Lists

```
const items = signal(['Apple', 'Banana', 'Cherry']);

html`
  <ul>
    ${() => items.value.map(item => html`<li>${item}</li>`)}
  </ul>
`
```

When `items` changes, the entire `<ul>` content is replaced. Old nodes out. New nodes in. This is not keyed reconciliation like React -- it is a full swap. For lists under a few hundred items, the browser handles this in under a millisecond. For massive lists, consider a virtualization library.

Nested Reactivity

Reactive blocks can contain signals:

```
const items = signal([
  { name: signal('Apple'), price: signal(1.50) },
]);

html`
  <ul>
    ${() => items.value.map(item => html`
      <li>${item.name} - ${item.price}</li>
    `)}
  </ul>
`
```

Each `${item.name}` is a reactive text node. If you change `items.value[0].name.value = 'Pear'`, only that text node updates. The list does not re-render.

5. DocumentFragments and Arrays

You can nest templates and pass arrays:

```
const header = html`<h1>Title</h1>`;
const items = ['one', 'two', 'three'];

html`
  ${header}
  <ul>
    ${items.map(i => html`<li>${i}</li>`)}
  </ul>
`
```

`DocumentFragment` values are inserted directly. Arrays are flattened -- each item is converted to nodes.

6. Event Handlers -- @event

The `@` prefix binds event listeners:

```
html`
  <button @click=${() => console.log('clicked!')}>Click me</button>
  <input @input=${(e: Event) => console.log((e.target as HTMLInputElement).value)} />
  <form @submit=${(e: Event) => {
    e.preventDefault();
    handleSubmit();
  }}>
    ...
  </form>
```

Any DOM event works. `@click`. `@input`. `@change`. `@submit`. `@keydown`. `@mouseenter`. `@focus`. `@blur`. If the browser fires it, tina4-js can bind it.

Auto-Batching

Since v1.0.9, all event handlers are automatically wrapped in `batch()`. This means you can write to multiple signals in one handler and only get one DOM update:

```
html`
  <button @click=${() => {
    firstName.value = 'Bob';
    lastName.value = 'Jones';
    age.value = 30;
  }}>Update All</button>
`

// Three signal writes, one DOM update
```

You do not need explicit `batch()` inside event handlers. It happens automatically.

7. Boolean Attributes -- ?attr

A button should be disabled while a form submits. A div should be hidden until data loads. A checkbox should be checked when a task is done. HTML boolean attributes -- `disabled`, `hidden`, `checked`, `readonly`, `required` -- need the `?` prefix:

```
const isDisabled = signal(false);
const isHidden = signal(true);

html`
  <button ?disabled=${isDisabled}>Submit</button>
  <div ?hidden=${isHidden}>Secret content</div>
`
```

When the value is truthy, the attribute is added (e.g., `<button disabled>`). When falsy, the attribute is removed entirely.

The Intelligent Native Application Framework

Without the `?` prefix, you get a string attribute:

```
// WRONG -- sets disabled="false" (which is still disabled!)
html`<button disabled=${isDisabled}>Submit</button>`

// RIGHT -- adds/removes the disabled attribute
html`<button ?disabled=${isDisabled}>Submit</button>`
```

This distinction catches everyone at least once. In HTML, `<button disabled="false">` is still disabled. The attribute exists. The browser does not read its value. The `?` prefix solves this by adding or removing the attribute entirely -- present means true, absent means false.

Boolean attributes accept signals, functions, and computed values:

```
// Signal
html`<button ?disabled=${isDisabled}>Submit</button>`

// Function
html`<button ?disabled=${() => items.value.length === 0}>Submit</button>`

// Computed
const canSubmit = computed(() => name.value.length > 0);
html`<button ?disabled=${() => !canSubmit.value}>Submit</button>`
```

8. Property Bindings -- `.prop`

The `.` prefix sets DOM properties (not HTML attributes):

```
const inputValue = signal('hello');

html`
  <input .value=${inputValue} />
  <div .innerHTML=${html`<strong>Bold text</strong>`} />
`
```

`.value` is the most common property binding. It sets the input's `value` property directly, which is different from the `value` attribute (the attribute is the initial value; the property is the current value).

`.innerHTML` is how you inject raw HTML. This is the only way to render HTML strings or inline SVG in tina4-js:

```
const svgIcon = '<svg viewBox="0 0 24 24"><path d="M12 2L2 22h20L12 2z"/></svg>';

// WRONG -- renders as escaped text
html`<div>${svgIcon}</div>`
// Shows: <svg viewBox="0 0 24 24">...

// RIGHT -- renders as HTML
html`<div .innerHTML=${svgIcon}></div>`
// Shows the actual SVG triangle
```

Warning: `.innerHTML` bypasses XSS protection. Only use it with trusted content. Never pass user input to `.innerHTML`.

When a signal is passed to a property binding, it updates reactively:

The Intelligent Native Application Framework

```
const content = signal('<em>Loading...</em>');

html`<div .innerHTML=${content}></div>`

// Later:
content.value = '<em>Done!</em>';
// The div's innerHTML updates
```

CRITICAL: Never Put Inputs Inside Reactive Blocks

Inputs inside `${() => ...}` lose focus on every keystroke. The most common mistake in tina4-js is wrapping `<input>` elements inside reactive arrow functions. When a signal changes, the reactive block re-renders its entire DOM subtree, destroying the input and creating a new one. The cursor disappears, the input loses focus, and typing is interrupted.

BAD: input inside a reactive block (broken):

```
const name = signal('');
const showExtra = signal(false);

// WRONG: the input is INSIDE ${() => ...} - it gets destroyed on every keystroke
html`
  <div>
    ${() => html`
      <input .value=${name} @input=${(e: Event) => {
        name.value = (e.target as HTMLInputElement).value;
      }} />
      ${showExtra.value ? html`<p>Extra info</p>` : null}
    `}
  </div>
`;
```

GOOD: input outside, only dynamic content in reactive blocks:

```
const name = signal('');
const showExtra = signal(false);

// RIGHT: the input stays in the static template - only the conditional goes in ${() => ...}
html`
  <div>
    <input .value=${name} @input=${(e: Event) => {
      name.value = (e.target as HTMLInputElement).value;
    }} />
    ${() => showExtra.value ? html`<p>Extra info</p>` : null}
  </div>
`;
```

The rule: Inputs, textareas, and selects belong in the **static part** of your template. Use `.value`, `@input`, and `?disabled` bindings to make them reactive. Only wrap **computed output** (conditional messages, dynamic lists, calculated text) in `${() => ...}` blocks.

9. Dynamic Attributes: `attr=${value}`

Regular attributes accept signals and functions for reactive updates:

```
const color = signal('red');
const className = signal('active');

html`
  <div class=${className} style=${() => `color: ${color.value}`}>
    Styled text
  </div>
`
```

When `className` changes, the `class` attribute updates. When `color` changes, the `style` attribute updates.

Reactive Classes

```
const isActive = signal(true);

// Simple string signal
const cls = signal('btn btn-primary');
html`<button class=${cls}>Click</button>`

// Function-based
html`<div class=${() => isActive.value ? 'tab active' : 'tab'}>Tab</div>`
```

10. The false/null/undefined Trap

You will hit this. Everyone does. Here are the rules:

```
{false}      // Renders nothing (empty) - treated the same as null
{null}       // Renders nothing (empty)
{undefined}  // Renders nothing (empty)
{true}       // Renders the TEXT "true"
{0}         // Renders the TEXT "0"
```

`false`, `null`, and `undefined` all render as nothing; they are explicitly swallowed by the template engine. Every other value is converted to text via `String(value)`.

This means the `&&` shortcut actually works for hiding content: `false` disappears cleanly:

```
// This works -- false renders nothing
html`${() => show.value && html`<p>Content</p>`}

// This also works -- explicit ternary is clearer and preferred
html`${() => show.value ? html`<p>Content</p>` : null}`
```

The ternary is still the recommended style because intent is explicit and it handles both branches. But unlike React, you will not see the stray text `"false"` appear in the DOM if you forget.

11. Template Caching

The `html` tag caches parsed templates by the identity of the template strings array. The first call parses the HTML and creates a `<template>` element. Subsequent calls with the same template literal clone the cached template and bind fresh values.

This means:

- Templates in loops are only parsed once
- Cloning a `<template>` is fast (browser-native)
- The overhead per render is binding, not parsing

You do not need to do anything to benefit from this. It is automatic.

12. Putting It All Together

Static text. Reactive text. Reactive blocks. Event handlers. Boolean attributes. Property bindings. Conditional rendering. Here they all are, working together in a login form:

```
import { signal, computed, html } from 'tina4js';

function loginForm() {
  const email = signal('');
  const password = signal('');
  const loading = signal(false);
  const error = signal<string | null>(null);

  const isValid = computed(() =>
    email.value.includes('@') && password.value.length >= 8
  );

  const handleSubmit = async (e: Event) => {
    e.preventDefault();
    loading.value = true;
    error.value = null;

    try {
      const response = await fetch('/api/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          email: email.value,
          password: password.value,
        }),
      });
      if (!response.ok) throw new Error('Invalid credentials');
      // handle success...
    } catch (err) {
      error.value = (err as Error).message;
    } finally {
      loading.value = false;
    }
  };
}
```

The Intelligent Native Application 4ramework

```

return html`
  <form @submit=${handleSubmit}>
    <h2>Login</h2>

    ${() => error.value
      ? html`<p style="color: red">${error}</p>`
      : null
    }

    <label>
      Email
      <input
        type="email"
        .value=${email}
        @input=${(e: Event) => { email.value = (e.target as HTMLInputElement).value; }}
        ?disabled=${loading}
      />
    </label>

    <label>
      Password
      <input
        type="password"
        .value=${password}
        @input=${(e: Event) => { password.value = (e.target as HTMLInputElement).value; }}
        ?disabled=${loading}
      />
    </label>

    <button
      type="submit"
      ?disabled=${() => !isValid.value || loading.value}
    >
      ${() => loading.value ? 'Logging in...' : 'Login'}
    </button>
  </form>
`;
}

```

Every binding type from this chapter appears in this form:

- `.value=${email}` -- property binding keeps the input's DOM property in sync with the signal
- `@input` -- event handler updates the signal when the user types
- `?disabled=${loading}` -- boolean attribute toggles from a signal
- `?disabled=${() => !isValid.value || loading.value}` -- boolean attribute from a function
- `${() => loading.value ? 'Logging in...' : 'Login'}` -- reactive text block swaps the button label
- `${() => error.value ? html`...` : null}` -- conditional rendering with the ternary pattern

One template. Six binding types. Zero manual DOM updates. The template engine handles every transition between states, and the form responds the moment data changes.

Summary

	What it does	Reactive?		
	Static text node, XSS-safe	No		
	Reactive text node	Yes		
<code>{</code>	Reactive block (conditionals, lists)	Yes		
	Insert DocumentFragment	No		
	Render each item	No		
	<code>null</code>	<code>undefined</code>	Renders nothing (empty)	-
<code>}</code>	Event listener (auto-batched)	-		
<code>{x}</code>	Boolean attribute (add/remove)	If signal/function		
	DOM property binding	If signal		
<code>\${x}</code>	Raw HTML injection	If signal		
	Regular attribute	If signal/function		

Components

Web Components That Don't Suck

You build a button component in React. It works in React. You build one in Vue. It works in Vue. You build one in Angular. It works in Angular. Three frameworks. Three components. Same button.

Web Components solve this. One component. Every framework. Every page. Every context. The browser itself is the runtime.

The problem is that raw Web Components are verbose. Boilerplate for observed attributes. Manual attribute-to-property reflection. No reactive rendering. [Tina4Element](#) strips all of that away. You get reactive props, scoped styles, and template rendering -- with the full portability of native Web Components underneath.

1. What Are Tina4 Components

A Tina4 component is a Web Component. A real one. It extends [HTML⁴Element](#), registers with [customElements.define\(\)](#), and works anywhere the browser runs. Drop it in a React app. Drop it in a static HTML page. Drop it in a WordPress theme. No framework runtime required to consume it.

[Tina4Element](#) adds three things on top of native Web Components:

- **Reactive props** -- attributes become signals
- **Scoped styles** -- CSS that cannot leak in or out (via Shadow DOM)
- **Template rendering** -- your [render\(\)](#) method returns [html](#) tagged templates

Everything else is standard Web Components. No magic. No hidden state. No framework lock-in.

2. Your First Component

```
import { Tina4Element, html } from 'tina4js';

class GreetingCard extends Tina4Element {
  static props = { name: String };

  render() {
    return html`
      <div class="card">
        <h2>Hello, ${this.prop('name')}!</h2>
        <p>Welcome to tina4-js.</p>
      </div>
    `;
  }
}

customElements.define('greeting-card', GreetingCard);
```

Use it in HTML:

```
<greeting-card name="Alice"></greeting-card>
```

Or in a template:

```
html`<greeting-card name="Alice"></greeting-card>`
```

Change the attribute and the component updates:

```
document.querySelector('greeting-card')!.setAttribute('name', 'Bob');
// The heading updates to "Hello, Bob!"
```

3. static props -- Declaring Reactive Props

The `static props` object declares which attributes the component observes. Each key is an attribute name, and the value is a type constructor:

```
class UserCard extends Tina4Element {
  static props = {
    name: String,
    age: Number,
    active: Boolean,
  };

  render() {
    return html`
      <div>
        <h3>${this.prop('name')}</h3>
        <p>Age: ${this.prop('age')}</p>
        <p>${() => this.prop<boolean>('active').value ? 'Active' : 'Inactive'}</p>
      </div>
    `;
  }
}
```

Type Coercion

HTML attributes are always strings. tina4-js coerces them based on the type you declare:

Type	Coercion Rule	Example
String	Attribute value as-is, or <code>' '</code> if absent	<code>name="Alice" -> 'Alice'</code>
Number	<code>Number(value)</code> , or <code>0</code> if absent	<code>age="30" -> 30</code>
Boolean	<code>true</code> if attribute exists, <code>false</code> if absent	<code>active -> true</code> , no attribute -> <code>false</code>

Boolean props follow the HTML convention: the attribute's presence means `true`, its absence means `false`. The attribute value does not matter.

```
<user-card name="Alice" age="30" active></user-card>
```

- `name` -> `'Alice'` (String)
- `age` -> `30` (Number)
- `active` -> `true` (Boolean, attribute is present)

```
<user-card name="Bob" age="25"></user-card>
```

- `active` -> `false` (Boolean, attribute is absent)

4. this.prop(name) -- Reading Props

`this.prop('name')` returns a **signal** for the named prop. This means you can:

- **Drop it in a template** for reactive rendering:

```
html`<span>${this.prop('name')}</span>`
```

- **Read its value** in methods:

```
const currentName = this.prop<string>('name').value;
```

- **Use it in computed/effects:**

```
const greeting = computed(() => `Hello, ${this.prop<string>('name').value}!`);
```

When the HTML attribute changes (via `setAttribute` or from a parent template), the prop signal updates and everything that depends on it re-renders.

5. static styles -- Scoped CSS

```
class StatusBadge extends Tina4Element {
  static props = { status: String };

  static styles = `
    :host {
      display: inline-block;
    }
    .badge {
      padding: 0.25rem 0.75rem;
      border-radius: 999px;
      font-size: 0.75rem;
      font-weight: 600;
      text-transform: uppercase;
    }
    .badge.active { background: #dcfce7; color: #166534; }
    .badge.inactive { background: #fee2e2; color: #991b1b; }
    .badge.pending { background: #fef3c7; color: #92400e; }
  `;

  render() {
    return html`
      <span class=${() => `badge ${this.prop<string>('status').value}`}>
        ${this.prop('status')}
      </span>
    `;
  }
}

customElements.define('status-badge', StatusBadge);
```

Styles are injected into the Shadow DOM `<style>` tag. They are completely scoped:

- `.badge` inside this component does not affect any `.badge` outside it
- External CSS does not affect elements inside this component
- The `:host` selector targets the component element itself

The `:host` Selector

`:host` is a Shadow DOM feature. It selects the component's outer element (the custom element tag). Use it to set `display`, `margin`, `padding`, and other box-model properties:

```
:host {
  display: block;
  margin-bottom: 1rem;
}
```

Without `:host { display: block }`, custom elements default to `display: inline`, which often causes unexpected layout behavior.

6. static shadow -- Light DOM vs Shadow DOM

By default, components use Shadow DOM (`static shadow = true`). Shadow DOM provides style encapsulation but has tradeoffs:

- **Shadow DOM (default):** Styles are scoped. External CSS cannot reach inside. Good for reusable components.
- **Light DOM:** No style encapsulation. Component renders directly into the page DOM. External CSS applies normally.

To use light DOM:

```
class PageSection extends Tina4Element {
  static shadow = false;
  static props = { title: String };

  render() {
    return html`
      <section>
        <h2>${this.prop('title')}</h2>
        <p>This content is rendered directly into the page DOM.</p>
      </section>
    `;
  }
}
```

When `shadow` is `false`:

- `static styles` is ignored (no Shadow DOM to scope them)
- The component renders its content directly as children of the custom element
- Global CSS applies to the component's internal elements
- `<slot>` elements do not work -- slots are a Shadow DOM feature. If you need content projection, use Shadow DOM (`static shadow = true`)
- This is useful for layout components that need to inherit page styles

7. this.emit() -- Custom Events

Props flow down. Events flow up. A parent tells a child what to display through attributes. The child tells the parent what happened through custom events. This is the same pattern React uses with callbacks and Vue uses with `$emit` -- but here it is native DOM events, and they work across framework boundaries.

```
class TodoItem extends Tina4Element {
  static props = { text: String, done: Boolean };

  render() {
    return html`
      <li>
        <input
          type="checkbox"
          ?checked=${this.prop('done')}
        >
      </li>
    `;
  }
}
```

The Intelligent Native Application Framework

```

        @change=${() => this.emit('toggle')}}
      />
      <span>${this.prop('text')}</span>
      <button @click=${() => this.emit('remove')}}>x</button>
    </li>
  `;
}
}

```

```
customElements.define('todo-item', TodoItem);
```

Listen for the event in the parent:

```

html`
  <todo-item
    text="Buy milk"
    done
    @toggle=${() => toggleItem(0)}
    @remove=${() => removeItem(0)}
  ></todo-item>
`

```

Passing Data with Events

```
this.emit('select', { detail: { id: 42, name: 'Alice' } });
```

Listen for it:

```

html`
  <user-list @select=${(e: CustomEvent) => {
    console.log(e.detail.id); // 42
    console.log(e.detail.name); // 'Alice'
  }}></user-list>
`

```

Events are dispatched with **bubbles: true** and **composed: true** by default, so they cross Shadow DOM boundaries and bubble up the tree. You can listen for them anywhere above the component.

8. Lifecycle Hooks

Tina4Element provides two lifecycle hooks:

onMount()

Called after the component's first render. Use it for setup that needs DOM access:

```

class ChartWidget extends Tina4Element {
  static props = { data: String };

  onMount() {
    // DOM is ready, shadow root has content
    console.log('Chart mounted');
    // Initialize third-party library, set up intervals, etc.
  }

  render() {

```

```

        return html`<canvas id="chart"></canvas>`;
    }
}

```

onUnmount()

Called when the component is removed from the DOM. Use it for cleanup:

```

class LiveClock extends Tina4Element {
    private intervalId = 0;
    private time = signal(new Date().toLocaleTimeString());

    onMount() {
        this.intervalId = window.setInterval(() => {
            this.time.value = new Date().toLocaleTimeString();
        }, 1000);
    }

    onUnmount() {
        clearInterval(this.intervalId);
    }

    render() {
        return html`<span>${this.time}</span>`;
    }
}

```

Timers. Event listeners. WebSocket connections. Anything that outlives the DOM needs cleanup here. If you create it in `onMount()`, destroy it in `onUnmount()`. No exceptions.

9. Composing Components

Components compose naturally. Build small, focused components and combine them:

```

// status-badge.ts
class StatusBadge extends Tina4Element {
    static props = { status: String };
    static styles = `
        :host { display: inline-block; }
        span { padding: 0.2rem 0.5rem; border-radius: 4px; font-size: 0.75rem; }
        .online { background: #dcfce7; color: #166534; }
        .offline { background: #fee2e2; color: #991b1b; }
    `;

    render() {
        return html`<span class=${this.prop('status')}>${this.prop('status')}</span>`;
    }
}
customElements.define('status-badge', StatusBadge);

// user-row.ts
class UserRow extends Tina4Element {
    static props = { name: String, email: String, status: String };
    static styles = `
        :host { display: flex; align-items: center; gap: 1rem; padding: 0.5rem 0; }
        .name { font-weight: 600; }
        .email { color: #6b7280; }
    `;
}

```

The Intelligent Native Application Framework


```

    `;

    render() {
      return html`
        <span class="name">${this.prop('name')}</span>
        <span class="email">${this.prop('email')}</span>
        <status-badge status=${this.prop('status')}></status-badge>
      `;
    }
  }
}
customElements.define('user-row', UserRow);

```

Use in a page:

```

html`
  <div>
    <user-row name="Alice" email="alice@test.com" status="online"></user-row>
    <user-row name="Bob" email="bob@test.com" status="offline"></user-row>
  </div>
`

```

10. The Store Pattern -- Shared Signals

A nav bar needs to know who is logged in. A dashboard needs the same data. A settings page can change it. Three components, one piece of state.

In React, you reach for Context or Redux. In Vue, you reach for Pinia. In tina4-js, you export signals from a module. That is it. No store library. No provider components. No boilerplate.

```

// store.ts
import { signal, computed } from 'tina4js';

export const user = signal<{ name: string; role: string } | null>(null, 'current-user');
export const isLoggedIn = computed(() => user.value !== null);
export const isAdmin = computed(() => user.value?.role === 'admin');

export function login(name: string, role: string) {
  user.value = { name, role };
}

export function logout() {
  user.value = null;
}

```

Any component can import and use these signals:

```

// nav-bar.ts
import { Tina4Element, html } from 'tina4js';
import { user, isLoggedIn, logout } from '../store';

class NavBar extends Tina4Element {
  static styles = `
    :host { display: flex; justify-content: space-between; padding: 1rem; }
    button { cursor: pointer; }
  `;

  render() {

```

```

return html`
  <span>My App</span>
  <div>
    ${() => isLoggedIn.value
      ? html`
        <span>Welcome, ${user}!</span>
        <button @click=${() => logout()}>Logout</button>
      `
      : html`<a href="/login">Login</a>`
    }
  </div>
`;
}
}
customElements.define('nav-bar', NavBar);

```

No boilerplate. No providers. No context wrappers. No `mapStateToProps`. Signals are values. Import them. Use them. Every component that imports the same signal shares the same state, and every one of them updates when that state changes.

11. Complete Example -- A Card Component

```
import { Tina4Element, html, signal } from 'tina4js';

class ProductCard extends Tina4Element {
  static props = {
    name: String,
    price: Number,
    image: String,
    instock: Boolean,
  };

  static styles = `
    :host { display: block; border: 1px solid #e5e7eb; border-radius: 8px; overflow: hidden; }
    img { width: 100%; height: 200px; object-fit: cover; }
    .content { padding: 1rem; }
    h3 { margin: 0 0 0.5rem; }
    .price { font-size: 1.25rem; font-weight: 700; color: #059669; }
    .out-of-stock { color: #dc2626; font-size: 0.875rem; }
    button {
      width: 100%; padding: 0.75rem; border: none; border-radius: 4px;
      background: #2563eb; color: white; font-size: 1rem; cursor: pointer;
      margin-top: 0.5rem;
    }
    button:disabled { background: #9ca3af; cursor: not-allowed; }
  `;

  render() {
    const quantity = signal(1);

    return html`
      <img src=${this.prop('image')} alt=${this.prop('name')} />
      <div class="content">
        <h3>${this.prop('name')}</h3>
        <p class="price">${this.prop('price')}</p>

        ${() => this.prop<boolean>('instock').value
          ? html`
              <div>
                <button @click=${() => { if (quantity.value > 1) quantity.value--; }}>-</button>
                <span>${quantity}</span>
                <button @click=${() => quantity.value++}>+</button>
              </div>
                <button @click=${() => {
                  this.emit('add-to-cart', {
                    detail: {
                      name: this.prop<string>('name').value,
                      quantity: quantity.value,
                    },
                  });
                }}>Add to Cart</button>
              `
            : html`<p class="out-of-stock">Out of Stock</p>`
        }
      </div>
    `;
  }
}
```

```
customElements.define('product-card', ProductCard);
```

Use it:

```
html`
  <product-card
    name="Wireless Mouse"
    price="29"
    image="/images/mouse.jpg"
    instock
    @add-to-cart=${(e: CustomEvent) => {
      console.log(`Added ${e.detail.quantity}x ${e.detail.name}`);
    }}
  ></product-card>
```

12. Wildcard Imports -- Registering Every Component

`customElements.define()` runs at module scope, so a component registers the moment its file is imported. The import is the registration. Nothing scans your project for components, which means an app with thirty of them needs thirty import lines somewhere, and the tag silently fails to upgrade the day someone forgets one.

Vite expands a wildcard instead. `import.meta.glob` takes a glob pattern and turns it into one import per matching file at build time:

```
// src/main.ts
import 'tina4js/debug';
import { router } from 'tina4js';

// Every file under src/components/ is imported, so every tag is defined.
import.meta.glob('./components/*.ts', { eager: true });

router.start({ target: '#root', mode: 'hash' });
```

Add `src/components/user-card.ts` and `<user-card>` works on the next reload. No list to update.

Use `**` to walk subdirectories: `./components/**/*.ts` also picks up `components/forms/text-field.ts`.

`{ eager: true }` is required. Without it the glob gives you an object of loader functions and no module body has run, so no tag is defined. Your components render as empty unknown elements:

```
import.meta.glob('./components/*.ts'); // WRONG -- loaders, never called
import.meta.glob('./components/*.ts', { eager: true }); // Correct -- modules executed
```

Order does not matter here, unlike routes. `customElements.define()` upgrades any matching element already in the DOM, so a component defined late still renders. Routes are the opposite case, and [Routing](#), Section 12 covers the trap.

Two constraints come from the bundler rather than from tina4-js:

- **The pattern must be a literal string.** Vite expands it while building, so it cannot be assembled from a variable at runtime.
- **Every match ships.** An eager glob imports the whole folder, so tree-shaking cannot drop a component you have not used yet. That is the trade for dropping the import list. Keep half-finished experiments outside the globbed directory.

Summary

Feature	How
Define a component	<code>class X extends Tina4Element</code>
Declare props	<code>static props = { name: String }</code>
Read a prop	<code>this.prop('name')</code> returns a signal
Scoped styles	<code>static styles = '...'</code>
Light DOM mode	<code>static shadow = false</code>
Fire events	<code>this.emit('name', { detail })</code>
After first render	<code>onMount()</code>
Before removal	<code>onUnmount()</code>
Register tag	<code>customElements.define('tag-name', Class)</code>
Shared state	Export signals from a module
Register every component	<code>import.meta.glob('./components/*.ts', { eager: true })</code>

Routing

Navigation Without the Router Library

A user clicks "Dashboard." The page goes white. A full HTML document loads from the server. The CSS re-parses. The JavaScript re-executes. The user waits.

Single-page applications fix this. The browser stays on one HTML page. JavaScript swaps content in and out. The URL changes, but the page never reloads. Navigation feels instant because it is -- only the content that changed gets replaced.

tina4-js gives you this in about 20 lines of routing code. Parameterized routes. Guards that protect pages. A 404 catch-all. Programmatic navigation. No separate router library to install.

1. How Routing Works

The tina4-js router follows three steps:

- You call `route(pattern, handler)` to register paths
- You call `router.start({ target, mode })` to start listening
- When the URL changes, the router finds the matching route, calls the handler, and renders the result into the target element

No file-based routing. No dynamic imports by convention. No data loaders. You register routes with explicit calls, and the router invokes your handler function when the URL matches.

2. Registering Routes

```
import { route, html } from 'tina4js';

route('/', () => html`<h1>Home</h1>`);
route('/about', () => html`<h1>About</h1>`);
route('/contact', () => html`<h1>Contact</h1>`);
```

Pattern is always the first argument. This is a convention across all tina4 frameworks. Do not swap the arguments.

The handler is a function that returns content. It can return:

- A `DocumentFragment` (from `html` tagged templates)
- A `Node` (any DOM node)
- A `string` (set as `innerHTML`)
- A `Promise` that resolves to any of the above (async routes)

3. Route Parameters

Use `{param}` syntax in patterns to capture URL segments:

```
route('/users/{id}', ({ id }) => {
  return html`<h1>User ${id}</h1>`;
});

route('/posts/{year}/{slug}', ({ year, slug }) => {
  return html`<h1>${slug} (${year})</h1>`;
});
```

Parameters are extracted from the URL and passed to the handler as a `Record<string, string>`. All values are strings -- cast them yourself if you need numbers:

```
route('/products/{id}', ({ id }) => {
  const productId = parseInt(id, 10);
  return html`<product-detail id="${productId}"></product-detail>`;
});
```

How Matching Works

The router converts `{param}` to `([^\/]++)` regex groups:

- `/users/{id}` matches `/users/42`, `/users/alice`, `/users/abc-123`
- `/posts/{year}/{slug}` matches `/posts/2024/my-post`
- It does not match `/users/` (trailing slash, no id) or `/users/42/edit` (extra segment)

Routes are checked in registration order. The first match wins. Put specific routes before general ones.

4. The Wildcard -- 404 Routes

```
route('*', () => html`
  <div>
    <h1>404</h1>
    <p>Page not found.</p>
    <a href="/">Go home</a>
  </div>
`);
```

The `*` pattern matches any path. Register it last so it only catches URLs that no other route matched.

5. Route Guards

A user types `/admin` into the address bar. They are not logged in. They should never see that page. Without guards, the handler runs, the admin panel renders, and your application has a security hole.

Guards protect routes. A guard runs before the handler. It can:

The Intelligent Native Application Framework

- Return `true` to allow navigation
- Return `false` to block navigation (nothing happens)
- Return a string to redirect to that path

```
import { route, html, signal, computed } from 'tina4js';

const token = signal<string | null>(null);
const isLoggedIn = computed(() => token.value !== null);

// Protected route
route('/dashboard', {
  guard: () => isLoggedIn.value || '/login',
  handler: () => html`<h1>Dashboard</h1>`,
});

// Login page
route('/login', () => {
  return html`
    <div>
      <h1>Login</h1>
      <button @click=${() => { token.value = 'abc123'; }}>
        Log In
      </button>
    </div>
  `;
});
```

When a user navigates to `/dashboard`:

- The guard runs: `isLoggedIn.value || '/login'`
- If `isLoggedIn` is `true`, the guard returns `true` and the handler renders
- If `isLoggedIn` is `false`, `false || '/login'` returns `'/login'` -- the router redirects

The redirect uses `navigate(path, { replace: true })`, so the protected URL does not appear in browser history. The user cannot press Back to get to the guarded page.

Admin Guard Pattern

```
const user = signal<{ role: string } | null>(null);

route('/admin', {
  guard: () => {
    if (!user.value) return '/login';
    if (user.value.role !== 'admin') return '/unauthorized';
    return true;
  },
  handler: () => html`<admin-panel></admin-panel>`,
});

---
```


6. Starting the Router

```
import { router } from 'tina4js';

router.start({
  target: '#root',
  mode: 'history',
});
```

target

A CSS selector for the element where route content renders. The router finds this element with `document.querySelector()`. If the element does not exist, the router throws.

```
<body>
  <nav>...</nav>
  <main id="root"></main>  <!-- routes render here -->
  <footer>...</footer>
</body>
```

mode

Two options:

Mode	URLs	Requires
'history'	/about , /users/42	Server-side URL rewriting
'hash'	/#/about , /#/users/42	Nothing -- works everywhere

Hash mode is the default in scaffolded projects. URLs carry a `#` prefix. No server configuration needed. Works on static hosts, GitHub Pages, S3, anywhere you can drop files.

History mode gives clean URLs without the hash. But the server must return [index.html](#) for all routes. When a user bookmarks <https://myapp.com/users/42> and loads it, the server needs to serve the SPA -- not search for a [/users/42](#) file that does not exist.

Vite dev server handles this automatically. For production, configure your web server:

```
# Nginx
location / {
  try_files $uri /index.html;
}

# Apache (.htaccess)
RewriteEngine On
RewriteRule ^index\.html$ - [L]
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . /index.html [L]
```

7. Programmatic Navigation

```
import { navigate } from 'tina4js';

// Push to history (user can press Back)
navigate('/dashboard');

// Replace current history entry (no Back)
navigate('/login', { replace: true });
```

Use `navigate()` in event handlers, after API calls, in guards -- anywhere you need to move the user to a different page.

```
const handleLogin = async () => {
  const success = await api.post('/auth/login', credentials);
  if (success) {
    navigate('/dashboard');
  }
};
```

Link Interception

The router automatically intercepts clicks on `<a>` tags with same-origin `href` attributes. You do not need a special `<Link>` component:

```
html`<a href="/about">About</a>`
// Clicking this navigates without a page reload
```

The router ignores:

- Links with `target="_blank"` or any `target` attribute
- Links with the `download` attribute
- Links with `rel="external"`
- Modified clicks (Ctrl+click, Cmd+click, Shift+click, Alt+click)
- Links to different origins (external URLs)

8. Route Change Events

Listen for navigation events:

```
import { router } from 'tina4js';

const unsubscribe = router.on('change', ({ path, params, pattern, durationMs }) => {
  console.log(`Navigated to ${path} (matched ${pattern}) in ${durationMs}ms`);
  console.log('Params:', params);
});

// Later, to stop listening:
unsubscribe();
```

The change event includes:

Property	Type	Description
----------	------	-------------

The Intelligent Native Application 4ramework

path	string	The current URL path
params	Record<string, string>	Extracted route parameters
pattern	string	The matched route pattern
durationMs	number	Time to render the route (ms)

Use this for analytics, breadcrumbs, active nav highlighting, or debugging:

```
// Analytics
router.on('change', ({ path }) => {
  analytics.pageView(path);
});

// Active nav highlighting
const currentPath = signal('/');
router.on('change', ({ path }) => {
  currentPath.value = path;
});

html`
  <nav>
    <a href="/" class=${() => currentPath.value === '/' ? 'active' : ''}>Home</a>
    <a href="/about" class=${() => currentPath.value === '/about' ? 'active' : ''}>About</a>
  </nav>
`
---
```

9. Async Routes

A user profile page needs data from the server. You cannot render the page until the data arrives. Route handlers can be async -- the router awaits the result before rendering:

```
route('/users/{id}', async ({ id }) => {
  const user = await api.get(`/users/${id}`);
  return html`
    <div>
      <h1>${user.name}</h1>
      <p>${user.email}</p>
    </div>
  `;
});
```

If the user navigates away before the async handler resolves, the stale result is discarded. The router tracks a version counter and renders only if the version matches. No race conditions. No stale data flashing on screen.

Loading States

For a better user experience, show a loading indicator:

```
route('/users/{id}', async ({ id }) => {
  const loading = signal(true);
  const user = signal<any>(null);

  // Show loading immediately
```

The Intelligent Native Application Framework

```

const view = html`
  <div>
    ${() => loading.value
      ? html`<p>Loading...</p>`
      : html`
        <h1>${() => user.value?.name}</h1>
        <p>${() => user.value?.email}</p>
      `}
  </div>
`;

// Fetch in background
api.get(`/users/${id}`).then(data => {
  user.value = data;
  loading.value = false;
});

return view;
});

```

10. Effect Cleanup on Route Change

You create a signal and an effect on one route. The user navigates away. Without cleanup, that effect keeps running -- updating state for a page that no longer exists, consuming memory for nodes that have been removed from the DOM.

The router handles this. When a route changes, effects created during the previous route's rendering are disposed. No memory leaks. No ghost updates.

```

route('/live', () => {
  const count = signal(0);

  // This effect is automatically cleaned up when the user navigates away
  effect(() => {
    const interval = setInterval(() => {
      count.value++;
    }, 1000);

    // But setInterval is NOT cleaned up automatically -- you need to handle that
  });

  return html`<p>Count: ${count}</p>`;
});

```

The router disposes the effect (unsubscribes from signals), but it cannot clean up timers, event listeners, or other resources you created outside the signal system. For those, use [onUnmount\(\)](#) in a component, or track cleanup manually. The rule: if the router did not create it, the router cannot destroy it.

11. Complete Example -- Multi-Page App

```
// src/store.ts
import { signal, computed } from 'tina4js';

export const token = signal<string | null>(null, 'auth-token');
export const isLoggedIn = computed(() => token.value !== null);

// src/routes/index.ts
import { route, navigate, html, signal } from 'tina4js';
import { token, isLoggedIn } from '../store';

// Public routes
route('/', () => html`
  <div>
    <h1>Home</h1>
    <nav>
      <a href="/about">About</a>
      ${() => isLoggedIn.value
        ? html`<a href="/dashboard">Dashboard</a>`
        : html`<a href="/login">Login</a>`}
    </nav>
  </div>
`);

route('/about', () => html`
  <div>
    <h1>About</h1>
    <a href="/">Back</a>
  </div>
`);

// Login
route('/login', () => {
  const email = signal('');
  const password = signal('');

  return html`
    <div>
      <h1>Login</h1>
      <form @submit=${(e: Event) => {
        e.preventDefault();
        // Fake login
        token.value = 'fake-jwt-token';
        navigate('/dashboard');
      }}>
        <input
          type="email"
          placeholder="Email"
          @input=${(e: Event) => { email.value = (e.target as HTMLInputElement).value; }}
        />
        <input
          type="password"
          placeholder="Password"
          @input=${(e: Event) => { password.value = (e.target as HTMLInputElement).value; }}
        />
        <button type="submit">Login</button>
      </form>
    </div>
  `;
});
```

```

    </div>
  `;
});

// Protected routes
route('/dashboard', {
  guard: () => isLoggedIn.value || '/login',
  handler: () => html`
    <div>
      <h1>Dashboard</h1>
      <p>You are logged in.</p>
      <button @click=${() => {
        token.value = null;
        navigate('/');
      }}>Logout</button>
    </div>
  `;
});

// 404
route('*', () => html`
  <div>
    <h1>404</h1>
    <a href="/">Go home</a>
  </div>
`);

// src/main.ts
import { router } from 'tina4js';
import './routes/index';

router.start({ target: '#root', mode: 'hash' });
---

```

12. Wildcard Imports -- Loading Every Route File

The example above keeps every route in one [src/routes/index.ts](#). That file grows. Split it per feature and the barrel becomes bookkeeping you have to remember:

```

// src/routes/index.ts -- one line per file, forever
import './home';
import './products';
import './checkout';
import './admin';

```

Forget a line and the route silently does not exist. Vite expands a wildcard for you instead. [import.meta.glob](#) takes a glob pattern and turns it into one import per matching file at build time:

```

// src/main.ts
import { router } from 'tina4js';

// Every file under src/routes/ is imported, so every route registers.
import.meta.glob('./routes/*.ts', { eager: true });

router.start({ target: '#root', mode: 'hash' });

```

This works because `route()` registers on import. Calling it appends to the router's table as the module body runs, so importing the file *is* registering its routes. There is no registry to populate and no return value to collect. Drop in `src/routes/reports.ts`, reload, and its routes are live.

Use `**` to walk subdirectories: `./routes/**/*.ts` also picks up `routes/admin/users.ts`.

`{ eager: true }` is required. Without it the glob gives you an object of loader functions and no module body has run, so nothing is registered and every path falls through:

```
import.meta.glob('./routes/*.ts'); // WRONG -- loaders, never called
import.meta.glob('./routes/*.ts', { eager: true }); // Correct -- modules executed
```

Keep the catch-all out of the glob

Section 4 said to register `route('*')` last, because the router walks its table in registration order and takes the first pattern that matches. A glob does not respect your intent about order: it imports the files it finds, and a catch-all sitting in the globbed folder can register before the real routes and swallow all of them. Every page renders your 404.

Register it explicitly, after the glob, where the order is yours to control:

```
// src/main.ts
import { route, router, html } from 'tina4js';

import.meta.glob('./routes/*.ts', { eager: true }); // no 404 file in here

route('*', () => html`<h1>404</h1><a href="/">Go home</a>`);

router.start({ target: '#root', mode: 'hash' });
```

Renaming the file so it sorts last works too, and breaks the day someone renames it back. Keep the catch-all out of the folder.

One constraint comes from the bundler rather than tina4-js: **the pattern must be a literal string**. Vite expands it while building, so it cannot be assembled from a variable at runtime.

Components register the same way. See [Components](#), Section 12.

Summary

	How	
for route	<code>route(pattern, handler)</code>	
params	<code>route('/users/{id}', ({ id }) => ...)</code>	
	<code>`route('/x', { guard: () => bool\`</code>	<code>string, handler })`</code>
404	<code>route('*', handler)</code>	
router	<code>router.start({ target: '#root', mode: 'history' })</code>	
re	<code>navigate('/path')</code>	
e (no back)	<code>navigate('/path', { replace: true })</code>	
or changes	<code>router.on('change', ({ path, params, pattern, durationMs }) => ...)</code>	
outes	Handler returns a Promise	
erception	Automatic for same-origin <code><a href></code>	
very route file	<code>import.meta.glob('./routes/*.ts', { eager: true })</code>	

API

Talking to Your Backend

Your frontend renders a list of users. The data lives in a database. Between your signal and that database sits an HTTP request -- and a surprising amount of ceremony. Auth headers. CSRF tokens. Token rotation. Error handling. JSON parsing. In most projects, you install Axios or ky, configure interceptors, and write wrapper functions before you make your first call.

tina4-js includes all of this. One import. No extra packages. If you use a tina4-php or tina4-python backend, auth and CSRF protection wire up with a single configuration flag.

1. The API Client

The built-in HTTP client wraps `fetch()` with the features you need in every application:

- Automatic **Authorization: Bearer** headers
- Token rotation via **FreshToken** response headers
- CSRF **formToken** injection in POST/PUT/PATCH/DELETE bodies
- Per-request headers and query params
- Request and response interceptors
- JSON parsing by default

```
import { api } from 'tina4js';
```

One import. No Axios. No ky. No dependencies. The API client ships with the framework.

2. Configuration

Call `api.configure()` once at app startup:

```
api.configure({
  baseUrl: 'https://api.example.com',
  auth: true,
});
```

Options

Option	Type	Default	Description
<code>baseUrl</code>	<code>string</code>	<code>''</code>	Prepended to all request paths
<code>auth</code>	<code>boolean</code>	<code>false</code>	Enable Bearer token and formToken
<code>tokenKey</code>	<code>string</code>	<code>'tina4_token'</code>	localStorage key for the auth token
<code>headers</code>	<code>Record<string, string></code>	<code>{}</code>	Default headers on every request

A typical setup for a tina4-php backend:

```
api.configure({
  baseUrl: '/api',
  auth: true,
  headers: {
    'Accept': 'application/json',
  },
});
```

During development with Vite, proxy API calls to your backend:

```
// vite.config.ts
export default defineConfig({
  server: {
    proxy: {
      '/api': 'http://localhost:7145',
    },
  },
});
```

3. Making Requests

GET

```
const users = await api.get('/users');
```

With query parameters:

```
const users = await api.get('/users', {
  params: { page: 2, limit: 20, search: 'alice' },
});
// Request: GET /users?page=2&limit=20&search=alice
```

Parameters are automatically URL-encoded.

POST

```
await api.post('/users', {
  name: 'Alice',
  email: 'alice@example.com',
  role: 'editor',
});
```

The body is serialized as JSON. The `Content-Type: application/json` header is added automatically.

PUT (Full Replace)

```
await api.put('/users/42', {
  name: 'Alice',
  email: 'alice@new-email.com',
  role: 'admin',
});
```

PATCH (Partial Update)

```
await api.patch('/users/42', {
  role: 'admin',
});
```

DELETE

```
await api.delete('/users/42');
```

4. RequestOptions

Every method accepts an optional `RequestOptions` object:

```
interface RequestOptions {
  headers?: Record<string, string>;
  params?: Record<string, string | number | boolean>;
}
```

Per-Request Headers

```
const data = await api.get('/reports/export', {
  headers: {
    'Accept': 'text/csv',
    'X-Custom': 'value',
  },
});
```

Per-request headers merge with (and override) the default headers from `configure()`.

Query Params on Any Method

```
await api.post('/search', { query: 'tina4' }, {
  params: { format: 'detailed', lang: 'en' },
});
// POST /search?format=detailed&lang=en
// Body: { "query": "tina4" }
```

5. Auth Flow

One flag -- `auth: true` -- activates three mechanisms that handle authentication for every request your application makes:

1. Bearer Token

Every request includes an `Authorization` header:

```
Authorization: Bearer <token>
```

The token is read from `localStorage` using the `tokenKey` (default: `'tina4_token'`).

2. formToken

For POST, PUT, PATCH, and DELETE requests, a `formToken` property is injected into the JSON body:

```
// You send:
await api.post('/users', { name: 'Alice' });

// Actual request body:
{ "name": "Alice", "formToken": "the-current-token" }
```

This is CSRF protection for tina4-php and tina4-python backends. They validate the `formToken` on every write operation.

3. FreshToken Rotation

When the server responds with a `FreshToken` header, the client automatically stores it:

```
HTTP/1.1 200 OK
FreshToken: new-jwt-token-here
```

The new token replaces the old one in `localStorage` and is used for subsequent requests. This allows the backend to rotate tokens on every request or on a schedule.

Login Example

```
import { api, navigate, signal } from 'tina4js';

const loginError = signal<string | null>(null);

async function login(email: string, password: string) {
  try {
    const result = await api.post<{ token: string }>('/auth/login', {
      email,
      password,
    });

    // Store the token (the API client reads it from localStorage)
    localStorage.setItem('tina4_token', result.token);

    navigate('/dashboard');
  } catch (err: any) {
    loginError.value = err.data?.message ?? 'Login failed';
  }
}
```

6. Error Handling

A 404. A 500. A network timeout. Every API call can fail, and your application needs to handle the failure without crashing. When the server returns a non-2xx status, [api](#) throws the response object:

```
try {
  await api.get('/users/999');
} catch (err: any) {
  console.log(err.status); // 404
  console.log(err.data);   // { message: "User not found" }
  console.log(err.ok);     // false
}
```

The thrown object has the [ApiResponse](#) shape:

```
interface ApiResponse<T = unknown> {
  status: number;
  data: T;
  ok: boolean;
  headers: Headers;
}
```

Pattern: Centralized Error Handling with Interceptors

```
api.intercept('response', (response) => {
  if (response.status === 401) {
    localStorage.removeItem('tina4_token');
    navigate('/login');
  }
  if (response.status === 403) {
    navigate('/unauthorized');
  }
});
```

7. Interceptors

Every request passes through a pipeline. Interceptors let you insert logic at two points: before the request leaves and after the response arrives. Add a client version header to every request. Unwrap a response envelope. Log slow calls. Redirect on 401. Interceptors handle all of it in one place.

Request Interceptor

```
api.intercept('request', (config) => {
  // Add a custom header to every request
  config.headers['X-Client-Version'] = '1.0.0';

  // Add a timestamp
  config.headers['X-Request-Time'] = new Date().toISOString();
});
```

The `config` parameter is a `RequestInit` with a `headers` record. Modify it in place or return a new object.

Response Interceptor

```
api.intercept('response', (response) => {
  // Log slow requests
  if (response.status === 200) {
    console.log(`API: ${response.status}`);
  }

  // Transform data
  if (response.data && typeof response.data === 'object') {
    // unwrap a common envelope
    const envelope = response.data as any;
    if (envelope.data) {
      return { ...response, data: envelope.data };
    }
  }
});
```

Return a modified response to transform the data before it reaches your application code. Return nothing (or `undefined`) to pass the response through unchanged.

8. Real Example: CRUD Data Table

A list of users. Add one. Edit one. Delete one. This is the bread and butter of business applications, and it exercises every feature of the API client -- GET for loading, POST for creating, PUT for updating, DELETE for removing, batch for coordinating signal updates:

```
import { signal, computed, html, api, batch } from 'tina4js';

interface User {
  id: number;
  name: string;
  email: string;
}

function usersPage() {
  const users = signal<User[]>([], 'users');
  const loading = signal(true, 'users-loading');
  const editingId = signal<number | null>(null);
  const editName = signal('');
  const editEmail = signal('');

  // Load users
  async function loadUsers() {
    loading.value = true;
    try {
      const data = await api.get<User[]>('/users');
      users.value = data;
    } finally {
      loading.value = false;
    }
  }
}
```

```

// Create
async function createUser(name: string, email: string) {
  const newUser = await api.post<User>('/users', { name, email });
  users.value = [...users.value, newUser];
}

// Update
async function updateUser(id: number) {
  const updated = await api.put<User>(`/users/${id}`, {
    name: editName.value,
    email: editEmail.value,
  });
  batch(() => {
    users.value = users.value.map(u => u.id === id ? updated : u);
    editingId.value = null;
  });
}

// Delete
async function deleteUser(id: number) {
  await api.delete(`/users/${id}`);
  users.value = users.value.filter(u => u.id !== id);
}

// Start editing
function startEdit(user: User) {
  batch(() => {
    editingId.value = user.id;
    editName.value = user.name;
    editEmail.value = user.email;
  });
}

// Initial load
loadUsers();

return html`
  <div>
    <h1>Users</h1>

    ${() => loading.value
      ? html`<p>Loading...</p>`
      : html`
        <table>
          <thead>
            <tr><th>Name</th><th>Email</th><th>Actions</th></tr>
          </thead>
          <tbody>
            ${() => users.value.map(user => html`
              <tr>
                ${() => editingId.value === user.id
                  ? html`
                    <td>
                      <input .value=${editName}>
                      @input=${(e: Event) => { editName.value = (e.target as HTMLInputElement).value; }}
                    </td>
                    <td>
                      <input .value=${editEmail}>
                      @input=${(e: Event) => { editEmail.value = (e.target as HTMLInputElement).value; }}
                    </td>
                  </html>
                : html`
                  <td>
                    <button>Edit</button>
                  </td>
                </html>
            `}
            </tr>
          </tbody>
        </table>
      `}
  `;

```

```

        </td>
        <td>
            <button @click=${() => updateUser(user.id)}>Save</button>
            <button @click=${() => { editingId.value = null; }}>Cancel</button>
        </td>
    `
    : html`
        <td>${user.name}</td>
        <td>${user.email}</td>
        <td>
            <button @click=${() => startEdit(user)}>Edit</button>
            <button @click=${() => deleteUser(user.id)}>Delete</button>
        </td>
    `
    }
</tr>
`))
</tbody>
</table>
}

<h2>Add User</h2>
<form @submit=${(e: Event) => {
    e.preventDefault();
    const form = e.target as HTMLFormElement;
    const formData = new FormData(form);
    createUser(formData.get('name') as string, formData.get('email') as string);
    form.reset();
}}>
    <input name="name" placeholder="Name" required />
    <input name="email" type="email" placeholder="Email" required />
    <button type="submit">Add</button>
</form>
</div>
`
;
}
---

```

9. File Upload

The API client sends JSON by default. Files are not JSON. For file uploads, use `fetch()` with the same base URL and auth token. The browser sets the correct `Content-Type` with multipart boundaries -- do not set it yourself:

```

async function uploadFile(file: File) {
    const formData = new FormData();
    formData.append('file', file);

    const token = localStorage.getItem('tina4_token');

    const response = await fetch('/api/upload', {
        method: 'POST',
        headers: {
            ...(token ? { Authorization: `Bearer ${token}` } : {}),
        },
        body: formData, // Do NOT set Content-Type -- browser sets it with boundary
    });
}

```

The Intelligent Native Application Framework


```

    });

    return response.json();
  }
}

```

Use in a template:

```

html`
  <input type="file" @change=${(e: Event) => {
    const file = (e.target as HTMLInputElement).files?.[0];
    if (file) uploadFile(file);
  }} />
`

```

Summary

What	How
Configure	<code>api.configure({ baseUrl, auth, tokenKey, headers })</code>
GET	<code>api.get(path, options?)</code>
POST	<code>api.post(path, body?, options?)</code>
PUT	<code>api.put(path, body?, options?)</code>
PATCH	<code>api.patch(path, body?, options?)</code>
DELETE	<code>api.delete(path, options?)</code>
Query params	<code>{ params: { key: value } }</code>
Per-request headers	<code>{ headers: { key: value } }</code>
Auth token	Automatic Bearer header when <code>auth: true</code>
Token rotation	Automatic via FreshToken response header
CSRF protection	Automatic <code>formToken</code> in POST/PUT/PATCH/DELETE
Error handling	Non-2xx throws <code>ApiResponse</code>
Request interceptor	<code>api.intercept('request', fn)</code>
Response interceptor	<code>api.intercept('response', fn)</code>

WebSocket

Real-Time Without the Headache

A chat message arrives. A stock price changes. A deployment finishes. A sensor reading spikes. The data exists on the server right now, but your user stares at stale numbers until they refresh the page.

HTTP requests are pull. You ask, the server answers. WebSocket is push. The server sends data the moment it exists. The user sees it the moment it arrives.

The raw WebSocket API gives you this, but it hands you a connection object and wishes you luck. Reconnection? Write it yourself. State tracking? Manual. Feeding messages into your reactive UI? Also manual.

tina4-js wraps all of it. One function call opens the connection. Five reactive signals track the state. A pipe function streams messages into your signals with a reducer. Auto-reconnect handles dropped connections with exponential backoff.

1. The WebSocket Client

The tina4-js WebSocket client adds three things on top of the browser API:

- **Reactive signals** -- `status`, `connected`, `lastMessage`, `error`, `reconnectCount`
- **Auto-reconnect** -- exponential backoff, configurable attempts
- **Signal piping** -- stream messages into signals with a reducer

```
import { ws } from 'tina4js';
```

2. Connecting

```
const socket = ws.connect('wss://api.example.com/ws');
```

One line. The connection starts. The `socket` object gives you reactive state and methods to send, listen, pipe, and close.

With Options

```
const socket = ws.connect('wss://api.example.com/ws', {
  reconnect: true,           // auto-reconnect on disconnect (default: true)
  reconnectDelay: 1000,     // initial delay in ms (default: 1000)
  reconnectMaxDelay: 30000, // max delay with backoff (default: 30000)
  reconnectAttempts: 10,    // max attempts (default: Infinity)
  protocols: 'chat-v1',    // WebSocket sub-protocols
});
```

Options Reference

Option	Type	Default	Description	
<code>reconnect</code>	<code>boolean</code>	<code>true</code>	Enable auto-reconnect	
<code>reconnectDelay</code>	<code>number</code>	<code>1000</code>	Initial reconnect delay (ms)	
<code>reconnectMaxDelay</code>	<code>number</code>	<code>30000</code>	Max delay after backoff (ms)	
<code>reconnectAttempts</code>	<code>number</code>	<code>Infinity</code>	Max reconnect attempts	
<code>protocols</code>	<code>`string \</code>	<code>string[]`</code>	<code>[]</code>	WebSocket sub-protocols

3. Reactive Signals

The socket exposes five signals:

```
const socket = ws.connect('wss://api.example.com/ws');

socket.status;           // Signal<'connecting' | 'open' | 'closed' | 'reconnecting'>
socket.connected;        // Signal<boolean>
socket.lastMessage;      // Signal<unknown>
socket.error;             // Signal<Event | null>
socket.reconnectCount;    // Signal<number>
```

Use them directly in templates:

```
import { html } from 'tina4js';

html`
  <div>
    <p>Status: ${socket.status}</p>
    <p>Connected: ${() => socket.connected.value ? 'Yes' : 'No'}</p>
    <p>Reconnect attempts: ${socket.reconnectCount}</p>
    <p>Last message: ${() => JSON.stringify(socket.lastMessage.value)}</p>
  </div>`
```

These update the moment the connection state changes. No polling. No callbacks. No manual state management. Drop them in a template and the UI stays current.

Status Flow

connecting -> open -> closed -> reconnecting -> open -> ...

- `connecting` -- initial connection attempt
- `open` -- connected and ready

- `closed` -- disconnected
- `reconnecting` -- waiting to reconnect (auto-reconnect active)

4. Sending Messages

```
// Send a string
socket.send('hello');

// Send an object (auto-JSON.stringify)
socket.send({ type: 'chat', message: 'Hello!' });

// Send with a specific structure
socket.send({
  action: 'subscribe',
  channel: 'notifications',
});
```

Objects are automatically serialized with `JSON.stringify()`. Strings are sent as-is.

If the socket is not connected, `send()` throws:

```
try {
  socket.send('hello');
} catch (e) {
  console.error('Not connected');
}
```

Check the `connected` signal before sending:

```
if (socket.connected.value) {
  socket.send(data);
}
```

5. Listening for Events

The `on()` method returns an unsubscribe function:

```
// Messages
const unsub = socket.on('message', (data) => {
  console.log('Received:', data);
});

// Connection opened
socket.on('open', () => {
  console.log('Connected!');
  socket.send({ type: 'auth', token: 'my-token' });
});

// Connection closed
socket.on('close', (code, reason) => {
  console.log(`Closed: ${code} ${reason}`);
});

// Errors
```

```
socket.on('error', (event) => {
  console.error('WebSocket error:', event);
});

// Later, unsubscribe
unsub();
```

Message Parsing

Messages are automatically JSON-parsed. If the message is valid JSON, you get an object. If not, you get the raw string:

```
socket.on('message', (data) => {
  // data is already parsed if it was JSON
  if (typeof data === 'object') {
    console.log(data.type, data.payload);
  } else {
    console.log('Raw string:', data);
  }
});
```

6. pipe() -- Stream Messages Into Signals

This is where WebSocket meets reactivity. A message arrives. It flows into a signal through a reducer function. The signal updates. Every subscriber -- text nodes, computed values, effects -- responds. The data moves from server to DOM without a single manual step.

`pipe()` connects a WebSocket message stream to a signal with a reducer:

```
import { signal } from 'tina4js';

const messages = signal<string[]>([]);

socket.pipe(messages, (msg, current) => {
  const chatMsg = msg as { text: string };
  return [...current, chatMsg.text];
});
```

Every time a message arrives:

- The reducer runs with `(message, currentSignalValue)`
- The return value is assigned to `messages.value`
- Everything subscribed to `messages` updates

No event listener that grabs a signal reference and mutates it. No manual state management. The pipe is declarative, testable, and composable. Data flows in one direction: server to socket to reducer to signal to DOM.

Multiple Pipes

You can pipe the same socket to multiple signals:

```
const notifications = signal<any[]>([]);
const userCount = signal(0);
```

```
// Route messages to different signals based on type
socket.pipe(notifications, (msg, current) => {
  const m = msg as { type: string; data: any };
  if (m.type === 'notification') return [...current, m.data];
  return current; // ignore non-notification messages
});

socket.pipe(userCount, (msg, current) => {
  const m = msg as { type: string; count: number };
  if (m.type === 'user_count') return m.count;
  return current;
});
```

Unsubscribing

`pipe()` returns an unsubscribe function:

```
const unsub = socket.pipe(messages, reducer);

// Later
unsub();
```

7. Auto-Reconnect

By default, the socket reconnects automatically when the connection drops. The delay doubles after each attempt (exponential backoff):

- Attempt 1: 1000ms delay
- Attempt 2: 2000ms delay
- Attempt 3: 4000ms delay
- Attempt 4: 8000ms delay
- ...up to `reconnectMaxDelay` (default: 30000ms)

When reconnection succeeds:

- `status` goes to `'open'`
- `connected` becomes `true`
- `reconnectCount` resets to `0`
- The delay resets to the initial value

Track reconnection in the UI:

```
html`
  ${() => socket.status.value === 'reconnecting'
    ? html`<div class="banner">
      Reconnecting... (attempt ${socket.reconnectCount})
    </div>`
    : null
  }
`
```

Disabling Reconnect

```
const socket = ws.connect('wss://api.example.com/ws', {
  reconnect: false,
});
```

Limiting Attempts

```
const socket = ws.connect('wss://api.example.com/ws', {
  reconnectAttempts: 5, // give up after 5 tries
});
```

8. Closing the Connection

```
socket.close();
```

An intentional close stops reconnecting. The status goes to `'closed'` and stays there. The user logs out. The component unmounts. The page no longer needs live data. Call `close()` and the connection ends.

With code and reason:

```
socket.close(1000, 'User logged out');
```

9. Real Example: Chat Room

Two users. One room. Messages appear the moment they are sent. The connection status shows at the top. The input disables when the socket disconnects. Everything is reactive -- signals drive every piece of state.

```
import { signal, html, ws, batch } from 'tina4js';

function chatRoom() {
  const messages = signal<{ user: string; text: string; time: string }[]>([]);
  const input = signal('');
  const username = signal('Anonymous');
  const socket = ws.connect('wss://api.example.com/chat');

  // Pipe incoming messages into the messages signal
  socket.pipe(messages, (msg, current) => {
    const m = msg as { type: string; user: string; text: string; time: string };
    if (m.type === 'chat') {
      return [...current, { user: m.user, text: m.text, time: m.time }];
    }
    return current;
  });

  // Send on enter
  const sendMessage = () => {
    const text = input.value.trim();
    if (!text || !socket.connected.value) return;

    socket.send({
      type: 'chat',
```

The Intelligent Native Application Framework

```

        user: username.value,
        text,
    });
    input.value = '';
};

return html`
    <div>
        <h1>Chat</h1>

        ${() => !socket.connected.value
            ? html`<p>Connecting...</p>`
            : null
        }

        <div style="height: 400px; overflow-y: auto; border: 1px solid #e5e7eb; padding: 1rem;">
            ${() => messages.value.map(msg => html`
                <div>
                    <strong>${msg.user}</strong>
                    <span style="color: #6b7280; font-size: 0.75rem">${msg.time}</span>
                    <p>${msg.text}</p>
                </div>
            `)}
        </div>

        <form @submit=${(e: Event) => { e.preventDefault(); sendMessage(); }}>
            <input
                type="text"
                placeholder="Type a message..."
                .value=${input}
                @input=${(e: Event) => { input.value = (e.target as HTMLInputElement).value; }}
                ?disabled=${() => !socket.connected.value}
            />
            <button type="submit" ?disabled=${() => !socket.connected.value}>
                Send
            </button>
        </form>
    </div>
`;
}

```

10. Real Example: Live Dashboard

A server streams metrics every second. Active users. Requests per second. Error rate. CPU usage. Alerts arrive when thresholds are exceeded. The dashboard updates without a single API poll -- the data pushes itself to the screen.

```

import { signal, computed, html, ws } from 'tina4js';

function liveDashboard() {
    const metrics = signal({
        activeUsers: 0,
        requestsPerSecond: 0,
        errorRate: 0,
        cpuUsage: 0,
    });

```



```

const alerts = signal<string[]>([]);

const socket = ws.connect('wss://api.example.com/metrics');

// Pipe metrics updates
socket.pipe(metrics, (msg, current) => {
  const m = msg as { type: string; data: any };
  if (m.type === 'metrics') {
    return { ...current, ...m.data };
  }
  return current;
});

// Pipe alerts
socket.pipe(alerts, (msg, current) => {
  const m = msg as { type: string; message: string };
  if (m.type === 'alert') {
    return [...current.slice(-9), m.message]; // keep last 10
  }
  return current;
});

const cpuColor = computed(() => {
  const cpu = metrics.value.cpuUsage;
  if (cpu > 80) return '#dc2626';
  if (cpu > 50) return '#f59e0b';
  return '#059669';
});

return html`
  <div>
    <h1>Live Dashboard</h1>
    <p>Status: ${socket.status}</p>

    <div style="display: grid; grid-template-columns: repeat(4, 1fr); gap: 1rem;">
      <div class="card">
        <h3>Active Users</h3>
        <p style="font-size: 2rem">${() => metrics.value.activeUsers}</p>
      </div>
      <div class="card">
        <h3>Requests/s</h3>
        <p style="font-size: 2rem">${() => metrics.value.requestsPerSecond}</p>
      </div>
      <div class="card">
        <h3>Error Rate</h3>
        <p style="font-size: 2rem">${() => metrics.value.errorRate}%</p>
      </div>
      <div class="card">
        <h3>CPU</h3>
        <p style="font-size: 2rem; color: ${cpuColor}">
          ${() => metrics.value.cpuUsage}%
        </p>
      </div>
    </div>

    <h2>Alerts</h2>
    <ul>
      ${() => alerts.value.map(alert => html`
        <li style="color: #dc2626">${alert}</li>
      `)}
    </ul>
  `

```

```

        `)}
      </ul>
    </div>
  `;
}
---

```

Summary

<code>connect(url, options?)</code>		
<code>c.status</code> -- <code>`connecting` \</code>	<code>'open' \</code>	<code>'closed' \</code>
<code>c.connected</code> -- boolean		
<code>c.lastMessage</code>		
<code>c.error</code>		
<code>c.reconnectCount</code>		
<code>c.send(data)</code> -- auto-stringify		
<code>c.on('message' \</code>	<code>'open' \</code>	<code>'close' \</code>
<code>c.pipe(signal, (msg, current) \</code> <code>Value)</code>		
<code>c.close(code?, reason?)</code> -- reconnect		
default, exponential backoff		

SSE / NDJSON Streaming

Streaming Without WebSocket

An AI chatbot types back one token at a time. A dashboard counter ticks up as orders arrive. A notification feed shows events the moment they happen. The data flows from server to client, but you don't need the complexity of WebSocket for it.

Server-Sent Events (SSE) is the browser's built-in streaming protocol. One-directional: server pushes, client listens. It reconnects automatically. It works over HTTP/2. It's simpler than WebSocket for every case where you don't need to send data back.

NDJSON (Newline-Delimited JSON) is the same idea over a regular fetch response. One JSON object per line, streamed as the server generates them. This is how most AI APIs stream tokens, and it supports POST requests and custom headers, which SSE's EventSource does not.

tina4-js wraps both in a single module. One function call opens the stream. Six reactive signals track the state. A pipe function accumulates messages into your signals with a reducer. Auto-reconnect handles dropped connections.

1. The SSE Client

The tina4-js SSE client provides:

- **Dual mode:** native EventSource for SSE, fetch+ReadableStream for NDJSON
- **Reactive signals:** `status`, `connected`, `lastMessage`, `lastEvent`, `error`, `reconnectCount`
- **Auto-reconnect:** exponential backoff, configurable attempts
- **Signal piping:** stream messages into signals with a reducer

```
import { sse } from 'tina4js';
```

2. EventSource Mode (Default)

```
const stream = sse.connect('/api/events');
```

One line. The browser opens an EventSource connection. Messages arrive as the server sends them.

```
import { sse, effect } from 'tina4js';

const stream = sse.connect('/api/events');

effect(() => {
  console.log('Status:', stream.status.value);
});

effect(() => {
```

The Intelligent Native Application 4ramework

```

    if (stream.lastMessage.value) {
      console.log('Received:', stream.lastMessage.value);
    }
  });
}

```

3. Fetch Mode (NDJSON)

EventSource only supports GET requests with no custom headers. For POST requests, Bearer tokens, or NDJSON streaming, use fetch mode:

```

const stream = sse.connect('/api/chat', {
  mode: 'fetch',
  method: 'POST',
  headers: { 'Authorization': 'Bearer eyJ...' },
  body: { prompt: 'Explain signals in tina4-js' },
});

```

The client reads the response body as a stream, splits on newlines, and parses each line as JSON. Every parsed object becomes a message.

4. Options

```

const stream = sse.connect('/api/events', {
  mode: 'eventsource', // 'eventsource' (default) or 'fetch'
  method: 'GET',       // HTTP method (fetch mode only)
  headers: {},         // Custom headers (fetch mode only)
  body: undefined,     // Request body (fetch mode only, auto-stringified)
  reconnect: true,     // Auto-reconnect on disconnect
  reconnectDelay: 1000, // Initial delay (ms)
  reconnectMaxDelay: 30000, // Max delay after backoff (ms)
  reconnectAttempts: Infinity, // Max attempts
  events: [],          // Named SSE events (eventsource mode only)
  json: true,          // Auto-parse JSON (default: true)
});

```

Option	Default	Description
mode	'eventsource'	Transport: native EventSource or fetch+ReadableStream
method	'GET'	HTTP method (fetch mode only)
headers	{}	Custom headers (fetch mode only)
body	undefined	Request body (fetch mode, auto JSON.stringify)

<code>reconnect</code>	<code>true</code>	Auto-reconnect on disconnect
<code>reconnectDelay</code>	<code>1000</code>	Initial reconnect delay in ms
<code>reconnectMaxDelay</code>	<code>30000</code>	Max delay for exponential backoff
<code>reconnectAttempts</code>	<code>Infinity</code>	Max reconnect attempts
<code>events</code>	<code>[]</code>	Named SSE events to listen for
<code>json</code>	<code>true</code>	Auto-parse messages as JSON

5. Reactive Signals

Every stream exposes six reactive signals:

```
const stream = sse.connect('/api/events');

stream.status      // Signal<'connecting' | 'open' | 'closed' | 'reconnecting'>
stream.connected   // Signal<boolean>
stream.lastMessage // Signal<unknown> - last parsed message
stream.lastEvent   // Signal<string | null> - SSE event name or null
stream.error       // Signal<Event | Error | null>
stream.reconnectCount // Signal<number>
```

Use them in effects, computed values, or html templates:

```
effect(() => {
  if (stream.connected.value) {
    console.log('Stream is live');
  }
});
```

6. Event Handlers

Register listeners for stream events. Every handler returns an unsubscribe function:

```
const unsub = stream.on('message', (data, event?) => {
  console.log('Data:', data);
  console.log('Event name:', event); // SSE event name or undefined
});

stream.on('open', () => console.log('Connected'));
stream.on('close', () => console.log('Disconnected'));
stream.on('error', (err) => console.error('Error:', err));

// Stop listening
unsub();
```

7. Named Events (EventSource Mode)

SSE supports named events. By default, EventSource only listens for unnamed `message` events. Pass event names in the options to listen for specific types:

```
const stream = sse.connect('/api/feed', {
  events: ['user_joined', 'message', 'user_left'],
});

stream.on('message', (data, event) => {
  switch (event) {
    case 'user_joined':
      console.log(`${data.name} joined`);
      break;
    case 'message':
      console.log(`${data.author}: ${data.text}`);
      break;
    case 'user_left':
      console.log(`${data.name} left`);
      break;
  }
});

// The lastEvent signal tracks the most recent event name
effect(() => console.log('Last event type:', stream.lastEvent.value));
```

8. Pipe to Signal

The pipe pattern streams messages into a signal through a reducer. This is the same pattern as the WebSocket module:

```
import { sse, signal } from 'tina4js';

const messages = signal([]);

const stream = sse.connect('/api/notifications');
stream.pipe(messages, (msg, current) => [...current, msg]);

// messages.value grows as notifications arrive
effect(() => {
  console.log(`${messages.value.length} notifications`);
});
```

Pipe returns an unsubscribe function:

```
const unsub = stream.pipe(messages, (msg, current) => [...current, msg]);

// Stop piping
unsub();
```

9. Auto-Reconnect

In EventSource mode, the browser handles reconnection natively. If the connection is fully closed, tina4-js schedules manual reconnection with exponential backoff.

In fetch mode, tina4-js handles all reconnection:

```
const stream = sse.connect('/api/stream', {
  mode: 'fetch',
  reconnect: true,
  reconnectDelay: 1000,      // Start at 1s
  reconnectMaxDelay: 30000,  // Cap at 30s
  reconnectAttempts: 10,     // Give up after 10 tries
});

effect(() => {
  if (stream.status.value === 'reconnecting') {
    console.log(`Reconnect attempt ${stream.reconnectCount.value}`);
  }
});
```

10. Closing

```
stream.close();
```

This stops the connection and prevents reconnection. In EventSource mode it calls [source.close\(\)](#). In fetch mode it aborts the fetch request.

11. SSE vs WebSocket

	SSE	WebSocket
Direction	Server → Client	Bidirectional
Protocol	HTTP	WS/WSS
Reconnect	Built-in (EventSource)	Manual (tina4-js handles it)
Headers	No (EventSource) / Yes (fetch mode)	Subprotocols only
POST body	No (EventSource) / Yes (fetch mode)	N/A
Binary data	No	Yes
HTTP/2 multiplexing	Yes	No
Use case	Notifications, feeds, AI streaming	Chat, gaming, live collaboration

Rule of thumb: If the client only needs to receive, use SSE. If the client needs to send too, use WebSocket.

12. Real-World Example: AI Chat Streaming

```
import { sse, signal, html } from 'tina4js';

const messages = signal([]);
const input = signal('');
const streaming = signal(false);

async function sendMessage() {
  const prompt = input.value.trim();
  if (!prompt) return;

  // Add user message
  messages.value = [...messages.value, { role: 'user', text: prompt }];
  input.value = '';

  // Add empty assistant message
  messages.value = [...messages.value, { role: 'assistant', text: '' }];
  streaming.value = true;

  // Stream tokens
  const stream = sse.connect('/api/chat', {
    mode: 'fetch',
    method: 'POST',
    headers: { 'Authorization': `Bearer ${localStorage.getItem('token')}` },
    body: { prompt },
  });

  stream.on('message', (data) => {
    const token = data.token || data;
    const msgs = [...messages.value];
    const last = msgs[msgs.length - 1];
    msgs[msgs.length - 1] = { ...last, text: last.text + token };
    messages.value = msgs;
  });

  stream.on('close', () => {
    streaming.value = false;
  });
}

const view = html`
<div class="chat">
  ${() => messages.value.map(m => html`
    <div class="message ${m.role}">
      <strong>${m.role}</strong> ${m.text}
    </div>
  `)}
  <div class="input-bar">
    <input
      type="text"
      .value=${input}
      @input=${(e) => { input.value = e.target.value; }}
      @keydown=${(e) => { if (e.key === 'Enter') sendMessage(); }}
      ?disabled=${streaming}
    />
    <button @click=${sendMessage} ?disabled=${streaming}>Send</button>
  </div>
</div>`
```

```
`;  
---
```

13. Real-World Example: Live Notification Feed

```
import { sse, signal, html } from 'tina4js';  
  
const notifications = signal([]);  
  
const stream = sse.connect('/api/notifications', {  
  events: ['info', 'warning', 'error'],  
});  
  
stream.pipe(notifications, (msg, current) => {  
  return [{ ...msg, event: stream.lastEvent.value, time: new Date() }, ...current].slice(0, 50);  
});  
  
const view = html`  
  <div class="feed">  
    <h2>Notifications ${() => stream.connected.value ? '(live)' : '(disconnected)'}</h2>  
    ${() => notifications.value.map(n => html`  
      <div class="notification ${n.event}">  
        <span class="badge">${n.event}</span>  
        ${n.message}  
        <small>${n.time.toLocaleTimeString()}</small>  
      </div>  
    `)}  
  </div>  
`;  
---
```

Bundle Size

Module	Raw	Gzipped
SSE	3.42 KB	1.30 KB

Import only what you need:

```
import { sse } from 'tina4js/sse'; // 1.30 KB gzip  
---
```

Summary

Task	Code
Connect (EventSource)	<code>sse.connect('/events')</code>
Connect (NDJSON/POST)	<code>sse.connect('/api', { mode: 'fetch', method: 'POST', body: {...} })</code>
Read status	<code>stream.status.value</code>
Listen for messages	<code>stream.on('message', (data, event?) => { ... })</code>
Named events	<code>sse.connect(url, { events: ['update', 'delete'] })</code>
Pipe to signal	<code>stream.pipe(signal, (msg, current) => [...current, msg])</code>
Close	<code>stream.close()</code>

GraphQL

One Method, Two Worlds

REST gives you URLs. GraphQL gives you a query language. Different philosophies, different trade-offs -- but your frontend shouldn't care. The `api` module you met in Chapter 6 already speaks GraphQL. Same auth. Same token rotation. Same error handling. One extra method and you're talking to a GraphQL endpoint.

No Apollo. No urql. No code generation. You write a query string, pass it to `api.graphql()`, and get back `{ data, errors }`. The framework handles the plumbing.

1. Your First Query

`api.graphql()` sends a POST with `{ query, variables }` and returns a typed response.

```
import { api } from "tina4js";

api.configure({ baseUrl: "/api", auth: true });

const { data, errors } = await api.graphql("/graphql",
  "{ products { id name price } }"
);

if (errors) {
  console.error("GraphQL errors:", errors);
} else {
  console.log(data.products);
}
```

The response shape never changes:

```
{
  data: T | null;           // Your result, or null on failure
  errors?: Array<{ message: string }>; // Present only when something went wrong
}
```

Query succeeds -- `data` holds your result, `errors` is undefined. Query fails -- `data` is null, `errors` tells you why. No status codes to decode. No response body formats to guess at.

2. Variables

Hardcoded values in query strings invite injection and kill reusability. Variables fix both problems. Pass them as the third argument.

```
const { data } = await api.graphql("/graphql",
  `query ($limit: Int!, $offset: Int!) {
    products(limit: $limit, offset: $offset) {
      id
      name
      price
    }
  }`
);
```

The Intelligent Native Application Framework

```

        stock
      }
    }`,
    { limit: 10, offset: 0 }
  );

```

A search query with user input:

```

const { data } = await api.graphql("/graphql",
  `query ($term: String!) {
    search_products(term: $term) {
      id
      name
      price
    }
  }`,
  { term: "widget" }
);

```

The variable goes into the JSON body, not the query string. The server handles escaping. Your frontend stays clean.

3. Mutations

Queries read. Mutations write. Same method, different keyword.

Create

```

const { data } = await api.graphql("/graphql",
  `mutation ($input: ProductInput!) {
    create_product(input: $input) {
      id
      name
      price
    }
  }`,
  {
    input: {
      name: "New Widget",
      price: 29.99,
      category_id: 1,
      stock: 100
    }
  }
);

console.log("Created:", data.create_product.id);

```

Update

```
const { data } = await api.graphql("/graphql",
  `mutation ($id: Int!, $input: ProductInput!) {
    update_product(id: $id, input: $input) {
      id
      name
      price
    }
  }`,
  { id: 42, input: { price: 24.99 } }
);
```

Delete

```
const { data } = await api.graphql("/graphql",
  `mutation ($id: Int!) {
    delete_product(id: $id) {
      success
    }
  }`,
  { id: 42 }
);
```

Three operations. One method. The server reads the `query` or `mutation` keyword and knows what to do.

4. Authentication

`api.graphql()` runs through the same auth pipeline as every other `api` method. Configure auth once and forget about it.

```
api.configure({
  baseUrl: "/api",
  auth: true,
});

// The Bearer token rides along
const { data } = await api.graphql("/graphql",
  "{ me { id name email role } }"
);
```

Token rotation works too. The server sends a `FreshToken` header. The client stores it. The next request uses the fresh token. You never touch any of this.

5. Error Handling

GraphQL responses carry two kinds of failure. The HTTP request itself can fail -- network down, server crashed, 500 response. Or the request succeeds but the query contains errors -- bad field name, permission denied, validation failure.

GraphQL Errors (query-level)

The server returns 200 but `errors` is populated. `data` might still hold partial results.

```
const { data, errors } = await api.graphql("/graphql",
  `{
    products { id name }
    categories { id name }
  }`
);

if (errors) {
  for (const err of errors) {
    console.warn("GraphQL error:", err.message);
  }
}

// Products might have loaded even if categories failed
if (data?.products) {
  renderProducts(data.products);
}
```

Network Errors (transport-level)

The HTTP request never completed. This throws.

```
try {
  const { data } = await api.graphql("/graphql", "{ products { id } }");
} catch (err) {
  console.error("Network error:", err);
}
```

Check `errors` for query problems. Catch exceptions for network problems. Handle both and your UI stays resilient.

6. Custom Headers

Per-request headers and query params go in the fourth argument. Same options object as `api.get()` and `api.post()`.

```
const { data } = await api.graphql("/graphql",
  "{ products { id name } }",
  {},
  {
    headers: { "X-Request-Id": "abc123" },
    params: { debug: "true" }
  }
);
```

The empty object in position three means "no variables." The options follow.

7. Tina4 Backend Integration

Every Tina4 backend -- Python, PHP, Ruby, Node.js -- generates a GraphQL endpoint from your ORM models. Register a model. The backend builds the schema. The frontend queries it. No SDL files. No resolvers to wire by hand.

The default endpoint is [/api/graphql](#).

```
api.configure({ baseUrl: "" });

// The backend generated this query from the User model
const { data } = await api.graphql("/api/graphql",
  "{ users(limit: 10) { id name email } }"
);

// Mutations are generated too
const { data: created } = await api.graphql("/api/graphql",
  `mutation {
    create_user(name: "Alice", email: "alice@example.com") {
      id name email
    }
  }`
);
```

Your ORM models define the schema. The GraphQL layer reads it. The frontend consumes it. Three layers, zero duplication.

8. TypeScript

[api.graphql\(\)](#) accepts a type parameter. Pass an interface that describes the response shape and TypeScript narrows the return type.

```
interface ProductsResponse {
  products: Array<{
    id: number;
    name: string;
    price: number;
  }>;
}

const { data } = await api.graphql<ProductsResponse>("/graphql",
  "{ products { id name price } }"
);

// data is ProductsResponse | null
data?.products.forEach(p => console.log(p.name));
```

The type flows through destructuring. Your editor autocompletes [data.products\[0\].name](#). No type assertions needed.

9. Reactive Queries with Signals

GraphQL and signals fit together. A signal changes. An effect fires the query. The result lands in another signal. The UI updates.

```
import { signal, effect, html } from "tina4js";
import { api } from "tina4js/api";

const searchTerm = signal("");
const products = signal([]);
const loading = signal(false);

effect(async () => {
  const term = searchTerm.value;
  if (term.length < 2) return;

  loading.value = true;
  const { data } = await api.graphql("/api/graphql",
    `query ($term: String!) {
      search_products(term: $term) { id name price }
    }`,
    { term }
  );
  products.value = data?.search_products || [];
  loading.value = false;
});

const view = html`
  <input @input=${(e) => { searchTerm.value = e.target.value; }}
    placeholder="Search products...">
  ${() => loading.value
    ? html`<p>Loading...</p>`
    : html`<ul>${() => products.value.map(p =>
        html`<li>${p.name} - ${p.price}</li>`
      )}</ul>`}
  `;
```

The user types. The signal updates. The effect queries. The list renders. No state management library. No cache normalisation layer. Signals and GraphQL, working together.

Summary

Call	What it does
<code>api.graphql(path, query)</code>	Run a query
<code>api.graphql(path, query, variables)</code>	Run a query with variables
<code>api.graphql(path, query, variables, options)</code>	Run a query with custom headers or params

One method. It sends `{ query, variables }` as JSON. It returns `{ data, errors }`. Auth, interceptors, and token rotation carry over from your `api.configure()` call. Everything you learned in Chapter 6 applies here -- GraphQL rides on the same transport.

PWA

Make It Installable

A user opens your app on their phone. They tap "Add to Home Screen." The app icon appears next to Instagram and WhatsApp. They open it on the subway with no signal, and it loads. Every page they visited before -- cached and waiting.

You built that with one function call.

1. What `pwa.register()` Does

```
import { pwa } from 'tina4js';

pwa.register({
  name: 'My App',
  shortName: 'MyApp',
  themeColor: '#1a1a2e',
  cacheStrategy: 'network-first',
  precache: ['/', '/src/public/css/default.css'],
});
```

That one call does three things:

- **Generates a web manifest** and injects it as a `<link rel="manifest">` in the document head
- **Sets the theme-color** meta tag
- **Generates and registers a service worker** with your chosen caching strategy

No build step. No config files. No `manifest.json` to maintain. Everything is generated at runtime from the config you pass.

2. Configuration

```
interface PWAConfig {
  name: string; // App name (shown in install prompt)
  shortName?: string; // Short name (shown on home screen)
  themeColor?: string; // Browser chrome color (default: '#000000')
  backgroundColor?: string; // Splash screen background (default: '#ffffff')
  display?: 'standalone' | 'fullscreen' | 'minimal-ui' | 'browser';
  icon?: string; // Path to app icon
  cacheStrategy?: 'cache-first' | 'network-first' | 'stale-while-revalidate';
  precache?: string[]; // URLs to cache on install
  offlineRoute?: string; // Fallback URL when offline and cache misses
}
```

Required

Option	Description
<code>name</code>	The full app name. Shown in the browser's install prompt and app switcher.

Optional

Option	Default	Description
<code>shortName</code>	Same as <code>name</code>	Shown on the home screen icon. Keep it short.
<code>themeColor</code>	<code>'#000000'</code>	Colors the browser chrome (address bar, status bar).
<code>backgroundColor</code>	<code>'#ffffff'</code>	Splash screen background while the app loads.
<code>display</code>	<code>'standalone'</code>	How the app looks when installed. <code>standalone</code> hides the browser chrome.
<code>icon</code>	<code>none</code>	Path to your app icon. Used for both 192x192 and 512x512 sizes.
<code>cacheStrategy</code>	<code>'network-first'</code>	How the service worker handles fetch requests.
<code>precache</code>	<code>[]</code>	URLs to download and cache on service worker install.
<code>offlineRoute</code>	<code>none</code>	URL to serve when the user is offline and the request is not in the cache.

3. Cache Strategies

The cache strategy determines what happens when the browser makes a network request. Three options. Each fits a different kind of application.

network-first (default)

The service worker tries the network. If the network fails, it falls back to the cache.

```
pwa.register({
  name: 'My App',
  cacheStrategy: 'network-first',
});
```

The Intelligent Native Application Framework

Best for apps where fresh data matters: news sites, dashboards, anything with content that changes by the hour.

How it works:

- Fetch from network
- If successful, cache the response and return it
- If network fails, return cached version
- If nothing cached, show offline fallback

cache-first

The service worker checks the cache first. It goes to the network only on a cache miss.

```
pwa.register({
  name: 'My App',
  cacheStrategy: 'cache-first',
});
```

Best for apps with stable content: documentation sites, reference apps, tools that should work in airplane mode.

How it works:

- Check cache
- If cached, return immediately (fast!)
- If not cached, fetch from network, cache it, return it
- If network fails and not cached, show offline fallback

stale-while-revalidate

The service worker returns the cached version now and updates the cache in the background. The user sees content in milliseconds. The next visit gets the fresh version.

```
pwa.register({
  name: 'My App',
  cacheStrategy: 'stale-while-revalidate',
});
```

Best for apps where speed matters but data should stay current: social feeds, product catalogs, wikis.

How it works:

- If cached, return cached version now
- In the background, fetch from network and update the cache
- Next visit gets the updated version
- If nothing cached, fetch from network

4. Precaching

Precache critical assets so they are available on the first offline visit -- before the user has navigated to them:

```
pwa.register({
  name: 'My App',
  cacheStrategy: 'network-first',
  precache: [
    '/',
    '/src/public/css/default.css',
    '/src/public/images/logo.png',
  ],
});
```

The service worker fetches and caches these URLs at install time. The user does not need to visit them first.

Precache your:

- HTML entry point (/)
- CSS files
- Critical images (logo, icons)
- Fonts

Do not precache:

- API responses (they change)
- Large files (waste bandwidth)
- User-specific content

The line between "precache" and "do not precache" is simple: if the asset is the same for every user and changes with deployments, precache it. If it varies per user or changes between requests, let the cache strategy handle it at runtime.

5. Offline Route

When the user is offline and requests a URL that is not in the cache, the service worker needs somewhere to redirect. That is the offline route:

```
pwa.register({
  name: 'My App',
  offlineRoute: '/offline.html',
  precache: ['/offline.html'],
});
```

Precache the offline route. If you skip this step, the fallback page itself will not be available when the user needs it most.

A simple offline page:

```
<!-- public/offline.html -->  
The Intelligent Native Application Framework
```

```
<!DOCTYPE html>
<html>
<head><title>Offline</title></head>
<body>
  <h1>You are offline</h1>
  <p>Check your connection and try again.</p>
</body>
</html>
```

6. The Generated Manifest

`pwa.register()` generates a manifest like this:

```
{
  "name": "My App",
  "short_name": "MyApp",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1a1a2e",
  "icons": [
    { "src": "/icon.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

It is injected as a Blob URL in a `<link rel="manifest">` tag. You do not need to create or maintain a `manifest.json` file.

Build-Time Generation

If you prefer a static manifest file (for CDN caching or server inspection), use the generation methods:

```
import { pwa } from 'tina4js';

const config = {
  name: 'My App',
  shortName: 'MyApp',
  themeColor: '#1a1a2e',
  icon: '/icon.png',
  cacheStrategy: 'network-first',
  precache: ['/', '/styles.css'],
};

// Get manifest as an object
const manifest = pwa.generateManifest(config);
// Write to file in your build script

// Get service worker as a string
const swCode = pwa.generateServiceWorker(config);
// Write to file: fs.writeFileSync('dist/sw.js', swCode);
```

Runtime generation works for most projects. Build-time generation is there when your deployment pipeline demands static files.

7. Complete Example

```
// src/main.ts
import { router, api } from 'tina4js';
import { pwa } from 'tina4js';
import './routes/index';

// Configure API
api.configure({ baseUrl: '/api', auth: true });

// Register PWA
pwa.register({
  name: 'Task Manager',
  shortName: 'Tasks',
  themeColor: '#2563eb',
  backgroundColor: '#f8fafc',
  display: 'standalone',
  icon: '/icon-512.png',
  cacheStrategy: 'network-first',
  precache: [
    '/',
    '/src/public/css/default.css',
    '/icon-512.png',
  ],
  offlineRoute: '/offline',
});

// Start router
router.start({ target: '#root', mode: 'history' });
```

When a user visits this app on a mobile browser, they see an "Add to Home Screen" prompt (if the browser supports it). After installing, the app launches without browser chrome, uses the blue theme color, and works offline for any cached page. The gap between "web app" and "native app" disappears -- and you closed it with twelve lines of configuration.

8. Tips

Icons: Provide at least one 512x512 PNG icon. Some browsers refuse to show the install prompt without it.

HTTPS required: Service workers only work over HTTPS (or localhost for development). Vite's dev server handles this for you.

Testing: In Chrome DevTools, use Application > Service Workers to see the registered service worker, and Application > Cache Storage to inspect cached assets. Use the Network panel's "Offline" checkbox to test offline behavior.

Updating: When you change your app and redeploy, the service worker detects the change and updates. The `skipWaiting()` call in the generated service worker ensures the new version activates without waiting for the user to close all tabs.

Scope: The service worker registers at the root scope. It intercepts all GET requests from your domain. Every fetch passes through your chosen cache strategy.

Summary

	How		
	<code>pwa.register(config)</code>		
	<code>name: 'My App'</code>		
	<code>themeColor: '#hex'</code>		
	<code>`cacheStrategy: 'network-first' \</code>	<code>'cache-first' \</code>	<code>'stale-while-rev</code>
	<code>precache: ['/path1', '/path2']</code>		
	<code>offlineRoute: '/offline'</code>		
	<code>pwa.generateManifest(config)</code>		
	<code>pwa.generateServiceWorker(config)</code>		

Debug Overlay

See Everything

A signal changes. A component mounts. A route resolves. An API call returns a 401. You did not see any of it. You open the console, add a `console.log`, refresh, reproduce the bug, read the output, add another `console.log`, refresh again. Twenty minutes pass. The bug was a misspelled signal label.

The debug overlay shows you everything in real time. One import. One keyboard shortcut. Full visibility into your running application.

1. Enabling the Debug Overlay

```
import 'tina4js/debug';
```

That is the entire setup. One import. The overlay is active.

For production builds, conditionally import it:

```
if (import.meta.env.DEV) {  
  import('tina4js/debug');  
}
```

Vite tree-shakes the debug module away in production. Zero bytes in your final bundle.

The dynamic import resolves after your app's modules have already run. That is fine -- signals created before the overlay loads, including module-level store signals, are buffered and show up the moment the overlay attaches. Import order does not matter.

When the debug module loads, you see a console message:

```
[tina4] Debug overlay enabled (Ctrl+Shift+D to toggle)
```

2. Opening the Overlay

Press **Ctrl+Shift+D** (or **Cmd+Shift+D** on Mac) to toggle the overlay. A panel appears at the bottom of the screen with four tabs: Signals, Components, Routes, and API.

Each tab is a window into a different layer of your application. Together, they replace dozens of `console.log` statements with a live, interactive dashboard.

3. The Signals Panel

This panel shows every signal in your application:

- **Label** -- the debug label you passed as the second argument to `signal\(\)`

The Intelligent Native Application 4ramework

- **Current value** -- the live value, updating in real time
- **Subscriber count** -- how many effects, computed signals, and DOM bindings depend on this signal
- **Update count** -- how many times the value has changed since creation

This is why debug labels matter:

```
// Without label -- shows as "Signal<number>" in the panel
const count = signal(0);

// With label -- shows as "count" in the panel
const count = signal(0, 'count');
```

Add labels to every signal you might need to debug. The cost is zero when the debug module is not imported.

What to Look For

- **Subscriber count of 0:** This signal is not connected to anything. Either it is unused, or you read `.value` instead of passing the signal directly to a template binding.
- **High update count:** This signal is updating with unusual frequency. Check if you have an effect that writes to it in a loop.
- **Unexpected value:** The signal contains something you did not expect. Trace back to every place that writes to it. The update count tells you how many writes have occurred -- if the count is higher than you expect, something is writing that should not be.

4. The Components Panel

This panel shows every mounted `Tina4Element`:

- **Tag name** -- the custom element tag (e.g., `app-header`, `user-card`)
- **Mount status** -- whether the component is in the DOM

When a component mounts, it appears in the list. When it unmounts (removed from DOM), it disappears.

What to Look For

- **Too many instances:** You might be creating components inside a reactive block that re-renders, spawning new elements on every signal change. If you see `user-card` appear 50 times and you only have 10 users, the component is being recreated instead of updated.
- **Missing components:** A component you expected to see is absent. Check two things: does it extend `Tina4Element`, and did you call `customElements.define()` for it? Both must be true for the component to appear in the panel.

5. The Routes Panel

This panel shows navigation history:

- **Path** -- the URL that was navigated to
- **Pattern** -- the route pattern that matched
- **Params** -- extracted route parameters
- **Duration** -- how long the route handler took to render (ms)

Every navigation (initial load, link click, `navigate()` call, browser back/forward) creates an entry. The history builds up as you use the app, giving you a timeline of every page transition.

What to Look For

- **Slow routes:** A high `durationMs` value means the route handler is doing expensive work -- a heavy DOM build, a slow API call, or both. Compare durations across routes to find the outliers.
- **Wrong pattern matched:** If a path matched an unexpected pattern, your route registration order might be wrong. The router uses first-match-wins. A broad pattern registered before a specific one swallows the specific route.
- **Guard redirects:** If you see a navigation to a guarded route followed by a redirect to `/login`, the guard is working. If the guarded route renders without a redirect, the guard function has a bug.

6. The API Panel

This panel shows every HTTP request made through the `api` client:

- **Method and URL** -- GET `/users`, POST `/auth/login`, etc.
- **Status code** -- 200, 404, 500, etc.
- **Request headers** -- what was sent
- **Response data** -- what came back

What to Look For

- **401 responses:** The token might be expired or missing. Check if `auth: true` is configured and the token exists in `localStorage`.
- **Request body:** Verify that `formToken` is present in POST/PUT/PATCH/DELETE requests when auth is enabled. A missing `formToken` means the API client is not configured for authentication, or the token was cleared.
- **Missing requests:** If an API call is not appearing, the code might be using `fetch()` directly instead of the `api` client. The debug panel tracks only requests that go through `api.get/post/put/patch/delete`.

7. How It Works Internally

The debug module hooks into the framework at four points:

- **Signal hooks:** `__debugSignalCreate` and `__debugSignalUpdate` fire when signals are created and updated. These are null in production (tree-shaken away).
- **Component hooks:** `__debugComponentMount` and `__debugComponentUnmount` fire on `connectedCallback/disconnectedCallback`.
- **Route tracking:** The debug module subscribes to `router.on('change', ...)`.
- **API tracking:** Request and response interceptors register via `api.intercept()`.

All hooks are set to `null` by default. They activate only when `import 'tina4js/debug'` runs. If you never import the debug module, the hooks compile away through tree-shaking. Zero runtime cost in production.

The overlay itself is a Web Component (`<tina4-debug>`) appended to `document.body`. It uses its own Shadow DOM, so its styles never interfere with your application. Your CSS cannot break the overlay. The overlay cannot break your CSS. Complete isolation.

8. Best Practices

Always Use Debug Labels

```
// Do this for every signal you care about:
const users = signal<User[]>([], 'users');
const currentPage = signal(1, 'current-page');
const searchQuery = signal('', 'search-query');
const isLoading = signal(false, 'is-loading');
```

When you open the debug panel and see 15 signals, labels are the difference between understanding your app state in seconds and spending minutes cross-referencing source files to figure out which `Signal<string>` is which.

Import Conditionally

```
// src/main.ts
if (import.meta.env.DEV) {
  import('tina4js/debug');
}
```

Never ship the debug overlay to production. It adds weight and exposes internal state to anyone who opens DevTools.

Use During Development, Not Just Debugging

Do not wait for a bug to open the debug overlay. Keep it open while you develop. Watch signals update as you click buttons. Watch API requests flow as pages load. Watch routes resolve as you navigate. The overlay builds your intuition for how data moves through your application -- and that intuition is what makes you fast when a bug does appear.

Summary

What	How
Enable	<code>import 'tina4js/debug'</code>
Conditional	<code>if (import.meta.env.DEV) import('tina4js/debug')</code>
Toggle	Ctrl+Shift+D
Signals panel	Shows label, value, subscribers, update count
Components panel	Shows mounted Tina4Element instances
Routes panel	Shows navigation history with timing
API panel	Shows requests and responses
Debug labels	<code>signal(value, 'label')</code>
Production cost	Zero (tree-shaken away)

tina4-css

Optional Styling

You build a prototype. The HTML is clean. The logic works. But the buttons look like 1998, the form inputs differ between Chrome and Safari, and the layout breaks on mobile. You reach for a CSS framework. It pulls in 300KB of utility classes you will never use.

tina4-css is the alternative. One stylesheet. Good defaults. Drop it in and your app looks professional. Override what you want, ignore the rest. And if you already have your own CSS -- skip this chapter entirely.

1. What Is tina4-css

tina4-css is a standalone CSS library. It is not part of tina4-js. It is a separate npm package that gives you:

- A CSS reset
- A responsive grid system
- Styled buttons, forms, tables, cards, badges, alerts
- Navigation components
- Modal dialogs
- Pagination
- A dark theme

The library is designed to look good with zero configuration. No theme files. No customization wizard. No build-time compilation. One file, and your app has a consistent, polished appearance across every browser.

2. Installation

With the CLI

```
tina4 init js my-app
```

The Rust CLI does not yet accept a `--css` flag on `init`, so add `tina4-css` to `package.json` after scaffolding, or use the fallback `npm install tina4-css` which adds the dependency and includes the stylesheet link in `index.html` in one step.

Manual Installation

```
npm install tina4-css
```

Then include it in your `index.html`:

```
<link rel="stylesheet" href="/node_modules/tina4-css/dist/tina4.min.css">
```

The Intelligent Native Application 4ramework

Or import it in your main TypeScript file (if your bundler supports CSS imports):

```
import 'tina4-css/dist/tina4.min.css';
```

3. The Reset

tina4-css includes a modern CSS reset. It:

- Removes default margins and padding
- Sets `box-sizing: border-box` on everything
- Uses system fonts
- Sets sensible defaults for headings, links, lists, and form elements

Every browser ships with different default styles. A `<button>` in Chrome looks different from a `<button>` in Firefox looks different from a `<button>` in Safari. The reset eliminates these differences. Your app starts from a clean, predictable baseline.

4. Grid System

A responsive grid based on CSS Grid:

```
<div class="grid grid-cols-3 gap-4">
  <div>Column 1</div>
  <div>Column 2</div>
  <div>Column 3</div>
</div>
```

Responsive variants:

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>
  <div>Item 4</div>
</div>
```

This renders 1 column on mobile, 2 on medium screens, and 4 on large screens. The layout adapts to the viewport. No JavaScript. No resize listeners. Pure CSS breakpoints.

5. Buttons

```
<button class="btn">Default</button>
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-outline">Outline</button>
<button class="btn btn-sm">Small</button>
<button class="btn btn-lg">Large</button>
```


Using with tina4-js:

```
html`
  <button class="btn btn-primary" @click=${handleClick}>
    Save
  </button>
  <button
    class="btn btn-danger"
    @click=${handleDelete}
    ?disabled=${isDeleting}
  >
    ${() => isDeleting.value ? 'Deleting...' : 'Delete'}
  </button>
`
```

The `?disabled` binding and the reactive text work with tina4-css classes without conflict. The framework handles the DOM. The stylesheet handles the appearance. Each stays in its lane.

6. Forms

tina4-css styles form elements to look consistent across browsers:

```
<form>
  <div class="form-group">
    <label>Name</label>
    <input type="text" class="form-control" placeholder="Enter name">
  </div>
  <div class="form-group">
    <label>Email</label>
    <input type="email" class="form-control" placeholder="Enter email">
  </div>
  <div class="form-group">
    <label>Role</label>
    <select class="form-control">
      <option>Admin</option>
      <option>Editor</option>
      <option>Viewer</option>
    </select>
  </div>
  <div class="form-group">
    <label>Notes</label>
    <textarea class="form-control" rows="3"></textarea>
  </div>
  <button type="submit" class="btn btn-primary">Submit</button>
</form>
```

With tina4-js signals:

```
const name = signal('');
const email = signal('');
```

```
html`
  <form @submit=${(e: Event) => { e.preventDefault(); handleSubmit(); }}>
    <div class="form-group">
      <label>Name</label>
      <input
        type="text"

```

The Intelligent Native Application Framework

```

        class="form-control"
        .value=${name}
        @input=${(e: Event) => { name.value = (e.target as HTMLInputElement).value; }}
    />
</div>
<div class="form-group">
    <label>Email</label>
    <input
        type="email"
        class="form-control"
        .value=${email}
        @input=${(e: Event) => { email.value = (e.target as HTMLInputElement).value; }}
    />
</div>
<button type="submit" class="btn btn-primary">Save</button>
</form>

```

The `.value` binding keeps the input synchronized with the signal. The `form-control` class keeps the input looking good. Signal-driven forms with polished styling -- no extra libraries required.

7. Tables

```

<table class="table">
  <thead>
    <tr>
      <th>Name</th>
      <th>Email</th>
      <th>Role</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Alice</td>
      <td>alice@example.com</td>
      <td>Admin</td>
    </tr>
  </tbody>
</table>

```

Variants:

```

<table class="table table-striped">...</table>
<table class="table table-hover">...</table>

```

With reactive data:

```

const users = signal<User[]>([]);

html`
  <table class="table table-striped">
    <thead>
      <tr><th>Name</th><th>Email</th><th>Actions</th></tr>
    </thead>
    <tbody>
      ${() => users.value.map(user => html`

```

The Intelligent Native Application Framework

```

        <tr>
          <td>${user.name}</td>
          <td>${user.email}</td>
          <td>
            <button class="btn btn-sm" @click=${() => editUser(user)}>Edit</button>
          </td>
        </tr>
      `)}
    </tbody>
  </table>
`

```

The table re-renders when the `users` signal changes. Striped rows. Hover highlights. Edit buttons on every row. A data table with ten lines of template code.

8. Cards

```

<div class="card">
  <div class="card-header">Card Title</div>
  <div class="card-body">
    <p>Card content goes here.</p>
  </div>
  <div class="card-footer">
    <button class="btn btn-primary">Action</button>
  </div>
</div>

```

Card grid:

```

html`
  <div class="grid grid-cols-1 md:grid-cols-3 gap-4">
    ${() => products.value.map(product => html`
      <div class="card">
        <div class="card-body">
          <h3>${product.name}</h3>
          <p>${product.price}</p>
        </div>
        <div class="card-footer">
          <button class="btn btn-primary btn-sm" @click=${() => addToCart(product)}>
            Add to Cart
          </button>
        </div>
      </div>
    `)}
  </div>
`

```

Three columns on desktop. One column on mobile. Each card has a body and a footer. The grid, the cards, and the reactive list all compose without friction.

9. Badges and Alerts

Badges

```
<span class="badge">Default</span>
<span class="badge badge-primary">Primary</span>
<span class="badge badge-success">Success</span>
<span class="badge badge-danger">Danger</span>
<span class="badge badge-warning">Warning</span>
```

Alerts

```
<div class="alert alert-info">This is an informational message.</div>
<div class="alert alert-success">Operation completed.</div>
<div class="alert alert-warning">Please check your input.</div>
<div class="alert alert-danger">Something went wrong.</div>
```

Reactive alerts:

```
const error = signal<string | null>(null);

html`
  ${() => error.value
    ? html`<div class="alert alert-danger">${error}</div>`
    : null
  }
`
```

The alert appears when the error signal has a value. It vanishes when the error clears. No show/hide logic. No CSS transitions to manage. Set the signal. The DOM follows.

10. Navigation

```
<nav class="navbar">
  <a class="navbar-brand" href="/">My App</a>
  <div class="navbar-nav">
    <a class="nav-link active" href="/">Home</a>
    <a class="nav-link" href="/about">About</a>
    <a class="nav-link" href="/contact">Contact</a>
  </div>
</nav>
```

With active route tracking:

```
const currentPath = signal('/');
router.on('change', ({ path }) => { currentPath.value = path; });
```

```
html`
  <nav class="navbar">
    <a class="navbar-brand" href="/">My App</a>
    <div class="navbar-nav">
      <a class=${() => `nav-link ${currentPath.value === '/' ? 'active' : ''}`} href="/">Home</a>
      <a class=${() => `nav-link ${currentPath.value === '/about' ? 'active' : ''}`} href="/about">About</a>
    </div>
  </nav>
`
```

The `active` class moves to the current link as the user navigates. The navbar highlights where you are without a single line of imperative DOM manipulation.

11. Modals

```
<div class="modal" id="myModal">
  <div class="modal-dialog">
    <div class="modal-header">
      <h3>Confirm Action</h3>
    </div>
    <div class="modal-body">
      <p>Are you sure?</p>
    </div>
    <div class="modal-footer">
      <button class="btn">Cancel</button>
      <button class="btn btn-danger">Delete</button>
    </div>
  </div>
</div>
```

Control with a signal:

```
const showModal = signal(false);
```

```
html`
  ${() => showModal.value
    ? html`
      <div class="modal active">
        <div class="modal-dialog">
          <div class="modal-header">
            <h3>Confirm Delete</h3>
          </div>
          <div class="modal-body">
            <p>This action cannot be undone.</p>
          </div>
          <div class="modal-footer">
            <button class="btn" @click=${() => { showModal.value = false; }}>Cancel</button>
            <button class="btn btn-danger" @click=${() => { doDelete(); showModal.value = false; }}>Delete
          </div>
        </div>
      </div>
    `
    : null
  }
`
```

One signal. One boolean. The modal exists in the DOM when the signal is true and vanishes when it is false. No jQuery. No imperative show/hide methods. The template is the single source of truth for what appears on screen.

12. Dark Theme

tina4-css includes a dark theme. Activate it by toggling a class on the `<html>` element:

```
const darkMode = signal(false);

effect(() => {
  document.documentElement.classList.toggle('dark', darkMode.value);
});

html`
  <button @click=${() => { darkMode.value = !darkMode.value; }}>
    ${() => darkMode.value ? 'Light Mode' : 'Dark Mode'}
  </button>
`
```

All tina4-css components adapt to dark mode -- backgrounds, text colors, borders, form elements, cards, tables, alerts. One class toggles the entire palette. You write the toggle. tina4-css handles every surface.

13. Using with Shadow DOM Components

Shadow DOM components do not inherit external CSS. If you use `Tina4Element` with Shadow DOM (the default), tina4-css classes will not work inside the component. The Shadow DOM boundary blocks them.

Two options:

Option 1: Light DOM

```
class MyPage extends Tina4Element {
  static shadow = false; // External CSS applies

  render() {
    return html`
      <div class="card">
        <div class="card-body">
          <h3>This uses tina4-css</h3>
        </div>
      </div>
    `;
  }
}
```

Option 2: Import CSS into Shadow DOM

```
class MyComponent extends Tina4Element {
  static styles = `
    @import url('/node_modules/tina4-css/dist/tina4.min.css');
    /* Additional component styles */
  `;

  render() {
    return html`<button class="btn btn-primary">Works!</button>`;
  }
}
```

For most applications, the best balance is light DOM for pages (where tina4-css classes apply) and Shadow DOM for reusable widgets (where encapsulation matters). Pages get the full stylesheet. Widgets get isolation.

Summary

Component	Class
Buttons	<code>btn</code> , <code>btn-primary</code> , <code>btn-secondary</code> , <code>btn-danger</code> , <code>btn-outline</code> , <code>btn-sm</code> , <code>btn-lg</code>
Forms	<code>form-group</code> , <code>form-control</code>
Tables	<code>table</code> , <code>table-striped</code> , <code>table-hover</code>
Cards	<code>card</code> , <code>card-header</code> , <code>card-body</code> , <code>card-footer</code>
Badges	<code>badge</code> , <code>badge-primary</code> , <code>badge-success</code> , <code>badge-danger</code> , <code>badge-warning</code>
Alerts	<code>alert</code> , <code>alert-info</code> , <code>alert-success</code> , <code>alert-warning</code> , <code>alert-danger</code>
Navigation	<code>navbar</code> , <code>navbar-brand</code> , <code>navbar-nav</code> , <code>nav-link</code> , <code>active</code>
Modals	<code>modal</code> , <code>modal-dialog</code> , <code>modal-header</code> , <code>modal-body</code> , <code>modal-footer</code> , <code>active</code>
Grid	<code>grid</code> , <code>grid-cols-{n}</code> , <code>md:grid-cols-{n}</code> , <code>lg:grid-cols-{n}</code> , <code>gap-{n}</code>
Dark mode	Add <code>dark</code> class to <code><html></code>
Pagination	<code>pagination</code>

Backend Integration

The Full Stack

Your frontend fetches data. Your backend serves it. Between them sits authentication, CSRF protection, token rotation, and the question every team asks: how do we wire these two together without it becoming a mess?

This chapter answers that question. You will connect tina4-js to tina4-php and tina4-python backends, follow the auth flow from login to token expiry, build for backend embedding, and add interactive islands to server-rendered pages.

1. The Tina4 Stack

tina4-js was built to pair with tina4-php and tina4-python. The API client speaks their protocol:

- **Authorization:** `Bearer <token>` on every request when auth is enabled
- `formToken` injected into POST/PUT/PATCH/DELETE bodies for CSRF protection
- `FreshToken` response header read for token rotation
- JSON in, JSON out

But tina4-js works with any backend that sends JSON and accepts Bearer tokens. The API client is a `fetch()` wrapper with opinions. If your backend follows REST conventions, tina4-js connects to it without modification.

2. tina4-js + tina4-php

Setup

Backend (tina4-php):

```
tina4 create my-backend --php
cd my-backend
tina4 serve
# Running on http://localhost:7145
```

Frontend (tina4-js):

```
tina4 init js my-frontend
cd my-frontend
npm install
```

In npm-only environments where the tina4 Rust CLI isn't available, the fallback is `npx tina4js create my-frontend`.

Configure the Vite proxy to forward API calls:

```
// vite.config.ts
import { defineConfig } from 'vite';
// The Intelligent Native Application 4ramework
```



```
export default defineConfig({
  server: {
    port: 3000,
    proxy: {
      '/api': 'http://localhost:7145',
    },
  },
});
```

Configure the API client:

```
// src/main.ts
import { api, router } from 'tina4js';
import './routes/index';

api.configure({
  baseUrl: '/api',
  auth: true,
});

router.start({ target: '#root', mode: 'hash' });
```

Now `api.get('/users')` hits `http://localhost:7145/api/users` during development. The Vite proxy handles the forwarding. The browser sees a same-origin request. No CORS headers needed.

tina4-php Routes

```
// src/routes/users.php
\Tina4\Get::add("/api/users", function(\Tina4\Response $response) {
    $users = (new User())->select("*")->toArray();
    return $response($users);
});

\Tina4\Post::add("/api/users", function(\Tina4\Response $response, \Tina4\Request $request) {
    $user = new User();
    $user->name = $request->data->name;
    $user->email = $request->data->email;
    $user->save();
    return $response($user->toArray());
});
```

tina4-js Frontend

```
// src/routes/users.ts
import { route, html, signal, api } from 'tina4js';

route('/users', async () => {
  const users = signal<any[]>([]);
  users.value = await api.get('/users');

  return html`
    <h1>Users</h1>
    <ul>
      ${() => users.value.map(u => html`<li>${u.name} (${u.email})</li>`}
    </ul>
  `;
});
```

The backend defines the endpoints. The frontend consumes them. The proxy bridges the ports during development. In production, they share the same origin.

3. tina4-js + tina4-python

Setup

Backend (tina4-python):

```
tina4 create my-backend --python
cd my-backend
tina4 serve
# Running on http://localhost:7145
```

The Vite proxy and API configuration are identical to the PHP setup. Same config. Same proxy. Same port.

tina4-python Routes

```
# src/routes/users.py
from tina4_python import get, post

@get("/api/users")
async def get_users(request, response):
    users = User().select("*").as_list()
    return response(users)

@post("/api/users")
async def create_user(request, response):
    user = User()
    user.name = request.body.get("name")
    user.email = request.body.get("email")
    user.save()
    return response(user.as_dict())
```

The frontend code is the same whether the backend runs PHP or Python. The API client does not care what language processes the request. It sends JSON. It receives JSON. The language behind the endpoint is invisible to the browser.

4. Authentication End-to-End

Here is the complete auth flow, from login to token expiry, in five steps.

Step 1: Login

The frontend sends credentials:

```
const result = await api.post<{ token: string }>('/api/auth/login', {
  email: 'alice@example.com',
  password: 'secret',
});
localStorage.setItem('tina4_token', result.token);
```

The backend validates and returns a JWT:

```
// tina4-php
\Tina4\Post::add("/api/auth/login", function($response, $request) {
    $user = (new User())->find("email = '{$request->data->email}'");
    if ($user && password_verify($request->data->password, $user->password)) {
        $token = \Tina4\Auth::generateToken(["userId" => $user->id]);
        return $response(["token" => $token]);
    }
    return $response(["error" => "Invalid credentials"], 401);
});
```

Step 2: Authenticated Requests

With `auth: true`, every request now includes:

```
Authorization: Bearer <the-jwt-token>
```

For POST/PUT/PATCH/DELETE, the body also includes:

```
{ "name": "Alice", "formToken": "<the-jwt-token>" }
```

The backend validates both the header and the body token. Two layers of verification. The header proves identity. The body token prevents CSRF.

Step 3: Token Rotation

Tokens expire. The backend can issue a fresh token on any response by sending a `FreshToken` header:

```
$freshToken = \Tina4\Auth::generateToken(["userId" => $user->id]);
$response->addHeader("FreshToken", $freshToken);
return $response($data);
```

The API client stores the fresh token in `localStorage`. The next request uses the new token. The user never sees a login prompt as long as they stay active. Token rotation happens in the background, invisible and automatic.

Step 4: Route Guards

Protect frontend routes:

```
import { signal, computed } from 'tina4js';

const token = signal<string | null>(
    localStorage.getItem('tina4_token'),
    'auth-token'
);

const isLoggedInIn = computed(() => token.value !== null);

route('/dashboard', {
    guard: () => isLoggedInIn.value || '/login',
    handler: dashboardPage,
});
```

The guard runs before the route handler. If `isLoggedInIn` is false, the router redirects to `/login`. The dashboard page never renders. The handler never executes. Guards are the frontend's first line of defense.

Step 5: 401 Handling

If the token expires and rotation did not happen, the backend returns 401. Handle it with a global interceptor:

```
api.intercept('response', (response) => {
  if (response.status === 401) {
    localStorage.removeItem('tina4_token');
    token.value = null;
    navigate('/login', { replace: true });
  }
});
```

Token gone. Signal cleared. User redirected. One interceptor handles every 401 across every API call in the application. No per-request error handling needed.

5. Building for Backend Embedding

When your tina4-js app is served by a tina4-php or tina4-python backend (not a separate frontend server), you build the JavaScript bundle and place it where the backend serves static files.

Build

```
npm run build
```

This creates a [dist/](#) folder with your bundled JavaScript and CSS.

Deploy to Backend

Copy the build output to the backend's public directory:

```
# For tina4-php
cp -r dist/* ../my-backend/src/public/js/
```

```
# For tina4-python
cp -r dist/* ../my-backend/src/public/js/
```

Or configure Vite to output to the backend directory:

```
// vite.config.ts
import { defineConfig } from 'vite';

export default defineConfig({
  build: {
    outDir: '../my-backend/src/public/js',
    emptyOutDir: true,
  },
});
```

The CLI Build Command

The tina4 CLI has a build command with target support:

```
npx tina4js build --target php
npx tina4js build --target python
```

This builds and places the output in the conventional location for each backend framework. One command. The files land where the backend expects them.

Backend Template

The backend serves an HTML page that loads the tina4-js bundle:

```
<!-- tina4-php: src/templates/index.twig -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My App</title>
  <link rel="stylesheet" href="/js/style.css">
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/js/main.js"></script>
</body>
</html>
```

The backend renders this template. The browser loads the JavaScript. The tina4-js router takes over. From the user's perspective, it is a single application. From the developer's perspective, it is two codebases that meet at the HTML page.

6. Islands Architecture

Most of this book assumes you are building a single-page application where tina4-js controls the entire page. Islands architecture flips that model. The server renders the page. tina4-js controls small, specific sections -- islands of interactivity in a sea of static HTML.

This approach works when:

- You have existing server-rendered pages and want to add interactivity without rewriting them
- You want fast initial loads with progressive enhancement
- SEO matters and you need server-rendered content that search engines can index

How It Works

- The backend renders the full HTML page
- tina4-js components hydrate specific sections
- Each component is independent -- it manages its own state and DOM

Example: Adding a Live Search to a Server-Rendered Page

The server renders the page:

```
<!-- Server-rendered page -->
<div class="product-listing">
  <h1>Products</h1>

  <!-- This island becomes interactive -->
  <div class="live-search">
    <input type="text" value="Search products" />
    <button type="button" value="Search" />
  </div>
</div>
```

The Intelligent Native Application Framework

```

<product-search api-url="/api/products"></product-search>

<!-- This stays static -->
<footer>Copyright 2024</footer>
</div>

<script type="module">
  import { Tina4Element, html, signal, api } from 'tina4js';

  class ProductSearch extends Tina4Element {
    static props = { 'api-url': String };
    static shadow = false;

    render() {
      const query = signal('');
      const results = signal<any[]>([]);

      const search = async () => {
        if (query.value.length < 2) return;
        results.value = await api.get(
          this.prop<string>('api-url').value,
          { params: { q: query.value } }
        );
      };

      return html`
        <div>
          <input
            type="search"
            placeholder="Search products..."
            @input=${(e: Event) => {
              query.value = (e.target as HTMLInputElement).value;
              search();
            }}
          />
          <ul>
            ${() => results.value.map(p => html`
              <li>${p.name} - ${p.price}</li>
            `)}
          </ul>
        </div>
      `;
    }
  }

  customElements.define('product-search', ProductSearch);
</script>

```

The server sends HTML. The browser parses it. When it reaches the `<product-search>` tag, the custom element activates. The search input becomes live. The rest of the page stays static. The server did the heavy lifting. tina4-js added the interactivity.

Multiple Islands

Each island is a separate component. They do not need to know about each other:

```

<body>
  <!-- Island 1: Live search -->
  <product-search></product-search>

```

The Intelligent Native Application Framework

```

<!-- Static server-rendered content -->
<section class="featured">
  <h2>Featured Products</h2>
  <!-- server-rendered list -->
</section>

<!-- Island 2: Shopping cart -->
<cart-widget></cart-widget>

<!-- Island 3: Notification bell -->
<notification-bell user-id="42"></notification-bell>
</body>

```

Each component self-initializes when the browser parses its tag. No router needed. No app shell. No bootstrapping ceremony. Place the tags where you want interactivity. The components wake up on their own.

Shared State Between Islands

If islands need to share state, use the store pattern from Chapter 4:

```

// store.ts
export const cart = signal<CartItem[]>([], 'cart');
export const cartCount = computed(() => cart.value.length);

```

Both [product-search](#) (add to cart) and [cart-widget](#) (display cart) import from the same store. When a user adds an item through the search island, the cart count updates in the cart widget. Two components. One signal. Synchronized without either knowing the other exists.

7. Development Workflow

Separate Frontend and Backend

Best for larger projects with dedicated frontend and backend teams.

```

my-project/
  frontend/      # tina4-js project
    src/
    package.json
    vite.config.ts
  backend/      # tina4-php or tina4-python
    src/

```

The frontend runs on [localhost:3000](#) with Vite. The backend runs on [localhost:7145](#). Vite proxies API calls. Each team works in its own directory, its own repository, its own deployment pipeline.

Embedded Frontend

Best for smaller projects or when one team owns both layers.

```

my-project/
  src/
    routes/      # Backend routes
    templates/   # Backend templates
    orm/         # Backend ORM models

```

The Intelligent Native Application Framework

```

    public/
      js/          # Built tina4-js output goes here
frontend/        # tina4-js source
  src/
  package.json
  vite.config.ts # builds to ../src/public/js/

```

One repository. One deployment. The frontend builds into the backend's public directory. The backend serves everything. For a team of one or two developers, this is the fastest path from code to production.

Summary

What	How
Proxy API in dev	Vite <code>proxy: { '/api': 'http://localhost:7145' }</code>
Configure API	<code>api.configure({ baseUrl: '/api', auth: true })</code>
Auth token	Stored in localStorage, auto-sent as Bearer
CSRF token	Auto-injected as <code>formToken</code> in request bodies
Token rotation	Automatic via <code>FreshToken</code> response header
401 handling	Response interceptor redirects to login
Build for backend	<code>npx tina4js build --target php</code> or <code>--target python</code>
Islands	Use <code>Tina4Element</code> components in server-rendered HTML
Shared state	Export signals from a store module

Building a Complete App

The Admin Dashboard

Picture the screen. A dark navbar across the top. Four stat cards showing live numbers -- total users, active sessions, revenue, orders -- updating without a page refresh. A notification bell with a red badge. A data table with search, pagination, and inline editing. A login page that guards everything behind it.

This chapter builds that dashboard from scratch. It uses every tina4-js module: signals, html, components, routing, API, WebSocket, and PWA. By the end, you will have a complete, deployable application -- and you will have seen how all the pieces from previous chapters fit together in production.

1. Project Setup

```
tina4 init js admin-dashboard
cd admin-dashboard
npm install
```

The Rust CLI `tina4 init js` does not yet accept `--css`. To pull in Tina4 CSS, either add `tina4-css` to `package.json` after scaffolding, or use the npm-only fallback `npmx tina4js create admin-dashboard --css` which wires it in automatically.

We will build this structure:

```
src/
  main.ts           # Entry point
  store.ts          # Global state
  routes/
    index.ts        # Route definitions
  pages/
    login.ts        # Login page
    dashboard.ts    # Dashboard with live stats
    users.ts        # CRUD user management
  components/
    app-layout.ts   # Layout wrapper with nav
    data-table.ts   # Reusable data table
    stat-card.ts    # Statistics card
    notification-bell.ts # WebSocket notification bell
```

2. Global Store

Start with the shared state. Authentication drives the entire application -- which pages render, which API calls include tokens, what the navbar displays. The store is the foundation.

```
// src/store.ts
import { signal, computed } from 'tina4js';
```

```
// Auth
```

The Intelligent Native Application 4ramework

```

export const token = signal<string | null>(
  localStorage.getItem('tina4_token'),
  'auth-token'
);
export const user = signal<{ id: number; name: string; role: string } | null>(
  null,
  'current-user'
);
export const isLoggedIn = computed(() => token.value !== null);
export const isAdmin = computed(() => user.value?.role === 'admin');

export function setAuth(newToken: string, userData: { id: number; name: string; role: string }) {
  localStorage.setItem('tina4_token', newToken);
  token.value = newToken;
  user.value = userData;
}

export function clearAuth() {
  localStorage.removeItem('tina4_token');
  token.value = null;
  user.value = null;
}

// Notifications
export const notifications = signal<{ id: number; text: string; time: string }[]>(
  [],
  'notifications'
);
export const unreadCount = computed(() => notifications.value.length);

```

Every signal has a debug label. Every piece of derived state is a computed signal. The store exports functions for mutations, not raw signal writes. This discipline pays for itself the moment you open the debug overlay and see named, traceable state.

3. App Entry

```
// src/main.ts
import { api, router } from 'tina4js';
import { pwa } from 'tina4js';
import { token, clearAuth } from './store';
import { navigate } from 'tina4js';
import './routes/index';
import './components/app-layout';
import './components/data-table';
import './components/stat-card';
import './components/notification-bell';

// Debug overlay in dev only
if (import.meta.env.DEV) {
  import('tina4js/debug');
}

// API configuration
api.configure({
  baseUrl: '/api',
  auth: true,
});

// Global 401 handler
api.intercept('response', (response) => {
  if (response.status === 401) {
    clearAuth();
    navigate('/login', { replace: true });
  }
});

// PWA
pwa.register({
  name: 'Admin Dashboard',
  shortName: 'Admin',
  themeColor: '#1e293b',
  cacheStrategy: 'network-first',
  precache: ['/'],
});

// Start router
router.start({ target: '#root', mode: 'hash' });
```

The entry point is the wiring diagram. API configured. Interceptor registered. PWA enabled. Debug overlay loaded in development. Router started. Every cross-cutting concern lives here, declared once, applied everywhere.

4. Routes

```
// src/routes/index.ts
import { route } from 'tina4js';
import { isLoggedIn } from '../store';
import { loginPage } from '../pages/login';
import { dashboardPage } from '../pages/dashboard';
import { usersPage } from '../pages/users';
import { html } from 'tina4js';

// Public
route('/login', loginPage);

// Protected
route('/', {
  guard: () => isLoggedIn.value || '/login',
  handler: dashboardPage,
});

route('/users', {
  guard: () => isLoggedIn.value || '/login',
  handler: usersPage,
});

// 404
route('*', () => html`
  <app-layout>
    <h1>404 - Page Not Found</h1>
    <a href="/">Go to Dashboard</a>
  </app-layout>
`);
```

Four routes. One is public. Two are guarded. One catches everything else. The guard is the same computed signal for both protected routes -- `isLoggedIn` returns true or redirects to `/login`. The entire routing table fits on one screen.

5. Layout Component

```
// src/components/app-layout.ts
import { Tina4Element, html } from 'tina4js';
import { user, isLoggedIn, clearAuth, unreadCount } from '../store';
import { navigate } from 'tina4js';

class AppLayout extends Tina4Element {
  // shadow=true so the documented <slot> projects route content correctly.
  // Light DOM (`shadow = false`) does not support slots in tina4-js.
  static shadow = true;

  render() {
    return html`
      <div style="min-height: 100vh; display: flex; flex-direction: column;">
        ${() => isLoggedIn.value
          ? html`
              <nav class="navbar" style="background: #1e293b; color: white; padding: 0.75rem 1.5rem;">
                <a class="navbar-brand" href="/" style="color: white; text-decoration: none; font-weight: bold;">Admin Dashboard</a>
                <div style="display: flex; align-items: center; gap: 1rem;">
                  <a href="/" style="color: #94a3b8; text-decoration: none;">Dashboard</a>
                  <a href="/users" style="color: #94a3b8; text-decoration: none;">Users</a>
                  <notification-bell></notification-bell>
                  <span style="color: #94a3b8;">${() => user.value?.name ?? ''}</span>
                  <button
                    class="btn btn-sm"
                    style="background: #475569; color: white; border: none;"
                    @click=${() => { clearAuth(); navigate('/login'); }}
                  >Logout</button>
                </div>
              </nav>
            `
          : null
        }
        <main style="flex: 1; padding: 1.5rem; max-width: 1200px; width: 100%; margin: 0 auto;">
          <slot></slot>
        </main>
      </div>
    `;
  }
}

customElements.define('app-layout', AppLayout);
```

The layout wraps every page. It renders the navbar when the user is logged in and hides it when they are not. The notification bell sits in the navbar. The user's name appears next to the logout button. The `<slot>` element passes through whatever the route handler renders.

One component. Every page gets a consistent shell.

6. Login Page

```
// src/pages/login.ts
import { signal, html, api, navigate, batch } from 'tina4js';
import { setAuth } from '../store';

export function loginPage() {
  const email = signal('', 'login-email');
  const password = signal('', 'login-password');
  const error = signal<string | null>(null, 'login-error');
  const loading = signal(false, 'login-loading');

  const handleLogin = async (e: Event) => {
    e.preventDefault();
    loading.value = true;
    error.value = null;

    try {
      const result = await api.post<{
        token: string;
        user: { id: number; name: string; role: string };
     }>('/auth/login', {
        email: email.value,
        password: password.value,
      });

      setAuth(result.token, result.user);
      navigate('/');
    } catch (err: any) {
      error.value = err.data?.message ?? 'Login failed. Please try again.';
    } finally {
      loading.value = false;
    }
  };

  return html`
    <div style="max-width: 400px; margin: 4rem auto; padding: 2rem;">
      <h1 style="text-align: center; margin-bottom: 2rem;">Admin Login</h1>

      ${() => error.value
        ? html`<div class="alert alert-danger">${error}</div>`
        : null
      }

      <form @submit=${handleLogin}>
        <div class="form-group">
          <label>Email</label>
          <input
            type="email"
            class="form-control"
            placeholder="admin@example.com"
            .value=${email}
            @input=${(e: Event) => { email.value = (e.target as HTMLInputElement).value; }}
            ?disabled=${loading}
            required
          />
        </div>

        <div class="form-group">
          <input
            type="password"
            class="form-control"
            placeholder="Password"
            .value=${password}
            @input=${(e: Event) => { password.value = (e.target as HTMLInputElement).value; }}
            required
          />
        </div>

        <button type="submit" class="btn btn-primary">Login</button>
      </form>
    </div>
  `;
}
```

The Intelligent Native Application Framework

```

        <label>Password</label>
        <input
            type="password"
            class="form-control"
            placeholder="Enter password"
            .value=${password}
            @input=${(e: Event) => { password.value = (e.target as HTMLInputElement).value; }}
            ?disabled=${loading}
            required
        />
    </div>

    <button
        type="submit"
        class="btn btn-primary"
        style="width: 100%"
        ?disabled=${loading}
    >
        ${() => loading.value ? 'Logging in...' : 'Login'}
    </button>
</form>
</div>
`;
}

```

Four signals drive this page. `email` and `password` hold the form state. `error` displays feedback. `loading` disables the form during the API call. The login handler calls the store's `setAuth` on success and navigates to the dashboard. On failure, the error signal updates and the alert appears.

No form library. No validation library. Signals and HTML.

7. Stat Card Component

```
// src/components/stat-card.ts
import { Tina4Element, html } from 'tina4js';

class StatCard extends Tina4Element {
  static props = { label: String, value: String, color: String };

  static styles = `
    :host { display: block; }
    .stat {
      padding: 1.5rem;
      border-radius: 8px;
      background: white;
      border: 1px solid #e2e8f0;
    }
    .label { font-size: 0.875rem; color: #64748b; margin-bottom: 0.25rem; }
    .value { font-size: 2rem; font-weight: 700; }
  `;

  render() {
    return html`
      <div class="stat">
        <div class="label">${this.prop('label')}</div>
        <div class="value" style=${() => `color: ${this.prop<string>('color').value || '#1e293b'}`}>
          ${this.prop('value')}
        </div>
      </div>
    `;
  }
}

customElements.define('stat-card', StatCard);
```

A small, focused component. Three props. Shadow DOM for style isolation. The value updates reactively when the parent passes a new attribute. Drop four of these in a grid and you have a statistics dashboard.

8. Notification Bell Component

```
// src/components/notification-bell.ts
import { Tina4Element, html, signal } from 'tina4js';
import { notifications, unreadCount } from '../store';

class NotificationBell extends Tina4Element {
  static styles = `
    :host { display: inline-block; position: relative; cursor: pointer; }
    .bell { font-size: 1.25rem; }
    .badge {
      position: absolute; top: -6px; right: -8px;
      background: #ef4444; color: white;
      font-size: 0.625rem; font-weight: 700;
      width: 18px; height: 18px;
      border-radius: 50%;
      display: flex; align-items: center; justify-content: center;
    }
    .dropdown {
      position: absolute; top: 100%; right: 0; margin-top: 0.5rem;
      background: white; border: 1px solid #e2e8f0; border-radius: 8px;
      width: 300px; max-height: 400px; overflow-y: auto;
      box-shadow: 0 4px 12px rgba(0,0,0,0.1); z-index: 100;
    }
    .item { padding: 0.75rem 1rem; border-bottom: 1px solid #f1f5f9; }
    .item:last-child { border-bottom: none; }
    .time { font-size: 0.75rem; color: #94a3b8; }
    .empty { padding: 1rem; text-align: center; color: #94a3b8; }
  `;

  render() {
    const open = signal(false);

    return html`
      <div @click=${() => { open.value = !open.value; }}>
        <span class="bell">🔔</span>
        ${() => unreadCount.value > 0
          ? html`<span class="badge">${unreadCount}</span>`
          : null
        }
      </div>

      ${() => open.value
        ? html`
          <div class="dropdown">
            ${() => notifications.value.length === 0
              ? html`<div class="empty">No notifications</div>`
              : null
            }
            ${() => notifications.value.map(n => html`
              <div class="item">
                <div>${n.text}</div>
                <div class="time">${n.time}</div>
              </div>
            `)}
            ${() => notifications.value.length > 0
              ? html`
                <div style="padding: 0.5rem; text-align: center;">
                  <button>
                    The Intelligent Native Application 4ramework
                  </button>
                </div>
              `
            : null
            }
          </div>
        `
        : null
      }
    `;
  }
}
```

```

        style="border: none; background: none; color: #3b82f6; cursor: pointer;"
        @click=${() => { notifications.value = []; }}
      >Clear all</button>
    </div>
  ,
  : null
}
</div>
,
: null
}
;
}
}
}

customElements.define('notification-bell', NotificationBell);

```

The bell reads from the global notification store. The badge appears when `unreadCount` is greater than zero. The dropdown opens on click. Each notification shows its text and timestamp. The "Clear all" button empties the array. All of this -- the badge count, the dropdown toggle, the notification list, the clear action -- runs on signals. No imperative state management. No event bus.

9. Dashboard Page with Live Stats

```
// src/pages/dashboard.ts
import { signal, computed, html, api, ws, batch } from 'tina4js';
import { user, notifications } from '../store';

export function dashboardPage() {
  // Stats from API
  const stats = signal({
    totalUsers: 0,
    activeUsers: 0,
    revenue: 0,
    orders: 0,
  }, 'dashboard-stats');

  const loading = signal(true, 'dashboard-loading');

  // Load initial stats
  async function loadStats() {
    loading.value = true;
    try {
      stats.value = await api.get('/dashboard/stats');
    } finally {
      loading.value = false;
    }
  }

  loadStats();

  // Live updates via WebSocket
  const socket = ws.connect(`${location.protocol === 'https:' ? 'wss:' : 'ws:'}//${location.host}`);

  // Pipe live stat updates
  socket.pipe(stats, (msg, current) => {
    const m = msg as { type: string; data: any };
    if (m.type === 'stats_update') {
      return { ...current, ...m.data };
    }
    return current;
  });

  // Pipe notifications
  socket.pipe(notifications, (msg, current) => {
    const m = msg as { type: string; text: string; time: string };
    if (m.type === 'notification') {
      return [{ id: Date.now(), text: m.text, time: m.time }, ...current].slice(0, 50);
    }
    return current;
  });

  // Recent activity
  const activity = signal<{ action: string; user: string; time: string }[]>([], 'recent-activity');

  socket.pipe(activity, (msg, current) => {
    const m = msg as { type: string; action: string; user: string; time: string };
    if (m.type === 'activity') {
      return [{ action: m.action, user: m.user, time: m.time }, ...current].slice(0, 20);
    }
    return current;
  });
}
```

```

});

return html`
  <app-layout>
    <h1>Dashboard</h1>
    <p style="color: #64748b; margin-bottom: 1.5rem;">
      Welcome back, ${() => user.value?.name ?? 'User'}
    </p>

    ${() => loading.value
      ? html`<p>Loading stats...</p>`
      : html`
        <div style="display: grid; grid-template-columns: repeat(4, 1fr); gap: 1rem; margin-
          <stat-card
            label="Total Users"
            value=${() => String(stats.value.totalUsers)}
            color="#3b82f6"
          ></stat-card>
          <stat-card
            label="Active Now"
            value=${() => String(stats.value.activeUsers)}
            color="#10b981"
          ></stat-card>
          <stat-card
            label="Revenue"
            value=${() => `$$${stats.value.revenue.toLocaleString()}`}
            color="#8b5cf6"
          ></stat-card>
          <stat-card
            label="Orders Today"
            value=${() => String(stats.value.orders)}
            color="#f59e0b"
          ></stat-card>
        </div>
      `}

    </div>

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem;">
      <div>
        <h2>Live Connection</h2>
        <p style="margin-bottom: 1rem;">
          Status:
          <span style=${() =>
            socket.status.value === 'open'
              ? 'color: #10b981'
              : 'color: #ef4444'
          }>${socket.status}</span>
        </p>
      </div>

      <div>
        <h2>Recent Activity</h2>
        <div style="max-height: 300px; overflow-y: auto;">
          ${() => activity.value.length === 0
            ? html`<p style="color: #94a3b8;">No recent activity</p>`
            : null
          }
          ${() => activity.value.map(a => html`
            <div style="padding: 0.5rem 0; border-bottom: 1px solid #f1f5f9;">

```

```

        <strong>${a.user}</strong> ${a.action}
        <span style="color: #94a3b8; font-size: 0.75rem; margin-left: 0.5rem;">${a.time}
      </div>
    `)}
  </div>
</div>
</div>
</app-layout>
`;
}

```

The page loads stats from the API, then opens a WebSocket for live updates. Three `pipe()` calls route incoming messages to three different signals: stats, notifications, and activity. The stat cards update in real time. Notifications push into the bell component. Activity entries stream into the feed.

The API provides the initial snapshot. The WebSocket provides the deltas. The signals merge both into a single reactive view.

10. CRUD Users Page

```
// src/pages/users.ts
import { signal, computed, html, api, batch } from 'tina4js';

interface User {
  id: number;
  name: string;
  email: string;
  role: string;
  active: boolean;
}

export function usersPage() {
  const users = signal<User[]>([], 'users-list');
  const loading = signal(true, 'users-loading');
  const search = signal('', 'users-search');
  const page = signal(1, 'users-page');
  const perPage = 10;

  // Editing state
  const editingUser = signal<User | null>(null, 'editing-user');
  const showCreateForm = signal(false, 'show-create-form');
  const formName = signal('');
  const formEmail = signal('');
  const formRole = signal('editor');
  const formError = signal<string | null>(null);

  // Filtered users
  const filtered = computed(() => {
    const q = search.value.toLowerCase();
    if (!q) return users.value;
    return users.value.filter(u =>
      u.name.toLowerCase().includes(q) ||
      u.email.toLowerCase().includes(q)
    );
  });

  // Paginated
  const paginated = computed(() => {
    const start = (page.value - 1) * perPage;
    return filtered.value.slice(start, start + perPage);
  });

  const totalPages = computed(() => Math.ceil(filtered.value.length / perPage));

  // Load
  async function loadUsers() {
    loading.value = true;
    try {
      users.value = await api.get<User[]>('/users');
    } finally {
      loading.value = false;
    }
  }

  // Create
  async function createUser() {
    formError.value = null;
    \_\_\_\_\_
  }
}
```

```

    try {
      const newUser = await api.post<User>('/users', {
        name: formName.value,
        email: formEmail.value,
        role: formRole.value,
      });
      batch(() => {
        users.value = [...users.value, newUser];
        showCreateForm.value = false;
        formName.value = '';
        formEmail.value = '';
        formRole.value = 'editor';
      });
    } catch (err: any) {
      formError.value = err.data?.message ?? 'Failed to create user';
    }
  }

  // Update
  async function updateUser() {
    if (!editingUser.value) return;
    formError.value = null;
    try {
      const updated = await api.put<User>(`/users/${editingUser.value.id}`, {
        name: formName.value,
        email: formEmail.value,
        role: formRole.value,
      });
      batch(() => {
        users.value = users.value.map(u => u.id === updated.id ? updated : u);
        editingUser.value = null;
      });
    } catch (err: any) {
      formError.value = err.data?.message ?? 'Failed to update user';
    }
  }

  // Delete
  async function deleteUser(id: number) {
    try {
      await api.delete(`/users/${id}`);
      users.value = users.value.filter(u => u.id !== id);
    } catch (err: any) {
      formError.value = err.data?.message ?? 'Failed to delete user';
    }
  }

  // Start editing
  function startEdit(user: User) {
    batch(() => {
      editingUser.value = user;
      showCreateForm.value = false;
      formName.value = user.name;
      formEmail.value = user.email;
      formRole.value = user.role;
      formError.value = null;
    });
  }
}

```

```

function cancelEdit() {
  batch(() => {
    editingUser.value = null;
    showCreateForm.value = false;
    formError.value = null;
  });
}

loadUsers();

return html`
  <app-layout>
    <div style="display: flex; justify-content: space-between; align-items: center; margin-bot
      <h1>Users</h1>
      <button class="btn btn-primary" @click=${() => {
        batch(() => {
          showCreateForm.value = true;
          editingUser.value = null;
          formName.value = '';
          formEmail.value = '';
          formRole.value = 'editor';
          formError.value = null;
        });
      }}>Add User</button>
    </div>

    ${() => formError.value
      ? html`<div class="alert alert-danger">${formError}</div>`
      : null
    }

    ${() => (showCreateForm.value || editingUser.value)
      ? html`
        <div class="card" style="margin-bottom: 1.5rem;">
          <div class="card-header">
            ${() => editingUser.value ? 'Edit User' : 'Create User'}
          </div>
          <div class="card-body">
            <div style="display: grid; grid-template-columns: 1fr 1fr 1fr auto; gap: 1rem; a
              <div class="form-group">
                <label>Name</label>
                <input class="form-control" .value=${formName}
                  @input=${(e: Event) => { formName.value = (e.target as HTMLInputElement).v
                </div>
              <div class="form-group">
                <label>Email</label>
                <input class="form-control" type="email" .value=${formEmail}
                  @input=${(e: Event) => { formEmail.value = (e.target as HTMLInputElement).
                </div>
              <div class="form-group">
                <label>Role</label>
                <select class="form-control" .value=${formRole}
                  @change=${(e: Event) => { formRole.value = (e.target as HTMLSelectElement)
                  <option value="admin">Admin</option>
                  <option value="editor">Editor</option>
                  <option value="viewer">Viewer</option>
                </select>
              </div>
            <div style="display: flex; gap: 0.5rem;">

```



```

        <button class="btn btn-primary"
            @click=${() => editingUser.value ? updateUser() : createUser()}>
            ${() => editingUser.value ? 'Update' : 'Create'}
        </button>
        <button class="btn" @click=${cancelEdit}>Cancel</button>
    </div>
</div>
</div>
</div>
</div>
,
: null
}

<div style="margin-bottom: 1rem;">
    <input
        type="search"
        class="form-control"
        placeholder="Search users..."
        .value=${search}
        @input=${(e: Event) => {
            search.value = (e.target as HTMLInputElement).value;
            page.value = 1;
        }}
    />
</div>

${() => loading.value
    ? html`<p>Loading users...</p>`
    : html`
        <table class="table table-striped table-hover">
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Email</th>
                    <th>Role</th>
                    <th>Status</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                ${() => paginated.value.map(u => html`
                    <tr>
                        <td>${u.name}</td>
                        <td>${u.email}</td>
                        <td><span class="badge">${u.role}</span></td>
                        <td>
                            <span class=${`badge ${u.active ? 'badge-success' : 'badge-danger'}`}>
                                ${u.active ? 'Active' : 'Inactive'}
                            </span>
                        </td>
                        <td>
                            <button class="btn btn-sm" @click=${() => startEdit(u)}>Edit</button>
                            <button class="btn btn-sm btn-danger" @click=${() => deleteUser(u.id)}>Del
                        </td>
                    </tr>
                `)}
            </tbody>
        </table>
    `

```

C

r

...

1

Module	Usage
Signals	<code>token</code> , <code>user</code> , <code>notifications</code> , <code>stats</code> , <code>users</code> , form state
Computed	<code>isLoggedInIn</code> , <code>isAdmin</code> , <code>filtered</code> , <code>paginated</code> , <code>totalPages</code> , <code>unreadCount</code>
Effects	Debug overlay auto-tracking, signal subscriptions in templates
Batch	Grouping form state resets, edit mode transitions
html	Every page and component template
Tina4Element	<code>stat-card</code> , <code>notification-bell</code> , <code>app-layout</code>
Routing	Login, dashboard, users, 404 with guards
API	Login, CRUD operations, stats loading, 401 interceptor

WebSocket	Live stats, notifications, activity feed
PWA	Installable with network-first caching
Debug	Labels on every signal, conditional import

Every module. One application. Deployable as-is.

The entire frontend -- framework, components, routes, state management, everything -- ships under 10KB gzipped. Try building this dashboard with React, Vue, or Angular. Count the packages. Measure the bundle.

Summary

A complete, production-grade admin dashboard does not need 50 npm packages, a state management library, a routing library, a form library, and a CSS-in-JS solution. It needs tina4-js and clear thinking.

The patterns repeat:

- **Signals for state.** One signal per piece of data.
- **Computed for derived data.** Filtering, pagination, auth checks.
- **Batch for grouped writes.** Form resets, mode transitions.
- **html for templates.** Reactive text, reactive blocks, events.
- **Components for reuse.** Props in, events out.
- **Routes with guards.** Public and protected.
- **API with interceptors.** Auth, error handling.
- **WebSocket for live data.** Pipe into signals.

Eight patterns. One framework. Now build your own.

Patterns and Pitfalls

What We Learned the Hard Way

You push a feature to production. The list does not update when items are added. You stare at the code for ten minutes. The logic is correct. The API returns the right data. The signal receives the value. But the DOM does not move.

Then you see it: `items.value.push(newItem)`. A mutation. No new reference. The signal never fired.

Every pitfall in this chapter was hit by a real developer on a real project. Every pattern was discovered through trial and error. This is the chapter you read before you ship -- and the chapter you return to when something breaks and you cannot figure out why.

1. When to Use tina4-js

Use tina4-js when:

- **Bundle size matters.** 1.5KB core gzipped, under 6KB for the full framework. Your app loads in under a second on a 3G connection.
- **You want simplicity.** Seven modules. One package. No decisions about state management libraries, routing libraries, or CSS-in-JS solutions.
- **You are building for a tina4 backend.** The API client speaks tina4-php and tina4-python natively -- auth tokens, CSRF protection, and token rotation work without configuration.
- **You want real Web Components.** Components that work outside the framework, in plain HTML, in other frameworks, in any context where custom elements are supported.
- **You are building islands.** Adding interactivity to server-rendered pages without taking over the entire DOM.
- **You are learning.** The entire source code is under 1,000 lines. You can read every module in an afternoon and understand how the framework works at the implementation level.

Use React, Vue, or Svelte when:

- **You need server-side rendering (SSR).** tina4-js is client-side only. If you need the server to render HTML for SEO or initial load performance, use a framework with SSR support.
- **You need a massive ecosystem.** Thousands of third-party components, hooks, plugins, and integrations.
- **Your team already knows React/Vue.** Developer familiarity has real value. Retraining a team has real cost.
- **You need fine-grained list reconciliation.** tina4-js re-renders entire lists when the signal changes. For lists of thousands of items with frequent individual updates, keyed reconciliation (React's virtual DOM, Svelte's each blocks) is the better tool.

Pick the right tool for the job. tina4-js is not trying to replace React. It is trying to replace the 90% of projects that never needed React in the first place.

2. The New-Reference Rule

This is the most important rule in tina4-js. It trips up everyone -- beginners and experienced developers alike.

Signals use `Object.is()` for change detection. `Object.is()` compares by reference for objects and arrays. If you mutate in place, the reference does not change. The signal does not fire. The DOM does not update. Your code is correct in every way except the one that matters.

```
// These do NOT trigger updates:
items.value.push(newItem);
items.value.splice(0, 1);
items.value.sort();
items.value[0] = 'new';
user.value.name = 'Alice';
config.value.theme = 'dark';

// These DO trigger updates:
items.value = [...items.value, newItem];
items.value = items.value.filter((_, i) => i !== 0);
items.value = [...items.value].sort();
items.value = items.value.map((item, i) => i === 0 ? 'new' : item);
user.value = { ...user.value, name: 'Alice' };
config.value = { ...config.value, theme: 'dark' };
```

The pattern: create a new array or object. The spread operator is your primary tool. Every write to an array or object signal should produce a new reference. No exceptions.

If this feels wasteful, it is not. JavaScript engines optimize short-lived object allocations. The performance cost of spreading an array of 100 items is negligible compared to the debugging cost of a mutation that silently fails.

Helper Functions

If you find yourself writing the same spread patterns over and over, make helpers:

```
function pushSignal<T>(sig: Signal<T[]>, item: T) {
  sig.value = [...sig.value, item];
}

function removeSignal<T>(sig: Signal<T[]>, index: number) {
  sig.value = sig.value.filter((_, i) => i !== index);
}

function updateSignal<T extends object>(sig: Signal<T>, partial: Partial<T>) {
  sig.value = { ...sig.value, ...partial };
}
```

Three functions. They encode the new-reference rule so you do not have to think about it on every write.

3. Computed Is Eager, Not Lazy

In some frameworks, computed values are lazy -- they recalculate only when you read them. In tina4-js, computed values are eager. They recalculate the moment a dependency changes.

This means:

```
const expensive = computed(() => {
  // This runs EVERY TIME any dependency changes,
  // even if nobody is reading expensive.value
  return heavyCalculation(data.value);
});
```

If you have an expensive computation that is only needed in certain states:

```
// Instead of computed, use a signal + effect with a guard
const result = signal<Result | null>(null);
const needsResult = signal(false);

effect(() => {
  if (needsResult.value) {
    result.value = heavyCalculation(data.value);
  }
});
```

Or compute on demand without caching:

```
function getExpensiveResult() {
  return heavyCalculation(data.value);
}
```

For most computed values -- filtering lists, calculating totals, deriving display strings -- eagerness is fine. The recalculation takes microseconds. You only need to worry about this when the computation involves heavy iteration, network-dependent data, or large dataset transformations. If the computed function takes more than a millisecond, consider guarding it.

4. Inputs Must Stay Outside Reactive Blocks

This is the most common tina4-js bug. Wrapping `<input>` elements inside `${() => ...}` reactive blocks causes them to lose focus on every keystroke. The reactive block destroys and recreates its entire DOM subtree when any signal it reads changes, so the input element is replaced mid-typing.

The rule: Form elements (`<input>`, `<textarea>`, `<select>`) go in the **static** part of the template. Use `.value`, `@input`, and `?disabled` bindings to make them reactive. Only wrap **computed output** (conditional messages, dropdown options, dynamic lists) in `${() => ...}`.

```
// WRONG - input inside reactive block, loses focus on every keystroke
html` ${() => html`<input .value=${name} @input=${(e: Event) => {
  name.value = (e.target as HTMLInputElement).value;
}} />` }
```

```
// RIGHT - input in static template, only output is reactive
html`
```

```
  <input .value=${name} @input=${(e: Event) => {
    name.value = (e.target as HTMLInputElement).value;
  }} />`
```

The Intelligent Native Application 4ramework

```

    name.value = (e.target as HTMLInputElement).value;
  }} />
  <p>${() => name.value ? `Hello, ${name.value}!` : 'Type your name'}</p>

```

5. Event Handler Auto-Batching

Since v1.0.9, all `@event` handlers in templates are wrapped in `batch()` for you. This means:

```

html`
  <button @click=${() => {
    a.value = 1;
    b.value = 2;
    c.value = 3;
  }}>Update</button>
`

// One DOM update, not three

```

You do NOT need to wrap event handlers in `batch()`. It happens behind the scenes.

But this applies only to handlers registered with the `@event` syntax in `html` templates. If you add event listeners through other means, you need explicit batching:

```

// Manual addEventListener -- NOT auto-batched
element.addEventListener('click', () => {
  batch(() => {
    a.value = 1;
    b.value = 2;
  });
});

```

The same applies to:

- `setTimeout` / `setInterval` callbacks
- `fetch().then()` handlers
- WebSocket `on('message')` handlers
- `Promise` handlers
- `requestAnimationFrame` callbacks

Any code that runs outside the `@event` handler context needs explicit `batch()` when writing to multiple signals. When in doubt, wrap in `batch()`. Batching an already-batched operation is harmless -- batches nest without side effects.

6. Effect Cleanup on Route Navigation

When the router navigates to a new route, it disposes all effects that were created during the previous route's handler execution. This is automatic and prevents memory leaks.

But it only disposes **effects**. It does not clean up:

- `setInterval` / `setTimeout` timers

The Intelligent Native Application Framework

- Manual `addEventListener` calls
- WebSocket connections
- Third-party library instances

For those, use a component with `onUnmount()`:

```
class LiveWidget extends Tina4Element {
  private interval = 0;
  private socket: ManagedSocket | null = null;

  onMount() {
    this.interval = window.setInterval(() => { /* ... */ }, 1000);
    this.socket = ws.connect('wss://...');
  }

  onUnmount() {
    clearInterval(this.interval);
    this.socket?.close();
  }

  render() { /* ... */ }
}
```

When the route changes and this component leaves the DOM, `onUnmount()` fires. The timer stops. The socket closes. No lingering connections. No phantom intervals ticking in the background.

The rule: if you create it in a component, destroy it in `onUnmount()`. If you create it in a route handler, put it in a component so `onUnmount()` can reach it.

7. Do Not Mix `addEventListener` Inside Reactive Blocks

```
// WRONG -- adds a new listener every time count changes
html`
  ${() => {
    document.addEventListener('keydown', handleKey); // LEAK!
    return html`<p>Count: ${count.value}</p>`;
  }}
`
```

Every time the reactive block re-runs, a new event listener is added. The old ones are never removed. After ten signal updates, ten listeners fire on every keypress. After a hundred updates, a hundred listeners fire.

Instead:

```
// RIGHT -- add the listener once, outside the template
document.addEventListener('keydown', handleKey);

html`<p>${count}</p>`
```

Or use a component:

```
class KeyHandler extends Tina4Element {
  private handler = (e: KeyboardEvent) => { /* ... */ };
}
```

The Intelligent Native Application Framework


```

onMount() {
  document.addEventListener('keydown', this.handler);
}

onUnmount() {
  document.removeEventListener('keydown', this.handler);
}

render() { return html`<slot></slot>`; }
}

```

The component adds the listener once on mount and removes it on unmount. Clean lifecycle. No leaks. No accumulation.

8. The false/null Rendering Trap

This is covered in Chapter 3 but deserves repetition because it causes subtle bugs that pass code review:

```

// WRONG -- renders "false" as text when show is false
html`${show.value} && html`<div>Content</div>`

// RIGHT -- returns null for nothing
html`${() => show.value ? html`<div>Content</div>` : null}`

```

The `&&` pattern comes from React JSX, where `false` is a rendering no-op. In tina4-js, `false` converts to the text `"false"` and displays on screen. The word "false" appears in your UI. Always use ternary. Always return `null` for "render nothing."

The same trap applies to zero:

```

// WRONG -- shows "0" when count is zero
html`${count.value} && html`<p>${count.value} items</p>`

// RIGHT
html`${() => count.value > 0 ? html`<p>${count.value} items</p>` : null}`

```

`0 && anything` evaluates to `0`, which renders as the text `"0"`. A zero appears in your UI with no context. The ternary with an explicit comparison avoids both traps.

9. Signal Scope and Lifetime

Signals live until they are garbage collected. If nothing references a signal, the runtime cleans it up. But if an effect or DOM binding holds a reference, the signal stays alive.

Avoid Global Signals for Temporary State

```
// BAD -- these signals live forever
const formName = signal('');
const formEmail = signal('');

route('/users/new', () => {
  return html`<input .value=${formName} ... />`;
});
```

When the user navigates away and comes back, `formName` still holds the old value. The form is pre-filled with stale data. The user did not expect that.

```
// GOOD -- signals scoped to the route handler
route('/users/new', () => {
  const formName = signal('');
  const formEmail = signal('');

  return html`<input .value=${formName} ... />`;
});
```

Each visit creates fresh signals. The old ones are garbage collected when the route changes and effects are disposed. The form starts empty every time.

Use Global Signals for Shared State

```
// store.ts -- these SHOULD be global
export const user = signal<User | null>(null, 'user');
export const token = signal<string | null>(null, 'token');
export const theme = signal<'light' | 'dark'>('light', 'theme');
```

Global signals are appropriate for state that persists across route changes: authentication, user preferences, cart contents, notification counts. The distinction is simple: if the data should survive navigation, make it global. If the data belongs to a single page visit, scope it to the route handler.

10. Debugging Techniques

Add Labels to Every Signal

```
const count = signal(0, 'count');          // DO THIS
const count = signal(0);                    // NOT THIS
```

When you open the debug overlay and see 20 unnamed signals, you will spend more time figuring out which signal is which than fixing the bug you came to investigate.

Check Subscriber Counts

In the debug overlay, a signal with 0 subscribers is either:

- Unused (remove it)
- Bound incorrectly (`${count.value}` instead of `${count}`)

A signal with an unexpectedly high subscriber count might be referenced in a reactive block that re-runs often, creating new subscriptions each time. The subscriber count is the fastest way to spot a binding error or a [subscription leak](#).

Use effect() for Debugging

```
effect(() => {
  console.log('User changed:', user.value);
  console.log('Token changed:', token.value);
});
```

This logs whenever `user` or `token` changes. It captures every write, from every source, in chronological order. Remove it before shipping -- but while debugging, it is more reliable than breakpoints because it shows you the data flow without pausing execution.

11. Performance Patterns

Keep Lists Reasonable

tina4-js re-renders entire lists when the signal changes. For lists under 200 items, the re-render takes under a millisecond. For larger lists:

- Paginate on the server (return 20 items per page, not 10,000)
- Use virtual scrolling (a separate library that renders only visible rows)
- Keep the visible list small with filtering and pagination

Avoid Unnecessary Computed

```
// UNNECESSARY -- computed with one dependency that transforms a value
const upperName = computed(() => name.value.toUpperCase());

// SIMPLER -- just use a function in the template
html`<p>${() => name.value.toUpperCase()}</p>`
```

Use computed when multiple parts of your app need the derived value. Use inline functions when only one template needs it. A computed signal adds a subscription, a cached value, and a node in the dependency graph. An inline function adds a function call. For single-use derivations, the function is the lighter tool.

Batch Async Results

```
// Multiple signal writes from an API response
const data = await api.get('/dashboard');
batch(() => {
  users.value = data.users;
  stats.value = data.stats;
  lastUpdated.value = new Date().toISOString();
});
```

Without batch, each assignment triggers a separate DOM update. Three writes, three re-renders. With batch, three writes, one re-render. The difference is invisible for two signals. It is visible for ten.

12. Common Error Messages

"[tina4] computed signals are read-only"

You tried to write to a computed signal. Computed values are derived -- they calculate from source signals. Write to the source signals instead, and the computed value updates on its own.

"[tina4] WebSocket is not connected"

You called `socket.send()` when the socket was not open. Check `socket.connected.value` before sending, or queue messages and flush them when the `status` signal changes to `'open'`.

"[tina4] Router target '...' not found in DOM"

The CSS selector you passed to `router.start({ target })` does not match any element. The element must exist in your HTML before `router.start()` runs. If you are using a component that renders the target element, make sure the component mounts before the router starts.

"[tina4] Prop '...' not declared in static props"

You called `this.prop('name')` but did not declare `name` in the component's `static props` object. The framework requires prop declarations so it can set up attribute observation. Add the prop to the static object and the error disappears.

13. Reactive `<select>`: Bind `?selected`, Not `.value`

A `<select>` whose options come from a `$(() => ...)` block loses its selection when those options re-render. Binding `.value` on the select is a separate effect: it tracks the bound signal, not the option list, so it never re-fires when the options rebuild. And setting `select.value` while the matching option is missing (the options render later, or get torn down and recreated) does nothing, then the browser clears the selection when the child list changes. The selection holds by luck when the timing lines up, and drops when it does not.

Bind the selected state on each option instead. Every `<option>` gets its own `?selected` effect, so a re-render rebuilds the list already correct, in any order.

```
// Fragile: selection drops when the options re-render
html`<select .value=${currency} @change=${e => { currency.value = e.target.value; }}>
  ${() => currencies.value.map(c => html`<option value=${c}>${c}</option>`)}
</select>`

// Robust: each option owns its selected state
html`<select @change=${e => { currency.value = e.target.value; }}>
  ${() => currencies.value.map(c => html`
    <option value=${c} ?selected=${() => c === currency.value}>${c}</option>`)}
</select>`
```

Keep the `@change` handler: it writes the choice back to the signal. Option values are always strings, so when the signal holds a number, compare `String(c) === currency.value`. Otherwise the match never fires and the field shows nothing selected.

Summary

Pitfall	Fix
Mutating arrays/objects in place	Always create new references (spread)
<code>\${count.value}</code> in templates	Use <code>\${count}</code> (pass the signal)
<code>\${condition && html\...\}</code>	Use ternary: <code>\${() => cond ? html\...\ : null}</code>
<code>addEventListener</code> in reactive blocks	Add listeners once, outside templates
No cleanup for timers/sockets	Use <code>onUnmount()</code> in components
Expensive computed	Guard with a condition or compute on demand
Unnamed signals	Always add debug labels
Multiple writes without batch	Wrap async signal writes in <code>batch()</code>
Global signals for form state	Scope signals to route handlers or components
<code>.value</code> on a <code><select></code> with reactive options	Bind <code>?selected</code> on each <code><option></code> instead

Vibe Coding with AI

Let AI Write Your tina4-js

You type a prompt: "Build a todo app." The AI generates 200 lines of code. You paste it in. The build fails. The state management is wrong. The router syntax is backwards. You spend 30 minutes fixing what the AI wrote. You could have written it from scratch in 20.

That is what happens with ambiguous frameworks. tina4-js is different. One state primitive. One template system. One router. One way to do everything. The AI has no wrong choices to make -- and the code it generates works on the first try.

1. Why tina4-js Works Well with AI

Most JavaScript frameworks are ambiguous. React offers three ways to manage state (useState, useReducer, external stores). Vue offers four ways to style components (scoped, modules, Tailwind, CSS-in-JS). An AI agent faces these choices and guesses. It often guesses wrong.

tina4-js has one way to do everything:

- One state primitive: `signal()`
- One template system: `html` tagged templates
- One component base: `Tina4Element`
- One router: `route()` + `router.start()`
- One API client: `api`
- One WebSocket client: `ws`

One choice. One answer. One correct output.

When an AI reads the tina4-js documentation, there is no ambiguity. No "it depends on your preference." No "you could also use X." The AI generates correct code on the first try because there is only one way to write it.

2. CLAUDE.md -- The AI's Instruction Manual

Every tina4-js project includes a `CLAUDE.md` file at the root. AI assistants (Claude Code, Cursor, GitHub Copilot) read this file when they open the project.

`CLAUDE.md` contains:

- **Build and test commands** -- `npm run build`, `npm run test`, `npm run dev`
- **Project structure** -- where signals, components, routes, and pages live
- **Key conventions** -- `route(pattern, handler)` pattern-first, `api.get(path, options)` with `{ params, headers }`

The Intelligent Native Application 4ramework

- **Template binding syntax** -- the complete table of `${signal}`, `@click`, `?disabled`, `.value`, etc.
- **Package exports** -- all seven entry points
- **Publishing instructions** -- version, build, publish

When an AI opens a tina4-js project and reads `CLAUDE.md`, it knows:

- How to build and test
- Where to put new code
- What syntax to use in templates
- What the API signatures look like
- What pitfalls to avoid

No guessing. No hallucinating method names. No inventing conventions. The instruction manual is in the repository, and the AI reads it before writing a single line.

3. `llms.txt` -- AI Context for the Framework

The tina4-js npm package ships an `llms.txt` file. This is a standardized format that AI tools consume to understand a library's API. It includes:

- Every exported function with its signature
- Every interface and type
- Usage examples for each module
- Common patterns

AI agents that support `llms.txt` load this as context when generating tina4-js code, even if they have never encountered the framework before. The file bridges the gap between "I have never seen this library" and "I know every function signature and every convention."

4. The tina4-js Skill

Beyond `CLAUDE.md` and `llms.txt`, tina4-js ships with an AI skill file at `.claude/skills/tina4-js/SKILL.md`. This is a deep reference that teaches AI agents the three rules that fix 90% of mistakes:

Rule 1: Static vs Reactive

```
// WRONG -- AI often generates this
html`<p>${count.value}</p>`
```

```
// RIGHT -- the skill teaches this
html`<p>${count}</p>`
```

AI agents love to be explicit. They write `count.value` because it shows they are reading the signal's current value. But in tina4-js templates, `.value` makes the binding static -- evaluated once, never updated. Passing the signal itself creates a reactive binding that updates when the signal changes. The skill file drills this distinction until the AI internalizes it.

Rule 2: New References

```
// WRONG -- AI loves Array.push()
items.value.push(newItem);

// RIGHT
items.value = [...items.value, newItem];
```

AI agents trained on React often generate mutation patterns because React's `useState` works with `setState(prev => [...prev, item])`. In tina4-js, the signal itself must receive a new reference. No setter function wraps the mutation. The signal compares references. The skill teaches the spread pattern for every array and object operation.

Rule 3: Boolean Attributes

```
// WRONG -- AI writes this because it looks logical
html`<button disabled=${isDisabled}>Click</button>`

// RIGHT
html`<button ?disabled=${isDisabled}>Click</button>`
```

Without the `?` prefix, `disabled` is set as a string attribute. `disabled="false"` still disables the button in HTML -- the attribute's presence is what matters, not its value. The `?` prefix tells tina4-js to add or remove the attribute based on the signal's truthiness. The skill file explains this with examples that prevent the mistake before it happens.

5. What AI Gets Wrong

Even with the skill file, AI agents make these mistakes. Watch for them in generated code:

Using .value in Templates

```
// AI generates:
html`<p>Count: ${count.value}</p>`
html`<button ?disabled=${loading.value}>Submit</button>`

// Should be:
html`<p>Count: ${count}</p>`
html`<button ?disabled=${loading}>Submit</button>`
```

When you see `.value` inside a template interpolation, it is almost always wrong. The signal itself should be passed for reactive binding. The `.value` version renders once and freezes.

The && Pattern

```
// AI generates (React habit):
html`${isLoading.value} && html`<p>Loading...</p>`

// Should be:
html`${() => isLoading.value ? html`<p>Loading...</p>` : null}`
```

The Intelligent Native Application 4ramework

AI agents trained on JSX use `&&` for conditional rendering. In tina4-js, `false` renders as the text "false." The ternary with `null` is the correct pattern. Every conditional in a template should use `() =>` with a ternary.

Missing @event Syntax

```
// AI generates:
html`<button onclick=${handler}>Click</button>`

// Should be:
html`<button @click=${handler}>Click</button>`
```

The `@` prefix is tina4-js syntax for event binding. Native `onclick` as an attribute does not work with template interpolation. The AI sometimes falls back to vanilla HTML event attributes out of habit.

Forgetting Reactive Blocks for Conditionals

```
// AI generates:
html`
  <div>
    ${showDetails ? html`<p>Details</p>` : html`<p>Summary</p>`}
  </div>
`

// Should be:
html`
  <div>
    ${() => showDetails.value ? html`<p>Details</p>` : html`<p>Summary</p>`}
  </div>
`
```

The function wrapper `() =>` is required for the template to re-evaluate when `showDetails` changes. Without it, the condition evaluates once at render time and never updates. The page stays frozen on whichever branch was true at creation.

Using route() with Handler First

```
// AI sometimes generates:
route(() => html`<h1>Home</h1>`, '/');

// Should be:
route('/', () => html`<h1>Home</h1>`);
```

Pattern first. Handler second. Always.

6. Working with Claude Code on tina4-js Projects

Claude Code reads `CLAUDE.md` on project open. When you work on a tina4-js project in Claude Code, the AI knows the framework. Here is how to get the best results:

Be Specific About Modules

"Add a route at `/users/{id}` that fetches the user from `/api/users/:id` and displays their name and

This tells Claude which modules to use (route, api, html) and what the result should look like. Specific prompts produce specific code.

Reference Existing Patterns

"Add a new page similar to src/pages/users.ts but for products"

Claude reads the existing file and replicates the pattern -- same signal structure, same API calls, same template style. Your codebase stays consistent.

Let It Run the Dev Server

"Create a counter component and show me it works"

Claude creates the component, adds it to a route, and runs `npm run dev` to verify. The feedback loop is immediate.

Review Generated Templates

After Claude generates tina4-js code, scan for five things:

- `.value` inside `${}` in templates -- should it be the signal itself?
- `&&` patterns -- should it be a ternary with `null`?
- `disabled=` without `?` prefix -- should it be `?disabled=`?
- Missing `() =>` wrappers around conditional expressions
- Array mutations without new references

Five checks. Ten seconds. They catch the mistakes that would cost you ten minutes of debugging.

7. The AI Advantage

tina4-js was designed with AI-assisted development in mind. The framework's constraints are the AI's strengths:

- **One way to do things** means the AI never picks the wrong approach
- **Small API surface** means the AI does not hallucinate non-existent methods
- **Consistent conventions** mean generated code matches existing code
- **CLAUDE.md and skills** mean the AI has perfect context from the first prompt

Tell an AI: "Build a todo app with tina4-js." The AI needs to decide on state management, components, routing, styling, and API communication. With React, each decision has three to five options. The AI picks a combination. It might work. It might not. With tina4-js, there is one answer for each decision. The AI generates working code faster with fewer errors because fewer decisions means fewer wrong decisions.

This is not accidental. When every tina4 project follows the same structure, uses the same conventions, and ships the same instruction files, AI assistance becomes reliable. Not probabilistic. Not "usually right." Reliable.

The Intelligent Native Application 4ramework

8. Setting Up Your Project for AI

To get the best AI experience with tina4-js:

- **Keep `CLAUDE.md` up to date.** If you add custom conventions -- naming patterns, file organization rules, API endpoint structures -- document them. The AI reads what you write.
- **Use debug labels on signals.** AI agents read your code to understand patterns. `signal(0, 'cart-count')` communicates intent. `signal(0)` communicates nothing.
- **Follow the project structure.** `src/routes/`, `src/pages/`, `src/components/`, `src/store.ts`. When the AI sees consistent structure, it generates code that fits. When the structure is chaotic, the AI generates code that adds to the chaos.
- **Write TypeScript.** Type annotations give AI agents precise context about what functions expect and return. A typed interface is worth a paragraph of explanation.
- **Use the store pattern.** Export signals from `store.ts`. AI agents recognize this pattern and replicate it. Scattered signals in random files confuse AI agents the same way they confuse human developers.

One framework. One structure. One set of conventions. The AI becomes an extension of your thinking instead of a source of surprises.

Summary

What	Where
Project instructions	<code>CLAUDE.md</code> at project root
Framework API reference	<code>llms.txt</code> in the npm package
Deep AI skill	<code>.claude/skills/tina4-js/SKILL.md</code>
Common AI mistake 1	<code>\${count.value}</code> instead of <code>\${count}</code>
Common AI mistake 2	<code>&&</code> pattern instead of ternary with <code>null</code>
Common AI mistake 3	<code>disabled=</code> instead of <code>?disabled=</code>
Common AI mistake 4	Missing <code>() =></code> for reactive conditionals
Common AI mistake 5	Array mutation instead of new reference

Review the AI's work with the Tina4 Code Viewer

When the AI has made its pass, review before you merge. The [Tina4 Code Viewer](#) is a read-only desktop app that fingerprints your Tina4 layout, lets you leave line-anchored comments grounded against the Tina4 RAG, and exports a portable bundle the next agent can pick up and fix. It views and comments, it never edits your code. Signed builds for macOS, Windows, and Linux.

The Intelligent Native Application 4ramework

Internationalization

One App, Every Language

Your app ships to Cape Town, Paris, and Cairo. The prices need commas in one place and full stops in another. The dates flip their day and month. One language reads right to left. You could special-case all of it by hand, or you could let the browser do the work it already knows how to do.

tina4-js gives you a small i18n module built on two things you already have: signals and the browser's native [Intl](#). The active locale is a signal. Translations and formatters read it. Switch the locale and every translated string and every formatted number updates in place. No reload, no locale data to download.

This chapter covers translations, number and date formatting, right-to-left text, and how to load language bundles at runtime.

Create an instance

[createI18n\(\)](#) returns an i18n instance. Give it a locale, a fallback, and your message bundles.

```
import { createI18n } from "tina4js";

const i18n = createI18n({
  locale: "en-US",
  fallbackLocale: "en-US",
  messages: {
    "en-US": {
      greeting: "Hello",
      welcome: "Welcome, {name}!",
      nav: { home: "Home", about: "About" },
    },
    "fr-FR": {
      greeting: "Bonjour",
      welcome: "Bienvenue, {name}!",
      nav: { home: "Accueil" },
    },
  },
});
```

Leave [locale](#) out and the instance reads [navigator.language](#), falling back to [en](#). The [fallbackLocale](#) is the locale consulted when a key is missing in the active one.

Translate

[t\(key\)](#) returns the string for the active locale. Pass a params object for [{placeholder}](#) interpolation.

```
i18n.t("greeting"); // "Hello"
i18n.t("welcome", { name: "Alice" }); // "Welcome, Alice!"
```

Nested messages are flattened on the way in, and you can reach them two ways: the full dot-path, or the leaf key on its own.

The Intelligent Native Application Framework

```
i18n.t("nav.home"); // "Home"
i18n.t("home");     // "Home" -- leaf shortcut
```

When a key is missing, the lookup walks a clear path: the active locale, then the fallback locale, then the key itself. It never throws. A missing translation shows you the key, which is exactly the breadcrumb you want in the console.

Switch the locale, and everything updates

The active locale is a signal. Read `t()` inside the reactive function form, and the moment you call `setLocale()` the DOM re-renders.

```
import { html } from "tina4js";

html`
  <h1>${() => i18n.t("greeting")}</h1>
  <button @click=${() => i18n.setLocale("fr-FR")}>Francais</button>
`;
```

Click the button. The heading changes from "Hello" to "Bonjour". You wrote no update code. The `${() => ...}` form is what makes it reactive: `${i18n.t("greeting")}` would format once and freeze.

Format numbers, currency, and dates

Translations cover words. Numbers, money, and dates are a different problem, and the browser already solves it. The formatters delegate to `Intl`, so there is no formatting data to ship, and they re-render reactively just like `t()`.

```
i18n.number(1234.5); // "1,234.5" (en-US) / "1.234,5" (de-DE)
i18n.currency(1999.5, "USD"); // "$1,999.50" (en-US)
i18n.date(new Date(), { dateStyle: "medium" });
i18n.relativeTime(-1, "day"); // "yesterday"
```

Each takes the same options object the underlying `Intl` constructor takes, so the full power of `Intl.NumberFormat` and `Intl.DateTimeFormat` is one argument away. `date()` accepts a `Date`, an epoch in milliseconds, or a parseable string.

In a template:

```
html`<p class="total">${() => i18n.currency(cart.total.value, "USD")}</p>`;
```

Right to left

Arabic, Hebrew, Farsi, and Urdu read right to left. The instance knows which locales those are. Ask it for the direction and bind it to a container.

```
html`<div dir=${() => i18n.dir()}>...</div>`; // "ltr" or "rtl"
```

`isRTL()` returns the boolean if you need it directly. Add your own locales with the `rtlLocales` option when you create the instance.

Load bundles at runtime

Shipping every translation in your main bundle is wasteful when a user reads one language. Fetch a bundle on demand instead.

```
await i18n.loadMessages("es-ES", "/i18n/es-ES.json");
i18n.setLocale("es-ES");
```

`loadMessages` fetches the JSON, merges it into the locale, and leaves the rest untouched. Pull bundles from your Tina4 backend, which has its own `i18n`, so the server and the browser read the same shape.

The default instance

Most apps configure one `i18n`. Import the default singleton and the shortcut functions, and skip passing the instance around.

```
import { i18n, t, setLocale } from "tina4js/i18n";

i18n.addMessages("en", { greeting: "Hello" });
i18n.addMessages("fr", { greeting: "Bonjour" });

t("greeting"); // delegates to the default instance
setLocale("fr"); // every t() in the app re-renders
```

`createI18n` is still there when you want an isolated instance for a widget or a test.

The full API

Method	What it does
<code>t(key, params?)</code>	Translate, with <code>{placeholder}</code> interpolation
<code>setLocale(code) / getLocale()</code>	Switch / read the active locale
<code>addMessages(locale, obj)</code>	Merge a bundle into a locale
<code>loadMessages(locale, url)</code>	Fetch a JSON bundle and merge it
<code>hasLocale(code) / availableLocales()</code>	Check / list loaded locales
<code>number / currency / date / relativeTime</code>	Intl-backed formatting
<code>isRTL() / dir()</code>	Direction for the active locale
<code>locale</code>	The active locale as a reactive signal

One module, one signal, and the browser's own Intl engine. Your app speaks every language your users do, and it never reloads to do it.

Real-time (WebRTC)

Peer-to-Peer Without a Media Server

A user clicks "Join call" and sees three other faces. Someone drops a file into the chat and it appears on every screen. Another person shares their terminal. All of it happens in the browser, live, with no polling and no round-trip through a media server you had to build and scale.

The reflex is to reach for a stack: an SFU, TURN infrastructure, a signalling protocol, a chat backend, an upload service. That is a lot of moving parts before the first "hello" crosses the wire.

tina4-js takes the mesh route instead. Browsers connect directly to each other. Audio, video, and screen share flow peer to peer. The server does two small jobs: it relays the WebRTC handshake (the SDP and ICE messages) over a signalling WebSocket, and it tells the client where everything lives through one config endpoint. The media never touches your backend.

The `rtc` module wraps all of it, calls and chat and file sharing, behind three signal-driven functions. Bind the signals into an `html` template and the UI stays live on its own.

1. The `rtc` Module

The client pairs with a Tina4 backend's `realtime()` mount. It has three surfaces:

- `rtc.call(room, options?)` - mesh WebRTC (audio, video, screen share) over a signalling WebSocket, using the perfect-negotiation pattern.
- `rtc.chat(channel, options?)` - persistent chat: messages, presence, typing indicators, read receipts, and REST history.
- `rtc.upload(channel, file, options?)` and `rtc.fetchBlob(key, options?)` - file transfer.

Everything reactive is a `Signal<T>` from `tina4js`, so it drops straight into templates. The new-reference rule from the Signals chapter applies: the module replaces the signal arrays wholesale (`peers`, `messages`, `presence`, `typing`). Read them; never `.push()` or mutate them in place.

```
import { rtc } from 'tina4js/rtc';
```

One distinction causes more bugs than any other, so learn it first. `rtc.call()` is async; `rtc.chat()` is sync. The call returns a `Promise`, because it fetches config and prompts for camera and mic, so you `await` it. The chat returns its session synchronously and connects the socket in the background.

```
const call = await rtc.call('standup'); // await
const chat = rtc.chat('general');       // no await
```

2. Bootstrapping: GET /api/rtc/config

The client hardcodes no URLs. It asks the backend where the ICE servers and the WS and HTTP routes live, then wires itself up from the answer. `rtc.call()` does this fetch for you, but you can also call it directly:

```
import { rtcConfig } from 'tina4js/rtc';

const cfg = await rtcConfig();          // GET /api/rtc/config
```

It is also exposed on the object as `rtc.config`. Note that this is the function itself (`rtc.config === rtcConfig`), not a cached config object. Call it:

```
const cfg = await rtc.config();          // same thing
```

A non-2xx response throws `[tina4] rtc config fetch failed: <status>`.

RtcConfig

```
interface RtcConfig {
  backend: string;
  iceServers?: RTCIceServer[]; // STUN/TURN for the peer connections
  signalling?: string;         // call WS path, e.g. "/ws/rtc/{room}"
  chat?: string;               // chat WS path, e.g. "/ws/chat/{channel}"
  messages?: string;           // history HTTP path, e.g. "/api/channels/{id}/messages"
  files?: string;              // upload/download path, e.g. "/api/files"
}
```

Field	Type	Meaning
<code>backend</code>	<code>string</code>	Backend identifier.
<code>iceServers</code>	<code>RTCIceServer[]</code>	STUN/TURN servers for the peer connections.
<code>signalling</code>	<code>string</code>	Call WS path template.
<code>chat</code>	<code>string</code>	Chat WS path template.
<code>messages</code>	<code>string</code>	REST history path template.
<code>files</code>	<code>string</code>	File upload/download path.

A `{room}`, `{channel}`, or `{id}` placeholder in a path template is substituted with the value you pass. If the template has no placeholder, the client appends `/<value>` instead.

Fetch once, reuse everywhere

Every `rtc.call()` fetches config unless you hand it one. If you open several rooms, fetch once and pass it along to skip the repeat network trips:

```
const config = await rtc.config();
const a = await rtc.call('room-a', { config });
const b = await rtc.call('room-b', { config });
```

3. Calls: `rtc.call(room, options?)`

Each participant opens one `RTCPeerConnection` per other participant in the room, a full mesh. The server relays only `hello`, `welcome`, `bye`, `desc`, and `ice` messages between peers; the audio and video are strictly peer to peer. Glare is handled for you with the perfect-negotiation pattern, so simultaneous offers never wedge the connection.

```
const call = await rtc.call('standup'); // camera + mic by default
```

CallOptions

	Type	Default	Behaviour		
g	<code>RtcConfig</code>	-	Pre-fetched config; skips the per-call fetch.		
gUrl	<code>string</code>	<code>'/api/rtc/config'</code>	Where to fetch config if <code>config</code> is not given.		
llingUrl	<code>string</code>	config's <code>signalling</code> or <code>/ws/rtc</code>	Explicit WS base; <code>{room}</code> is filled or appended.		
rvers	<code>RTCIceServer[]</code>	config's <code>iceServers</code> or <code>[]</code>	ICE server override.		
	<code>MediaStreamConstraints</code>	<code>MediaStream</code>	<code>false</code>	<code>{ audio: true, video: true }</code>	Local media. <code>MediaStream</code> used as-is; means receive (no <code>getUserMedia</code> prompt).

CallSession

```
interface CallSession {
  readonly status: Signal<CallStatus>; // 'idle' | 'connecting' | 'connected' | 'closed'
  readonly localStream: Signal<MediaStream | null>; // your camera/mic (null if media: false)
  readonly peers: Signal<RemotePeer[]>; // remote participants + their streams
  readonly screenSharing: Signal<boolean>;
  readonly error: Signal<Error | null>; // last negotiation/ICE error
  readonly id: string; // this peer's id in the room
  shareScreen(): Promise<void>;
  stopScreen(): Promise<void>;
  toggleAudio(on?: boolean): boolean; // returns new enabled state
  toggleVideo(on?: boolean): boolean;
  leave(): void;
}

interface RemotePeer { id: string; stream: MediaStream | null; }
```

The status signal

```
type CallStatus = 'idle' | 'connecting' | 'connected' | 'closed';
```

A live session starts at `'connecting'`, flips to `'connected'` when a peer connection actually reaches `connected`, and lands on `'closed'` after `leave()`. `'idle'` exists in the type but a live session never emits it, so do not wait for it. A lone occupant of a room stays at `'connecting'` until someone else joins.

Binding streams into the DOM

`MediaStream` objects are set on `<video>` elements through the `.srcObject` property. Use an `effect` for the local stream and bind the remote peers in a `mount`:

```
import { rtc } from 'tina4js/rtc';
import { html, mount, effect } from 'tina4js';

const call = await rtc.call('standup');

// local preview
effect(() => {
  const local = call.localStream.value;
  if (local) (document.querySelector('#me') as HTMLVideoElement).srcObject = local;
});

// remote peers - one <video> each, re-rendered as peers come and go
mount('#peers', () => html`
  ${call.peers.value.map(p => html`
    <video autoplay playsinline .srcObject=${p.stream}></video>
  `)}
`);
```

Controls

`toggleAudio()` and `toggleVideo()` enable or disable the local track and return the new state (or `false` if there is no such track). Pass an explicit boolean to force it:

```
call.toggleAudio(); // toggle mic
call.toggleVideo(false); // force camera off
await call.shareScreen(); // getDisplayMedia + swap the outgoing video track
await call.stopScreen(); // restore the camera track
```

`shareScreen()` replaces the outgoing video track on the existing peer connections. Clicking the browser's native "Stop sharing" restores the camera automatically.

Receive-only viewer

Pass `media: false` for a watcher that consumes streams without ever prompting for a camera:

```
const viewer = await rtc.call('standup', { media: false });
```

Leaving

```
call.leave();
```

`leave()` is terminal. It sends `bye`, closes every peer connection, stops your local tracks (the camera light goes off), closes the signalling socket, and sets `status` to `'closed'`. The session cannot be reused. Call `rtc.call()` again to rejoin.

Calls carry no token. The signalling WebSocket opens without auth, and there is no `token` option on `rtc.call()`. Secure a private room on the server (the signalling route), not from this client. Only `chat` and file operations take a `token`.

4. Chat: `rtc.chat(channel, options?)`

A message-and-presence channel over its own WebSocket, backed by a REST history endpoint. It returns synchronously; the socket connects in the background, so watch `session.connected`.

```
const chat = rtc.chat('general', { token: myJwt });
```

The `channel` argument accepts a `string` or a `number`.

ChatOptions

Option	Type	Default	Behaviour
<code>token</code>	<code>string</code>	-	JWT for the secured chat WS and the history calls (bearer on the WS, <code>Authorization</code> header on HTTP).
<code>url</code>	<code>string</code>	<code>'/ws/chat'</code>	Explicit WS base; <code>{channel}</code> is filled or appended.
<code>apiBase</code>	<code>string</code>	<code>''</code> (same origin)	HTTP base for history.
<code>messagesPath</code>	<code>string</code>	<code>'/api/channels/{id}/messages'</code>	History path template; <code>{id}</code> is the channel.
<code>typingTimeout</code>	<code>number</code>	<code>3000</code>	ms a typing indicator lingers before auto-clearing.

ChatSession

```
interface ChatSession {
  readonly status: Signal<SocketStatus>;    // 'connecting' | 'open' | 'closed' | 'reconnecting'
  readonly connected: Signal<boolean>;
  readonly messages: Signal<ChatMessage[]>; // live + prepended history
  readonly presence: Signal<string[]>;      // user ids currently in the channel
  readonly typing: Signal<string[]>;        // user ids currently typing (auto-expire)
  send(body: string, threadId?: number): void;
  sendTyping(): void;
  markRead(): void;
  history(before?: number, limit?: number): Promise<ChatMessage[]>;
  close(): void;
}

interface ChatMessage {
  id?: number;
  channel_id?: number;
  user_id?: string;
  body?: string;
  thread_id?: number | null;
  created_at?: string;
}
```

status and **connected** are the very same signals from the underlying WebSocket client in the WebSocket chapter, so reconnection and backoff come along for free.

A minimal chat panel

```
import { rtc } from 'tina4js/rtc';
import { html, mount } from 'tina4js';

const chat = rtc.chat('general', { token: myJwt });

await chat.history();          // load the last 50, prepended into chat.messages

mount('#log', () => html`
  ${chat.messages.value.map(m => html`<p><b>${m.user_id}</b> ${m.body}</p>`)}
`);

mount('#who', () => html`Online: ${chat.presence.value.join(', ')}`);
mount('#typing', () => html`
  ${chat.typing.value.length ? `${chat.typing.value.join(', ')} typing...` : ''}
`);

const input = document.querySelector('#msg') as HTMLInputElement;
input.addEventListener('input', () => chat.sendTyping());
input.addEventListener('keydown', (e) => {
  if (e.key === 'Enter' && input.value) { chat.send(input.value); input.value = ''; }
});
```

Sending, typing, and read receipts

```
chat.send('hello everyone');    // a top-level message
chat.send('in reply', 42);      // reply into thread_id 42
chat.sendTyping();              // emit a typing signal
chat.markRead();                // emit a read receipt
```

Paging older messages

`history()` both prepends the fetched (older) messages into the `messages` signal and returns the raw rows (newest-first). Render from the signal; do not also append the return value or you will duplicate every message. To load an earlier page, pass the oldest id you already hold as `before`:

```
const oldest = chat.messages.value[0]?.id;
await chat.history(oldest, 50);
```

Teardown

```
chat.close(); // clears typing timers and closes the socket
```

`markRead()` sends a `read` event and the server rebroadcasts it, but `ChatSession` does not expose the raw socket, so there is no built-in signal for incoming read receipts through the returned API.

5. Files: `rtc.upload` and `rtc.fetchBlob`

`rtc.upload()` POSTs a `multipart/form-data` body (`channel_id` plus `file`) to the files path and resolves with the stored record:

```
const res = await rtc.upload('general', fileInput.files![0], { token: myJwt });
// res: UploadResult

interface UploadResult {
  id: number;
  key: string;
  filename: string;
  mime: string;
  size: number;
  url: string; // presigned URL for S3 backends; a route path for local storage
}

interface FileOptions {
  token?: string;
  apiBase?: string; // default '' (same origin)
  filePath?: string; // default '/api/files'
}
```

A secured GET route cannot be reached by a bare `<img src>`, since there is no way to attach a bearer header to it. `rtc.fetchBlob()` fetches the file with the auth header and hands back an object URL you drop straight into `<img src>` or `<a href>`:

```
const src = await rtc.fetchBlob(res.key, { token: myJwt });
imgEl.src = src;
```

For an S3 backend, `UploadResult.url` is already a presigned URL. Pass it to `fetchBlob` (it is used directly) or set it as `src` without a fetch at all.

Object URLs must be revoked. `fetchBlob()` returns a `URL.createObjectURL(...)` handle. It leaks until you release it, so call `URL.revokeObjectURL(src)` when the element unmounts or the image is no longer needed:

```
URL.revokeObjectURL(src); _____
```

6. Real Example: A Call + Chat Room

One room. Video down the side, chat down the middle, a file drop at the bottom. Every piece of live state, peers and messages and presence and connection status, is a signal, so the template renders itself as the world changes. Nothing here polls.

```
import { rtc } from 'tina4js/rtc';
import { html, mount, effect, signal } from 'tina4js';

async function room(name: string, token: string) {
  // Fetch config once, share it with the call.
  const config = await rtc.config();

  const call = await rtc.call(name, { config });
  const chat = rtc.chat(name, { token });
  const draft = signal('');

  await chat.history();

  // local preview
  effect(() => {
    const local = call.localStream.value;
    if (local) (document.querySelector('#me') as HTMLVideoElement).srcObject = local;
  });

  // remote peers
  mount('#peers', () => html`
    ${call.peers.value.map(p => html`
      <video class="peer" autoplay playsinline .srcObject=${p.stream}></video>
    `)}
  `);

  const send = () => {
    const body = draft.value.trim();
    if (body) { chat.send(body); draft.value = ''; }
  };

  const onFile = async (e: Event) => {
    const file = (e.target as HTMLInputElement).files?.[0];
    if (!file) return;
    const res = await rtc.upload(name, file, { token });
    chat.send(`shared a file: ${res.filename}`); // announce it in the channel
  };

  mount('#app', () => html`
    <div class="room">
      <header>
        <span>Call: ${call.status}</span>
        <span>Chat: ${chat.status}</span>
        <span>Online: ${chat.presence.value.length}</span>
      </header>

      <section class="stage">
        <video id="me" class="me" autoplay playsinline muted></video>
        <div id="peers"></div>
      </section>
    </div>
  `);
```

```

<aside class="chat">
  <div class="log">
    ${chat.messages.value.map(m => html`
      <p><b>${m.user_id}</b> ${m.body}</p>
    `)}
  </div>
  <div class="typing">
    ${chat.typing.value.length ? `${chat.typing.value.join(', ')} typing...` : ''}
  </div>
  <form @submit=${(e: Event) => { e.preventDefault(); send(); }}>
    <input
      placeholder="Message..."
      .value=${draft}
      @input=${(e: Event) => {
        draft.value = (e.target as HTMLInputElement).value;
        chat.sendTyping();
      }}
    />
    <button type="submit" ?disabled=${() => !chat.connected.value}>Send</button>
  </form>
  <input type="file" @change=${onFile} />
</aside>

<footer>
  <button @click=${() => call.toggleAudio()}>Mute</button>
  <button @click=${() => call.toggleVideo()}>Camera</button>
  <button @click=${() => call.shareScreen()}>Share screen</button>
  <button @click=${() => { call.leave(); chat.close(); }}>Leave</button>
</footer>
</div>
`);
}

```

```
room('standup', localStorage.getItem('tina4_token') ?? '');
```

The `#me` video is `muted` on purpose. It is your own microphone echoing back, and unmuting it causes feedback. Always `leave()` the call and `close()` the chat together on teardown; both release sockets and timers, and `leave()` frees the camera and mic.

7. Hard Rules

WebRTC has sharp edges. These are the ones the module cannot paper over for you:

- `rtc.call()` is **async**; `rtc.chat()` is **sync**. `await` the call, not the chat.
- `rtc.config` is the **fetch function**, not a **config object**. It is `=== rtcConfig`. Call it: `await rtc.config()`.
- `status` **only reaches 'connected' when a remote peer connects**. A lone occupant stays `'connecting'`, and `'idle'` is never emitted by a live session.
- `leave()` is **terminal**. It stops your tracks, closes everything, and cannot be reused. Call `rtc.call()` again to rejoin.
- **Calls take no token**. Secure a private room on the signalling route server-side.

- **history()** mutates and returns. Render from the `messages` signal; do not also append its return value.
- **fetchBlob()** leaks without cleanup. Revoke the object URL with `URL.revokeObjectURL()` when done.
- **Signals follow the new-reference rule.** Read `peers`, `messages`, `presence`, and `typing`; never mutate them in place.
- **Always tear down.** `leave()` the call and `close()` the chat to release cameras, sockets, and timers.

8. Backend Pairing

This module is the front half of a pair. It talks to a Tina4 backend that mounts the realtime module, which serves `GET /api/rtc/config`, relays the signalling WebSocket, brokers the chat channel, and stores files. Nothing here works until that mount is in place, and the client deliberately learns every path and ICE server from the config endpoint, so the two sides stay in sync without you hardcoding anything.

Wire up the backend in your language of choice. Python, PHP, and Node.js mount it with `realtime()`; Ruby mounts it with `Tina4::Realtime.mount`. Each framework's own Real-time Collaboration chapter walks the server side and the same room this chapter's client connects to.

Summary

<code>import { rtc } from 'tina4js/rtc'</code>		
<code>rtc.config(url?)</code> - GET <code>rtc/config</code>		
<code>rtc.call(room, options?)</code> -		
<code>localStream / call.peers</code> - bind to <code>video.srcObject</code>		
<code>status</code> - 'idle' \	'connecting' \	'connected' \
<code>toggleAudio(on?)</code> / <code>toggleVideo(on?)</code>		
<code>shareScreen()</code> / <code>stopScreen()</code>		
<code>leave()</code> - terminal, releases the		
<code>chat(channel, options?)</code> - sync		
<code>send(body, threadId?)</code>		
<code>sendTyping()</code> / <code>chat.markRead()</code>		
<code>presence</code> / <code>chat.typing</code> signals		
<code>chat.history(before?, limit?)</code> - prepends into <code>messages</code>		
<code>close()</code>		
<code>rtc.upload(channel, file, options?)</code> returns <code>UploadResult</code>		
<code>rtc.fetchBlob(keyOrUrl, options?)</code> returns an object URL (revoke it)		
with a Tina4 <code>realtime()</code> mount		

You started with a call button and a fear of infrastructure. You end with calls, chat, presence, typing, history, screen share, and permissioned file transfer, every live value a signal that renders itself, and not one media server to run. The browser already knew how to talk peer to peer. tina4-js just handed it the room.